ThreatLake PFA

Line-by-line Walkthrough (src/threatlake/)

Etude approfondie mte3 el codebase – Franco-Arabe + English

August 23, 2026

Abstract

Hedhi el document ya3mel walkthrough kamel, near line-by-line, mte3 kol file f `src/threatlake/` mte3 ThreatLake PFA – la version PFA (cowrie bark, batch bark, 2 gold tables, 2 detectors). Kol function 3andha subsection mte3ha, kol ligne walla block muhim mfasser: *chnowa ya3mel* w *3lech kteb hakka w machi b tari9a o5ra as-hal*. Fama zouz box: **Analogy** (tashbih basit) w **Soual (Jury)** (soual li momken ye5arjek bih superviseur, ma3 réponse). L'objectif: tefham el code kifma kif rak katbou nta, mouch just summary sat-hi.

Contents

Chapter 1

Muqaddima – kifeh el pipeline ye5dem

ThreatLake PFA houwa version sghira mte3 ThreatLake AI: pipeline mte3 detection mte3 honeypot attacks, kolha local, batch bark (ma famach streaming), source wahda bark (cowrie – SSH/Telnet honeypot). El fikra el kbira: data tji raw JSON (NDJSON lines mte3 cowrie), tetba3 chain mte3 4 couches, ki lakehouse medallion architecture (bronze / silver / gold):

- **Bronze** (`ingestion/`): ya9ra el raw NDJSON lines mel landing zone, yparse-hom against schema explicit (`config/schema/cowrie.py`), y-stamp-hom b metadata (ingest_date, hash...), w y-save-hom f Delta table. Li ma-yparse-ch (JSON invalid) yemchi l "quarantine" – ma yentliwch.
- **Silver** (`transform/silver/`): ya5ou el bronze rows w yamapp-hom l schema unifié wahed (`SILVER_EVENT_SCHEMA`) – kol eventid mte3 cowrie (`cowrie.login.failed`, `cowrie.command.input`...) yetba3lou category w severity.
- **Gold** (`transform/gold/`): aggregations fi 2 tables bark – `attacker_profiles` (wahda l kol IP) w `attack_timeline` (par heure).
- **ML** (`ml/`): zouz detectors ye5edmou b3adhom – IsolationForest (unsupervised, trained 3la silver features) w rule fixe (port scan). Result-hom ye-append-iw f `ml_scores`.
- **Copilot** (`copilot/`): natural language question → SQL (Gemini) → guardrail (sqlglot AST check) → execution 3la gold tables bark.
- **API** (`api/`): FastAPI, read-only, 3andha 3 endpoints bark (`/alerts`, `/attacker_profiles`, `/copilot/query`).

Kol chapter jeya besh ta5ou file wahed walla package, w tefasser kol function line-by-line: *chnowa ta3mel* w *3lech makteba hakka*.

Chapter 2

common/ – el infrastructure el mouchtaraka

Hedha el package fih 4 files: `config.py` (settings), `paths.py` (kol location fi lakehouse), `spark.py` (factory mte3 SparkSession), w `fs.py` (file operations). Kol module 3la ba3dhom, w kol package l fou9 (ingestion, transform, ml, api...) yaimport minhom.

2.1 common/config.py

Hedha el file houwa single source of truth mte3 kol configuration – YAML file wahda (`config/local.yaml`) + environment variables. Ye5dem b `pydantic-settings`, li ya3mel validation automatique 3la kol value.

2.1.1 Precedence – mnin tji kol value

Docstring mte3 el file yban clair:

```
1. explicit keyword arguments to Settings
2. process environment (THREATLAKE_ prefix, __ nests)
3. config/local.yaml
4. field defaults
```

- Ye3ni: kif t3ammer `Settings(storage=...)` yeddik, ki testa3mel environment variable ki `THREATLAKE_SPARK__APP_NAME=x` yeddik (ahsen mel YAML), w el YAML file ahsen mel default mte3 field.
- `__` (double underscore) f nom mte3 env var ya3mel "nesting": `THREATLAKE_STORAGE__ROOT` yrouh l `settings.storage.root`. Hedha houwa `env_nested_delimiter="__"` f `model_config`.

Soual (Jury)

3lech ma-kteb-tech el config kif dictionary Python 3adi, walla just os.environ.get() f kol place?
Réponse: Pydantic ya3mel validation *automatique* 3la kol field (types, required/optional, custom validators ki `_not_blank`). Ki tketb `THREATLAKE_SPARK__SHUFFLE_PARTITIONS=abc` (mouch number), Pydantic ye5rejlek erreur clair 3nd el startup, mouch crash randomly f middle mte3 pipeline run ba3d se3at. W `frozen=True` y7ell el Settings object immutable – ken bech tbeddel-ha lezem tal9a bug, mouch typo li y-mutate-ha silently f place o5ra fi code.

2.1.2 Layer w LAYERS – el 3 couches

```
Layer = Literal["bronze", "silver", "gold"]
LAYERS: tuple[Layer, ...] = ("bronze", "silver", "gold")
```

- `Literal["bronze", "silver", "gold"]` houwa type hint – ye9oul l mypy/type-checker "hedha el variable ykoun wahed mel 3 strings hedhoula bark, machi ay string". Ken tketb `layer="platinum"` f function li ta5ou `Layer`, type-checker yban erreur *9bal* ma trun el code.
- `LAYERS` houwa tuple runtime mte3 nafs el 3 values – `Literal` ye5dem 3la static analysis bark (compile-time), lezem tuple séparé ken 7bit t-loop 3lihom walla t-validate runtime (`if layer not in LAYERS`).

2.1.3 ConfigError

```
class ConfigError(RuntimeError):
    """Raised when configuration is missing, unreadable, or inconsistent."""
```

Exception class 3adi, custom, bech kol erreur mte3 config yetsayed b type spécifique (`ConfigError`) mouch generic `Exception` – yfahhem el caller ken el erreur mel config walla min place o5ra.

2.1.4 StorageSettings

```
class StorageSettings(BaseModel):
    """Physical locations of data - plain local filesystem paths."""

    model_config = {"extra": "forbid"}

    root: str = Field(description="Root of the lakehouse; bronze/silver/gold hang off this.")
    quarantine: str = Field(description="Root for records rejected by the schema gate.")
    landing: str = Field(description="Directory the cowrie log source drops raw NDJSON into.")

    @field_validator("root", "quarantine", "landing")
    @classmethod
    def _not_blank(cls, value: str) -> str:
        if not value.strip():
            raise ValueError("storage locations must be non-empty")
        return value.rstrip("/")
```

- `model_config = {"extra": "forbid"}` – ken el YAML file 3andha key ma-mou9out-cha f el model (typo ki `roott` instead `root`), Pydantic ye5rej erreur *9bal* ma yestamer, mouch yetjaheluh silently. Hedhi tafsila sghira barcha muhimma – bidoun-ha, typo fi config yensa w tebda t-debug 3la wa9t twil bech tel9a el erreur mabch minha.
- `root` / `quarantine` / `landing`: 3 paths, kol wahed 3andou `description` – mouch just documentation, Pydantic ye5dem-ha f auto-generated schemas (JSON schema, error messages) ken 7bit.
- `@field_validator("root", "quarantine", "landing")`: nafs el validator ye5dem 3la 3 fields b marra (decorator ya59bel tuple mte3 names).
- `_not_blank`: ken el string fargha walla just whitespace, ye5rej `ValueError` (Pydantic ye-catch-ha w y-wrap-ha f validation error mte3ou). `value.rstrip("/")` yne99i trailing slash – hedhi mohemma bech `paths.py` ma-y-concat-ch `"root//bronze"` (double slash) ken el user 7at `root: ./data/lakehouse/` (b slash f a5er el value).

Analogy

`extra="forbid"` kifha bouncer 3nd bab – ken jeya field ma-mektoubch f guest list (el model), ma-ydakhal-cha, ye9tol el process kamel bech ta3rf fama typo, mouch t5alliha tdakhal w tban silently f runtime.

2.1.5 TableSettings

```python
class TableSettings(BaseModel):
    """Logical table key -> physical Delta table name, grouped by medallion layer."""

    model_config = {"extra": "forbid"}

    bronze: dict[str, str] = Field(default_factory=dict)
    silver: dict[str, str] = Field(default_factory=dict)
    gold: dict[str, str] = Field(default_factory=dict)

    def names(self, layer: Layer) -> dict[str, str]:
        """Return the logical -> physical table map for `layer`."""
        mapping: dict[str, str] = getattr(self, layer)
        return dict(mapping)

    def resolve(self, layer: Layer, table: str) -> str:
        """Resolve a logical table key to its physical name.

        Unregistered keys pass through unchanged so ad-hoc/test tables work
        without editing config/local.yaml.
        """
        return self.names(layer).get(table, table)
```

- El fikra: code ye-reference table b "logical key" (`"events"`, `"attacker_profiles"`) machi b physical name direct. El mapping (`bronze: {cowrie: bronze_cowrie}` f el YAML) ye5alli tbeddel smiya physical mte3 table (production rename, walla naming convention jdida) bidoun ma tmess el code.
- `names(self, layer)`: `getattr(self, layer)` – dynamic attribute access, `layer` houwa string ("bronze"/"silver"/"gold") w `getattr` ye5ou el dict mte3ha (`self.bronze` walla `self.silver`...) bidoun `if/elif` chain manuel.
- `dict(mapping)`: copy defensive – caller ma-y9dr-ch y-mutate el dict el original mte3 Settings (li houwa frozen barcha, tab3an, walima el dict inside momken y-mutate-ha ken ma-3malnach copy).
- `resolve`: `.get(table, table)` – ken el logical key *machi* f el mapping, yrajja3 nafs el key kifha (passthrough). Hedha ye5alli ad-hoc tables (tests, walla table jdida experimental) ye5edmou bidoun ma tzid entry f YAML l kol table jdida.

2.1.6 SparkSettings, CopilotSettings, MLSettings

```python
class SparkSettings(BaseModel):
    model_config = {"extra": "forbid"}

    app_name: str = "threatlake-pfa"
    master: str | None = Field(default="local[2]", description="Local Spark master.")
    conf: dict[str, str] = Field(default_factory=dict)
    shuffle_partitions: int = Field(default=8, ge=1)
    log_level: Literal[...] = "WARN"
```

- `master="local[2]"`: Spark ye5dem 3la 2 cores bark, machi `local[*]` (kol cores). 3la laptop, ken el API server w el pipeline run ye5edmou f nafs el wa9t, kol wahed y7eb kol el cores – `local[2]` y5alli headroom l OS w l processes o5rin.
- `shuffle_partitions=8`: default mte3 Spark houwa 200 – kbir barcha l data sghira (test corpus mte3na 3andou 177 rows). 8 partitions yban a9rab l size el data el 7a9i9i, ya3mel less overhead.
- `ge=1` (`Field(..., ge=1)`): "greater or equal 1" – constraint numérique, Pydantic ye-reject 0 walla negative automatiquement.

```python
class CopilotSettings(BaseModel):
    """SOC Copilot's NL-to-SQL provider.
```

```
    Deliberately NOT a place for the API key: it belongs in that provider's
    own conventional env var (GEMINI_API_KEY), not a THREATLAKE_-prefixed
    setting or a committed YAML file.
    """
    model_config = {"extra": "forbid"}

    provider: Literal["gemini"] = "gemini"
    model: str = "gemini-flash-latest"
    max_output_tokens: int = Field(default=2048, ge=64)
```

Soual (Jury)

3lech el API key (GEMINI_API_KEY) mahouch f hedha el Settings class, w howa configuration barra kif kolchay?
Réponse (mel docstring): secret keys ma-lezem-hech ykounou f YAML file (momken yetcommitiw b accident f git) walla f Settings object (li kif tal3ha Settings f logs walla error messages, el secret momken y-leak). El convention houwa: kol provider 3andou env var mte3ou el "conventional" (`GEMINI_API_KEY`), w `copilot/text_to_sql.py` ya9raha direct mel `os.environ` *f wa9t el call*, machi mel Settings.

```
class MLSettings(BaseModel):
    """Where the trained anomaly detector lives on disk."""
    model_config = {"extra": "forbid"}
    model_path: str = Field(description="Local path to the joblib-persisted IsolationForest.")
```

Field wahed bark: `model_path`. Kif nchoufou f `ml/train_anomaly.py` w `ml/score_events.py`, hedha el path houwa fein el trained model (`.joblib` file) ye-save w ye-load mennou.

2.1.7 `_find_config_dir()` – kifeh yal9a `config/`

```
def _find_config_dir() -> Path:
    override = os.environ.get(CONFIG_DIR_VAR)
    if override:
        path = Path(override).expanduser()
        if not path.is_dir():
            raise ConfigError(f"{CONFIG_DIR_VAR}={override!r} is not a directory")
        return path

    candidates = [Path.cwd(), *Path.cwd().parents, *Path(__file__).resolve().parents]
    for base in candidates:
        candidate = base / "config"
        if candidate.is_dir():
            return candidate

    raise ConfigError(
        f"Could not locate a 'config' directory. Run from the repo root or set {CONFIG_DIR_VAR}."
    )
```

- `THREATLAKE_CONFIG_DIR` (el `CONFIG_DIR_VAR`) win 3andha priority el kbira – ken el user 7ata explicit, ye5dem biha bark, w ken mouhich directory 7a9i9i ye5rej erreur direct (ma-y3addich silently).
- Ken ma-fama-ch override: `candidates` tban list mel current directory (`Path.cwd()`) + kol parent mte3ha (`.parents`) + kol parent mel file el actuel (`__file__`, ye3ni win el `config.py` rasha mrakez fi `src/threatlake/common/`).
- El loop y-check kol wahed mel candidates, ye7ell `config/` (subdirectory) fihom, w yrajja3 l wa7ad li mawjouda. Hedhi el logique te5alli el script ye5dem sawa min `cwd` el repo root, sawa min subdirectory ba3ida, sawa min package installé (editable install) – bidoun ma tzid hardcoded absolute path.

- Ken *7atta wa7ad* ma-tal9ach, `ConfigError` b message clair (win yfattech w kifeh yfixiha).

Analogy

Hedhi el logique kif `.git` discovery mte3 git nafsou – git ye-walk-up mel current directory l fou9 7atta yal9a `.git/` folder. Hna nafs el principe, ma3 fallback l parents mte3 `__file__` zeda ken `cwd` ma-3andha-ch `config/` qrib.

2.1.8 `YamlSettingsSource` – kifeh el YAML tji Settings source

```python
class YamlSettingsSource(PydanticBaseSettingsSource):
    """Feed config/local.yaml into the settings model."""

    def __call__(self) -> dict[str, Any]:
        path = _find_config_dir() / "local.yaml"
        if not path.is_file():
            raise ConfigError(f"Config file not found: {path}")

        with path.open("rb") as handle:
            loaded = yaml.safe_load(handle) or {}
        if not isinstance(loaded, dict):
            raise ConfigError(f"{path} must contain a YAML mapping at the top level")

        loaded["config_file"] = str(path)
        return loaded
```

- Hedhi class custom ta3mel implement mte3 `PydanticBaseSettingsSource` (interface mte3 pydantic-settings) – ye3ni "source" jdid mte3 configuration, ki el environment variables walla init kwargs, ken pydantic yal9a-ha 3andha `__call__` li yrajja3 dict.
- `yaml.safe_load` machi `yaml.load` – `safe_load` ma-ye5allich el YAML file ye5alle3 arbitrary Python objects (security: YAML momken tehtwi `!!python/object` tags li yexecutiw code ken testa3mel `load` el 3adi). `safe_load` ya9ra data structures basiques bark (dict, list, str, int...).
- `or {}`: ken el YAML file fargh (`yaml.safe_load` yrajja3 `None`), `loaded` ykoun dict fargh mouch `None` – ye5alli el code ba3dha (`isinstance(loaded, dict)`) ye5dem bidoun crash.
- `loaded["config_file"] = str(path)`: yzid field ekstra fi el dict el rajja3, bech `Settings.config_file` ykoun m3ammar automatiquement b win jat el config – useful l debugging (ken el settings ghalt, ta3raf mnin jaw).

Soual (Jury)

`get_field_value` ye5rej `NotImplementedError` – 3lech mouch just implémenter-ha correctly?
Réponse (mel comment f el code): hedhi method abstract mte3 el ABC (`PydanticBaseSettingsSource`) – pydantic-settings ye7tajha texist bech el class ma-tbe9ach abstract (Python ma-y5alli-cha t-instantiate class 3andha abstract method ma-mimplementéech), walakin `__call__` houwa el method el 7a9i9i li pydantic ya3ayet-lou – `get_field_value` ma-yet3ayet-lou 9att f el use-case hedha (source-level, mouch field-level).

2.1.9 `Settings` – el class el kbira

```python
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix=ENV_PREFIX,
        env_nested_delimiter="__",
        extra="forbid",
        frozen=True,
    )

```

```
    env: str = "local"
    config_file: str | None = None

    storage: StorageSettings
    tables: TableSettings
    spark: SparkSettings = Field(default_factory=SparkSettings)
    copilot: CopilotSettings = Field(default_factory=CopilotSettings)
    ml: MLSettings

    @classmethod
    def settings_customise_sources(cls, settings_cls, init_settings, env_settings,
                                   dotenv_settings, file_secret_settings):
        return (init_settings, env_settings, dotenv_settings,
                YamlSettingsSource(settings_cls), file_secret_settings)
```

- `frozen=True`: el Settings object immutable ba3d ma yetbena – `settings.spark.app_name = "x"` ye5rej erreur. Hedhi te-guarantee eno kol part mel code li 3andha reference l nafs Settings object ta5ou nafs el values, ma-fama-ch race condition walla mutation surprise.
- `storage`, `tables`, `ml`: 3andhom-ch default – required. Ken el YAML ma-fihech-hom, Settings() ye5rej validation error 3nd el startup, machi crash randomly wa9t ma el code y7awel ye5dem-hom.
- `spark`, `copilot`: 3andhom `default_factory` – optional, ken mouch f el YAML, ye5dou el defaults mte3 el sub-model.
- `settings_customise_sources`: hedhi el method li t7att order mte3 el 4 sources (kifma mkeltou fi `config.py`'s docstring): init kwargs awel (highest priority), ba3d env vars, ba3d dotenv (`.env` file – pydantic-settings 3andha support native lha walaw ma testa3malha-ch el project f manage.sh, testa3ml-ha directly hna kordre fallback), ba3d el YAML custom source, w akhiran file secrets (Docker secrets style, machi mousta3mla hna walakin lezem tkoun present f tuple bech el ABC signature ykoun sa7i7).

2.1.10 get_settings() w reset_settings()

```
@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """Return the process-wide settings, loading them on first use."""
    return Settings()


def reset_settings() -> None:
    """Drop the cached settings. Used by tests that swap environments."""
    get_settings.cache_clear()
```

- `@lru_cache(maxsize=1)` 3la function bidoun arguments: el marra el oula ma ye3ayet-lou 7ad, Python ya7seb `Settings()` (ya9ra YAML, yparse env vars...) w y5azzen result. Kol call jeya ba3d, yrajja3 *nafs* el object (identity, machi copy) – bidoun ma yre-parse el YAML file kol marra.
- Hedhi optimization muhimma: `Settings()` ta3mel I/O (ya9ra file mel disk), w el project ya3ayet `get_settings()` f barcha places (kol `silver_path()`, `gold_path()`...). Bidoul el cache, kol call ye3awed I/O – ma3 el cache, I/O wa7da bark f kol process.
- `reset_settings()`: ye-clear el cache – mousta3mla f tests, win kol test 7ab settings jdida (b environment variables mbeddla, ki `THREATLAKE_STORAGE__ROOT` l tmp directory). Bidoun `reset`, el test el theni ye5ou nafs el cached Settings mel test el oul.

Analogy

`@lru_cache` 3la function bidoun args kifha "singleton" pattern classic f OOP (instance wa7da mel class fi kol el application), walakin b decorator wa7ad bark, bidoun class 5assa. W `reset_settings()` kifha "reset button" – testa3mlha ken 7bit tbeddel el configuration f runtime (tests bark, el production code ma-ye3ayet-lha-ch abadan).

2.2 common/paths.py

Hedha el file houwa "single source of truth" mte3 kol location fi lakehouse. Docstring ye9oul kol chay clair:

```
No other module builds a path by string concatenation - everything goes
through these helpers. Trimmed from ThreatLake AI's version: no
``checkpoint_path`` (no streaming here) and no ``cache_path``/``table_fqn``
(no enrichment cache, no Unity Catalog metastore - PFA is local-path-only).
```

- Hedhi regle muhimma: *kol* path fi el codebase ye-construct mel functions hedhouma bark, 7atta test wa7ad ma-ye5dem-ch b `f"{root}/bronze/{table}"` manuel. Ken lezem tbeddel structure mte3 lakehouse (nesta3malou naming convention jdid mathalan) tbeddel f file wa7ad bark.

2.2.1 PathError

```python
class PathError(ValueError):
    """Raised for table or checkpoint names that cannot form a safe path."""
```

Custom exception, subclass mte3 `ValueError` (machi `RuntimeError` kif `ConfigError`) – 3la 5atr el erreur hedhi te-concerne *value* ghalta (esm table fih "/" walla "..") mouch configuration missing.

2.2.2 _normalise_root

```python
def _normalise_root(location: str) -> str:
    """Return an absolute, trailing-slash-free root."""
    return str(Path(location).expanduser().resolve())
```

- `.expanduser()`: ken el path fih `~` (home directory shorthand ki `~/data`), ye5dem-ha l absolute home path.
- `.resolve()`: ye5alli el path absolute, w ye5alli symlinks w `..` (parent references) yetsetliw – `./data/../data/x` w `/abs/data/x` ykounou nafs el result akhir.
- 3lech muhim ye-normalise: ken zouz configs mte3 test 3andhom `root` mektouba b tari9tin mo5telfin (relative vs absolute) mel nafs directory el 7a9i9i, el pipeline momken ye5el9 zouz Delta tables f zouz paths physically mo5telfin ken el comparison textuelle bark (bidoun normalisation) – bug s3ib ye-debug.

2.2.3 _validate_segment

```python
def _validate_segment(segment: str, kind: str) -> str:
    """Reject empty names and traversal, which would escape the storage root."""
    cleaned = segment.strip()
    if not cleaned:
        raise PathError(f"{kind} name must not be empty")
    if cleaned != segment:
        raise PathError(f"{kind} name {segment!r} has leading or trailing whitespace")
    if "/" in cleaned or "\\" in cleaned or cleaned in {".", ".."}:
        raise PathError(f"{kind} name {segment!r} must be a single path segment")
```

```
10      return cleaned
```

- `cleaned = segment.strip()`: yne99i whitespace mel bidaya w el a5er, ba3d ye-compare `cleaned != segment` – ken el original 3andou whitespace (`" cowrie"` mathalan), el function te-reject-ha b erreur clair, machi t9bel-ha silently b whitespace jowa esm el table.
- `"/" in cleaned or "\\" in cleaned or cleaned in {".", ".."}`: hedhi el check el security el muhimma – **path traversal prevention**. Ken `table` houwa user input (mathalan mel API walla mel copilot), w ma-3andkech el check hedha, wa7ad momken y3addi `table="../../etc/passwd"` w el path el rajja3 ye5rej barra mel `storage_root` el mou9arrar.
- `kind` (parameter): ye5alli nafs el function te5dem l "table" walla "source" walla ay esm segment o5or, w el error message ykoun specifique (`"table name must not be empty"` vs `"source name must not be empty"`).

Soual (Jury)

3lech testa3mel manual string checks (`"/" in cleaned`) mouch regex wahda walla `pathlib` validation built-in?
Réponse: el check basic (whitespace, "/", "\\", "." / "..") ye-cover kol el cas realistes mte3 "logical key" jeya mel YAML config walla mel API request – machi arbitrary filesystem paths. Regex momken ykoun over-engineered l use-case sghir hakka, w explicit checks (kifma hakka) as-hal ye9raw w ye-debug mel regex pattern kif ma yetkassar.

2.2.4 `_join`

```
1 def _join(root: str, *segments: str) -> str:
2     return "/".join([root.rstrip("/"), *segments])
```

Simple join b `"/"` – `root.rstrip("/")` yne99i trailing slash mel root 9bal el join, bech ma-tsir-ch `"root//segment"` (double slash) ken el root deja fih slash f a5er.

2.2.5 `storage_root`, `layer_path`, w el helpers el mbniya 3liha

```
1  def storage_root(settings: Settings | None = None) -> str:
2      """Absolute root of the lakehouse."""
3      settings = settings or get_settings()
4      return _normalise_root(settings.storage.root)
5
6
7  def layer_path(layer: Layer, table: str, settings: Settings | None = None) -> str:
8      """Location of a Delta table in ``layer``.
9
10     ``table`` is a *logical* key; the physical name comes from
11     ``tables.<layer>`` in ``config/local.yaml``, falling back to the key itself.
12     """
13     if layer not in LAYERS:
14         raise PathError(f"Unknown layer {layer!r}. Expected one of: {', '.join(LAYERS)}")
15     settings = settings or get_settings()
16     name = _validate_segment(settings.tables.resolve(layer, table), "table")
17     return _join(storage_root(settings), layer, name)
```

- `settings: Settings | None = None` – pattern ye-repeat rou7ou f kol function fi hedha el file: ken el caller ma-3atach settings (test mathalan li 7eb settings custom), `settings or get_settings()` ye5ou el cached global wa7ed. Hedhi te5alli el functions testable (dependency injection) w practical fi production (default automatique) fi nafs el wa9t.
- `layer_path`: ye-valid el `layer` l'awel (raise ken mahouch f `LAYERS`), ba3d ye-resolve el logical `table` key l physical name b `settings.tables.resolve(layer, table)` (mel `TableSettings`

li chefnaha f `config.py`), ba3d ye-validate el physical name (`_validate_segment`) 9bal ma y-join-ha m3a el root w el layer.

- Order el operations muhim: resolve el logical-to-physical *9bal* el validation, 3la 5atr el physical name (jeya mel YAML, trusted) houwa eli lezem ykoun path segment valide – el logical key (`"events"`) momken ykoun ay chay.

```python
def bronze_path(source_type: str, settings: Settings | None = None) -> str:
    """Location of a source type's own bronze table (only ``cowrie`` here)."""
    return layer_path("bronze", source_type, settings)


def silver_path(table: str, settings: Settings | None = None) -> str:
    """Location of the cleaned, conformed silver events table."""
    return layer_path("silver", table, settings)


def gold_path(table: str, settings: Settings | None = None) -> str:
    """Location of an aggregated, serving-ready gold table."""
    return layer_path("gold", table, settings)
```

Tlata thin wrappers 3la `layer_path` – kol wa7ad ye-hardcode el layer name mte3ou. Hedhi machi duplication mouzou3iya: el fikra houwa API clair (`bronze_path("cowrie")` as-hal ye9ra mel `layer_path("bronze", "cowrie")`), w ken 7bit tzid layer jdid (mathalan "platinum") tzid function wa7da bark hna, el logique el kbira ma-tetbeddel-ch.

```python
def landing_path(source: str, settings: Settings | None = None) -> str:
    """Directory the cowrie log source drops raw NDJSON files into, pre-ingest."""
    settings = settings or get_settings()
    name = _validate_segment(source, "source")
    return _join(_normalise_root(settings.storage.landing), name)


def quarantine_path(table: str, settings: Settings | None = None) -> str:
    """Where records rejected by the schema gate are parked."""
    settings = settings or get_settings()
    name = _validate_segment(table, "table")
    return _join(_normalise_root(settings.storage.quarantine), name)
```

`landing_path` w `quarantine_path` *ma-ye3addiwch* b `layer_path` – 3la 5atr mahomch "layers" mte3 medallion architecture (bronze/silver/gold), homa 2 zones separate: landing (win jeya raw data 9bal ma ta5ol el pipeline) w quarantine (win temchi data li rejected mel schema gate). Kol wa7ad 3andou root mte3ou fi `StorageSettings` (`landing`, `quarantine`), machi `root`.

2.3 common/fs.py

Hedha el file ya3mel operations basiques (list, move) 3la files, walakin *machi* b `os`/`shutil` el Python normal – ye5dem b `org.apache.hadoop.fs.FileSystem` (el classe Java li Spark yesta3melha tahtou).

Soual (Jury)

3lech testa3mel Hadoop FileSystem API (Java, 3ber py4j) mouch just `os.rename()` w `os.listdir()` el basiques mte3 Python?
Réponse (mel docstring): storage roots momken ykounou local paths (PFA, hedhi el configuration el mous-ta3mla hna) walla cloud URIs (`abfss://`, `dbfs:/` f ThreatLake AI el kbir 3la Databricks). `os.rename` ye5dem 3la local filesystem bark – ma-y3arraf-ch `abfss://`. El Hadoop `FileSystem` abstraction ye5dem nafs el kif 3la el zouz – `fs.rename(src, dst)` ye5dem sawa 3la `file://` sawa 3la `abfss://` bidoun ma tbeddel el code. Hedha ye5alli `bronze_writer.py` (li testa3mel `move`) ye5dem bidoun tbeddil ken el project yetla3 mel lap-

top l Databricks cluster.

Analogy

Hadoop FileSystem API kifha adapter universel – ki adapter mte3 prise kahraba li ye5dem fi kol pays (US, Europe, UK), `FileSystem` ye5dem 3la kol "storage backend" (local disk, Azure, S3, HDFS) b nafs el interface. Bidoulha, kol module li 7eb yemchi/ye-list files lezem ykoun 3andou code separate l kol backend.

2.3.1 _hadoop_path w _filesystem_for

```python
def _hadoop_path(spark: SparkSession, path: str) -> JavaObject:
    # _jvm/_jsc are only None before a SparkContext exists; an active `spark`
    # session (this function's precondition) always has both populated.
    jvm = spark.sparkContext._jvm
    assert jvm is not None # noqa: S101 - type narrowing, not a runtime guard
    result: JavaObject = jvm.org.apache.hadoop.fs.Path(path)
    return result


def _filesystem_for(spark: SparkSession, hadoop_path: JavaObject) -> JavaObject:
    jsc = spark.sparkContext._jsc
    assert jsc is not None # noqa: S101 - type narrowing, not a runtime guard
    hadoop_conf = jsc.hadoopConfiguration()
    result: JavaObject = hadoop_path.getFileSystem(hadoop_conf)
    return result
```

- `spark.sparkContext._jvm`: hedha el "gateway" li py4j ye5alli Python code ye3ayet Java classes direct (`jvm.org.apache.hadoop...` ye5dem kif import Java class mel Python).
- `assert jvm is not None`: hedha *machi* runtime check (comment ye9oul "type narrowing, not a runtime guard") – houwa hna bark bech mypy/type-checker yfahhem eli `jvm` mouch `None` ba3d el assert, w el return type (`JavaObject` mouch `JavaObject | None`) ykoun sa7i7 statically. `_jvm` ma-ykoun-ch `None` illa 9bal ma yetbena `SparkContext` – w el precondition mte3 hedhi el function (`spark` active session) te-guarantee ma yasal-ch l hedha el cas.
- `_filesystem_for`: ya5ou el `hadoop_conf` (configuration mte3 Hadoop, jeyya mel `SparkContext`) w y-resolve minha el `FileSystem` object el correct l el path el m3ayen (local, abfss...) – `getFileSystem` ye-decide automatiquement el implementation correcte b nazar l el scheme mte3 el path (`file://` vs `abfss://`).

2.3.2 list_files

```python
def list_files(spark: SparkSession, directory: str) -> list[str]:
    """Non-recursive list of file (not subdirectory) paths directly under ``directory``.

    Returns ``[]`` if the directory does not exist yet - an empty/absent
    landing directory is a normal state, not an error.
    """
    path = _hadoop_path(spark, directory)
    fs = _filesystem_for(spark, path)
    if not fs.exists(path):
        return []
    return [
        status.getPath().toString()
        for status in fs.listStatus(path)
        if not status.isDirectory() and not status.getPath().getName().startswith((".", "_"))
    ]
```

- `if not fs.exists(path): return []` – 9rar design muhim: directory li ma-mawjoud-ch (landing zone 9bal ma yjini 7atta file wa7ad) houwa *normal state* mouch erreur. Ken el

function te5rej exception, el pipeline el oula run (landing fargh 3ad) ye-crash bidoun 7atta sabab 7a9i9i.

- List comprehension b 2 filters: `not status.isDirectory()` (bark files, machi subdirectories – el function *non-recursive* kifma el docstring te9oul), w `not ...startswith((".", "_"))` yestab3ed hidden files (`.DS_Store` 3la macOS mathalan) w Spark/Hadoop internal marker files (li ye-start b `_`, ki `_SUCCESS`).

2.3.3 move

```python
def move(spark: SparkSession, source: str, destination: str) -> None:
    """Move a single file, creating destination parent directories as needed.

    A move, never a copy-and-delete-source performed from Python: this calls
    the filesystem's own atomic-where-possible rename, which is what makes it
    safe to use as an audit trail (the file never exists in two places at
    once from the caller's perspective, and nothing is ever deleted).
    """
    src_path = _hadoop_path(spark, source)
    fs = _filesystem_for(spark, src_path)
    dest_path = _hadoop_path(spark, destination)
    fs.mkdirs(dest_path.getParent())
    if not fs.rename(src_path, dest_path):
        raise OSError(f"Failed to move {source!r} -> {destination!r}")
```

- `fs.mkdirs(dest_path.getParent())`: ye5ala9 el parent directory mte3 el destination ken ma-mawjoud-ch (mathalan `processed/2026-08-22/`) – bidoulha `fs.rename` ye-fail ken el parent ma-mawjoud-ch.
- `fs.rename` machi copy-then-delete: `rename` houwa atomic (walla "atomic-where-possible" 3la el backend) 3nd el filesystem nafsou – el file ma-yban-ch f zouz places f nafs el wa9t, w ma-fama-ch risque data loss ken el process ye-crash f nos el operation (copy na9sa + delete temmet = data lost; rename atomic ma-3andou-ch hedha el risque).
- `if not fs.rename(...): raise OSError`: `fs.rename` yrajja3 boolean (machi exception 3la failure, kif barcha Java APIs el classic). El code hna ye-convert-ha l exception Python-style (`OSError`) bech el caller (`bronze_writer.py`) ma-y-forget-ch ye-check el return value.

Soual (Jury)

3lech "move" (mouch "copy") houwa el operation el mous-ta3mla l ingestion, w 3lech el docstring te-emphasize "audit trail"?
Réponse: `bronze_writer.py` ya9ra raw files mel landing zone, yparse-hom, w ba3d ye-move-hom l `processed/` (machi ye-delete-hom). Hedha ye5alli el system "at-least-once": ken el process ye-crash ba3d el parse walakin 9bal el move, el file yeb9a f landing w el run el jayya ye-reprocess-ha (dedup ye-sir b content-hash downstream, mouch b "did we see this file before"). W `move` (rename atomic) ye-guarantee eli el file ma-yeb9a-ch "nos processed" – sawa houwa fi landing (lazem processing) sawa fi processed (temmet), 7atta wa7ad mel 2.

2.4 common/spark.py – el partie el akthar critique

Hedha el file ya3mel factory l `SparkSession`, w fih el logique el akthar delicate f kol el codebase: verification eli el session ye5dem 7a9i9i f UTC, machi just "3andou config UTC". El docstring mte3 el file kamel ye-explique 3lech:

```
Every session is verified to actually run in UTC before ``get_spark()``
returns it. This exists because ``spark.sql.session.timeZone=UTC`` alone
does not guarantee it: that setting governs Spark SQL's own computation,
but PySpark materializes a collected TimestampType into a Python
```

```
``datetime`` via the JVM's OS-level default timezone instead, which can
silently disagree. A session where that gap holds would shift every
collected timestamp by the host's UTC offset without raising anything -
this check exists so that happens loudly, at session start, instead of
silently corrupting every ``event_time`` this pipeline ever reads back.
```

Analogy

5alli el fikra t-bin b exemple: 9oul el host machine (laptop mte3ek) rasha m3ammra b timezone `Africa/Tunis` (UTC+1). `spark.sql.session.timeZone = "UTC"` ye9oul l Spark SQL: "wa9t te-calculate walla te-format timestamp *jowa* SQL queries, esta3mel UTC". Walakin ken t-collect-i (`.collect()`) timestamp l Python `datetime` object, hedhi el conversion *ma te-3addich* b Spark SQL – ta3addi b el JVM's default timezone (li ye5ou-ha mel OS mte3 el host, machi mel Spark conf). Ken el JVM 3andou timezone mahouch UTC, kol timestamp `.collect()`-a ye-jini **shifted b sa3a** (walla akthar) bidoun 7atta erreur wahda. Kifha saa3a fi bit 3andha zouz cadrans mo5tlifin – wa7ad ye9oul el wa9t es-sa7i7, el theni ye9oul wa9t ghalt, w ma-fama-ch aya indication eli fama probleme.

Soual (Jury)

Kifeh a3raftou b eli hedhi el discrepancy (session.timeZone=UTC walakin collect() ye3tik wa9t ghalt) mawjouda 7a9i9atan, mouch just theoretical concern? Réponse (mel comment fi `_pin_jvm_timezone`): *"confirmed empirically: with session.timeZone=UTC but a non-UTC host, date_format() (pure SQL-side) shows the correct UTC instant while .collect() shows it shifted by the host's offset."* Ye3ni: testés real, machi assumption – b date_format() (li ye5dem 100% jowa Spark SQL) el output sa7i7, walakin nafs el instant, ki t-collect-ih l Python, ye5rej ghalt. Hedha el evidence li 5alla el authors yzidou el `TZ` environment variable fix (mouch just Spark conf fix).

2.4.1 SparkTimezoneError

```python
class SparkTimezoneError(RuntimeError):
    """Raised when a SparkSession cannot be confirmed to run in UTC.

    ThreatLake assumes every timestamp collected to Python is UTC
    wall-clock time - silver mappers, feature windows, and every test rely
    on it.
    """
```

Exception class custom – `RuntimeError` (machi `ValueError` kif `PathError`) 3la 5atr hedhi mochkla runtime/environment (JVM misconfiguration), machi valeur ghalta mel user.

2.4.2 _DELTA_CONF

```python
_DELTA_CONF: dict[str, str] = {
    "spark.sql.extensions": "io.delta.sql.DeltaSparkSessionExtension",
    "spark.sql.catalog.spark_catalog": "org.apache.spark.sql.delta.catalog.DeltaCatalog",
}
```

Zouz Spark confs required bech Delta Lake ye5dem: el awel ye-enable el extensions mte3 Delta 3la SQL parser (ye5alli `MERGE`, `DELETE`, `time travel` syntax ye5edmou), el theni ye-register Delta ki el catalog implementation predefini (`spark_catalog`) bech kol table tetsawwar/ta9ra automatiquement b format Delta bidoun ma tzid `USING delta` f kol query.

2.4.3 _pin_python_interpreter

```python
def _pin_python_interpreter() -> None:
    """Make Spark's Python workers use the same interpreter as the driver.
```

```
    Without this the JVM spawns workers with whatever ``python3`` is first
    on PATH, which fails with PYTHON_VERSION_MISMATCH whenever the active
    venv is not the system Python.
    """
    for var in ("PYSPARK_PYTHON", "PYSPARK_DRIVER_PYTHON"):
        os.environ.setdefault(var, sys.executable)
```

- `sys.executable`: el path exact mte3 l'interpreteur Python li rah ye5dem el process el actuel (yzid `.venv/bin/python` mathalan, machi `/usr/bin/python3` el system).
- `os.environ.setdefault`: ye7ott el variable *ken mahouch deja mou3ammar* – ken el user 3andou `PYSPARK_PYTHON` deja f environment mte3ou (override intentionnel), el function ma-t-override-ch.
- 3lech muhim: fi local mode, Spark yesta3mel subprocess Python separate ("workers") l el execution el 7a9i9i. Ken el driver ye5dem b virtual environment (`.venv`) w el workers ye5edmou b system Python (li momken ykoun version mo5talfa), Spark ye5rej erreur `PYTHON_VERSION_MISMATCH` – w hedhi el fix el standard.

2.4.4 _pin_jvm_timezone

```
def _pin_jvm_timezone() -> None:
    """Force the JVM's default timezone to UTC before it starts.

    ``spark.sql.session.timeZone`` (set below) only governs Spark SQL's own
    timestamp computation. When PySpark materializes a TimestampType into a
    Python ``datetime`` (e.g. ``.collect()``), that conversion goes through
    the JVM's OS-level default timezone instead - confirmed empirically:
    with session.timeZone=UTC but a non-UTC host, ``date_format()`` (pure
    SQL-side) shows the correct UTC instant while ``.collect()`` shows it
    shifted by the host's offset. Only the ``TZ`` process environment
    variable, set before the JVM is spawned, fixes this in local mode.
    """
    os.environ.setdefault("TZ", "UTC")
```

- Ligne wa7da bark f el code (`os.environ.setdefault("TZ", "UTC")`), walakin hedhi el *fix* el 7a9i9i l el probleme el mecheh fi el docstring mte3 el file kamel. `TZ` houwa el environment variable el standard (POSIX) li kol JVM ya9raha 3nd el startup bech ye-decide el default timezone mte3ha.
- *"before it starts"* (fi el function name w docstring): muhim `TZ` yet7att *9bal* el JVM yetbena – ken el JVM deja started (session existant), tbeddel `TZ` f Python process environment ma-yebdilech chay, 3la 5atr el JVM deja 9ra el `TZ` mel bidaya w `cache`-ha internally.
- `setdefault` nafs el logique: ma-y-override-ch ken el user 7ata `TZ` mou3ayen mel bara7.

2.4.5 _local_session

```
def _local_session(settings: Settings) -> SparkSession:
    """Build a Delta-enabled local session."""
    from delta import configure_spark_with_delta_pip
    from pyspark.sql import SparkSession

    _pin_python_interpreter()
    _pin_jvm_timezone()
    builder = SparkSession.builder.appName(settings.spark.app_name)
    if settings.spark.master:
        builder = builder.master(settings.spark.master)

    conf: dict[str, str] = {
        **_DELTA_CONF,
        "spark.sql.shuffle.partitions": str(settings.spark.shuffle_partitions),
```

```
        **settings.spark.conf,
    }
    for key, value in conf.items():
        builder = builder.config(key, value)

    return configure_spark_with_delta_pip(builder).getOrCreate()
```

- `from delta import ...` w `from pyspark.sql import ...` jowa el function, machi f top mte3 el file – imports "lazy". 3lech: `spark.py` ye-import mel modules o5rin (`config.py`) li *ma lezemhomch* PySpark bech ye5edmou (unit tests basiques mte3 `config.py` ma-lezemhomch install kamla mte3 PySpark). Ken el import ykoun global f top mte3 el file, kol chay li ye-import `threatlake.common.spark` (7atta indirectement) ye-force install PySpark, 7atta ken ma-yesta3melha-ch fi el path el actuel.
- `_pin_python_interpreter()` w `_pin_jvm_timezone()`: ye3ayet-lhom *9bal* ma yetbena el `SparkSession.builder` – lezem el environment variables (`PYSPARK_PYTHON`, `TZ`) ykounou mou3ammrin 9bal ma el JVM process yetsawwar.
- `if settings.spark.master:`: `master` optional (`str | None`), ken `None` (mathalan f environment Databricks-style win el master jeya automatiquement mel cluster), el code ma-y-set-ha-ch manuel w y5alli Spark ye5dem b default mte3ha.
- `conf` dict: b `**` unpacking, ye-merge tlata sources b ordre precis – `_DELTA_CONF` (base), ba3d `shuffle.partitions` mel settings, ba3d `**settings.spark.conf` (extra confs mel YAML, momken toverride 7atta `_DELTA_CONF` ken l'user 7eb – 3la 5atr houma a5er f el merge order, "last wins").
- `configure_spark_with_delta_pip(builder).getOrCreate()`: `configure_spark_with_delta_pip` houwa helper mel package `delta-spark` li ye-download/y-attach el JAR files el corrects mte3 Delta Lake automatiquement (b nazar l version mte3 PySpark installé) – bidoulha lezem tzid `-packages io.delta:...` manuel 3nd kol run.

2.4.6 El probe – _PROBE_WALL_CLOCK w _PROBE_EXPECTED_DATETIME

```
#: The instant the live probe below asks Spark to parse and hand back.
#: Chosen arbitrarily; only its exact round-trip matters.
_PROBE_WALL_CLOCK = "1970-01-01 00:00:01"
_PROBE_EXPECTED_DATETIME = datetime(1970, 1, 1, 0, 0, 1) # noqa: DTZ001 - deliberately naive, see below
```

- `_PROBE_WALL_CLOCK`: string wa9t arbitraire (Unix epoch + 1 second, walakin momken ykoun ay wa9t) – houwa el "input" li rah ye-round-trip-ih el probe: ye-parse-ha Spark SQL ki timestamp, ba3d ye-collect-ha l Python, w ye-9aren el result m3a el original.
- `# noqa: DTZ001 - deliberately naive, see below`: linter (ruff, rule DTZ001) 3adatan ye-warn 3la `datetime()` bidoun timezone info ("naive datetime"), 3la 5atr naive datetimes momken ykounou ambiguous. Hna, "naive" houwa *deliberate*: PySpark's `.collect()` ye-rajja3 hakka naive Python `datetime` objects (bidoun tzinfo) – assumption el system kamel houwa eli "naive datetime = UTC wall-clock" (kifma el docstring mte3 el class `SparkTimezoneError` ye9oul). `_PROBE_EXPECTED_DATETIME` lezem ykoun naive nafsou bech el comparison (`==`) ma3 el collected value ykoun sa7i7 (comparer naive m3a aware ye-crash f Python, w comparer aware m3a naive ye-crash zeda).

2.4.7 _check_utc_timezone – el core logic, factored out l test

```
def _check_utc_timezone(session_timezone: str | None, sql_side: str, collected: datetime) -> None:
    """Pure comparison logic behind the live probe, factored out from
    :func:`_verify_utc_timezone` so it can be unit tested directly: a
```

```
    genuinely divergent JVM default timezone can't be faked at runtime on
    an already-started JVM, so reproducing the failure mode for a test
    means calling this function with hand-picked disagreeing values
    instead of a real broken session.
    """
    if session_timezone != "UTC":
        raise SparkTimezoneError(
            f"spark.sql.session.timeZone is {session_timezone!r}, expected 'UTC'. "
            "ThreatLake PFA assumes every Spark session runs in UTC."
        )

    if sql_side != _PROBE_WALL_CLOCK:
        # Extremely unlikely - would mean session.timeZone itself doesn't do
        # what it says - but if Spark SQL's own formatting doesn't agree
        # with what was asked for, the Python-side comparison below is moot.
        raise SparkTimezoneError(
            f"Spark SQL formatted a UTC probe timestamp as {sql_side!r}, expected "
            f"{_PROBE_WALL_CLOCK!r}, despite spark.sql.session.timeZone=UTC."
        )

    if collected != _PROBE_EXPECTED_DATETIME:
        offset = collected - _PROBE_EXPECTED_DATETIME
        raise SparkTimezoneError(
            "spark.sql.session.timeZone reports UTC and Spark SQL agrees (a probe "
            f"timestamp formatted as {sql_side!r}), but materializing that same "
            f"timestamp to Python gives {collected} - off by {offset}. This means "
            "the JVM's OS-level default timezone is not UTC, which would silently "
            "shift every timestamp this pipeline collects to the driver."
        )
```

Soual (Jury)

3lech el authors 5arjou `_check_utc_timezone` f function séparée (pure, bidoun Spark) mnin `_verify_utc_timezone`, minstead ma yketbou kol chay f function wa7da?
Réponse: houwa el docstring li ye9olha directly – *"a genuinely divergent JVM default timezone can't be faked at runtime on an already-started JVM"*. Ye3ni: ken 7bit test-i el cas ("JVM timezone mahouch UTC, lezem yban erreur"), ma t9darch t-simule hedha b session Spark 7a9i9i f nafs el process (el JVM deja started b timezone wa7ed, ma t9darch tbeddel-ha runtime). Fa el fix houwa: tfasel el *comparison logic* (pure function, ta5ou strings/datetime, terja3 walla te-raise) mel *probe execution* (li lezemha Spark 7a9i9i). El test unitaire ye3ayet `_check_utc_timezone` direct b values "hand-picked disagreeing" (mathalan `collected` 3an-dou offset mte3 sa3a) bidoun 7atta ye5dem Spark – test rapide, deterministe, w ye-cover exactement el failure mode el mou5if bidoun infrastructure te9ila.

- 3 checks séquentiels, kol wa7ad y-raise b message clair w specifique:
  1. `session_timezone != "UTC"`: el check el basic – el conf nafsou (`spark.sql.session.timeZone`) 9al eli mahouch UTC.
  2. `sql_side != _PROBE_WALL_CLOCK`: el conf t9oul UTC, walakin Spark SQL nafsou (`date_format`) ma-formatiech el probe kifma el input. Comment ye9oul "extremely unlikely" – hedha el check ye-cover cas nadhari bark (bug fi Spark nafsou), rare barcha, walakin present bech el logique tkoun kamla w defensive.
  3. `collected != _PROBE_EXPECTED_DATETIME`: hedha el check el *7a9i9i el muhim* – el gap bin Spark SQL (correct) w Python-side collect (momken ghalt). `offset = collected - _PROBE_EXPECTED_DATETIME` y7seb el difference exacte (`timedelta`) w y-includi-ha f el error message, bech l'debugger ya3raf b kaddech el shift (sa3a wa7da? zouz?).

2.4.8 `_verify_utc_timezone` – el probe el 7a9i9i

```
def _verify_utc_timezone(session: SparkSession) -> None:
```

```
    """Run the live probe against ``session`` and raise if it isn't UTC.

    Checks ``spark.sql.session.timeZone`` AND independently re-derives the
    answer by round-tripping a known instant through Spark SQL and back to
    Python - the conf value alone is not trusted.
    """
    from pyspark.sql import functions as F

    probe = session.createDataFrame([(1,)], ["_i"]).select(
        F.to_timestamp(F.lit(_PROBE_WALL_CLOCK)).alias("ts")
    )
    row = probe.select(
        F.col("ts"), F.date_format(F.col("ts"), "yyyy-MM-dd HH:mm:ss").alias("sql_side")
    ).first()
    assert row is not None # noqa: S101 - always exactly one row from an unconditional select

    _check_utc_timezone(session.conf.get("spark.sql.session.timeZone"), row["sql_side"], row["ts"])
```

- `session.createDataFrame([(1,)], ["_i"])`: ye5al9 DataFrame b row wa7da bark, column wa7da (`_i`, value `1`) – el fikra hna mahich el data nafsou, houwa just "scaffold" bech na3mlou `.select()` 3lih w n7otou el probe expression.
- `F.to_timestamp(F.lit(_PROBE_WALL_CLOCK))`: ye-parse el string `"1970-01-01 00:00:01"` l `TimestampType` *jowa Spark SQL* (b nazar l `spark.sql.session.timeZone`, li lezemha tkoun UTC).
- `F.date_format(F.col("ts"), "yyyy-MM-dd HH:mm:ss")`: ye-format el timestamp *mel jdid* l string, dima jowa Spark SQL – hedha el "sql_side" el check el theni.
- `.first()`: ye5dem `.collect()` implicitement 3la row wa7da – hna houwa el moment el critique win `row["ts"]` (el TimestampType) ye-materialize l Python `datetime` *3ber el JVM's default timezone*, machi `session.timeZone`.
- `assert row is not None`: guarantee eli `.first()` ma rajja3-ch `None` – 3la 5atr el `select` bidoun `WHERE` 3la DataFrame b row wa7da, el result lezem ykoun exactement row wa7da, 9att `None`. Comment ye9oul "always exactly one row from an unconditional select" – nafs pattern el `assert` el mous-ta3mal f `fs.py` (type narrowing, machi runtime defensive check 7a9i9i).
- El call l `_check_utc_timezone`: y3addi tlata values – el conf (`session.conf.get(...)`), el sql-side result (`row["sql_side"]`), w el collected datetime (`row["ts"]`). Hedhi el "impure" part mte3 el probe (ta9ra mel session 7a9i9i) separee 3la el "pure" comparison logic el testable.

2.4.9 get_spark w stop_spark

```
def get_spark(settings: Settings | None = None) -> SparkSession:
    """Return the local SparkSession, verified to run in UTC.

    Raises ``SparkTimezoneError`` rather than return a session that would
    silently corrupt every timestamp it collects.
    """
    settings = settings or get_settings()
    session = _local_session(settings)
    _verify_utc_timezone(session)
    session.sparkContext.setLogLevel(settings.spark.log_level)
    return session


def stop_spark() -> None:
    """Stop the active session, if any."""
    from pyspark.sql import SparkSession

    session = SparkSession.getActiveSession()
    if session is not None:
        session.stop()
```

- `get_spark`: el entry point el public el wa7ed – kol module o5or (`ingestion/`, `transform/`, `ml/`...) ye3ayet `get_spark()` bark, ma ye3rafouch 7atta wa7ad mel details (`_local_session`, `_verify_utc_timezone`) – hedha houwa el "factory pattern": function wa7da public, kol el complexity mahfouza jowa.
- Order el operations: `_local_session` (session ye5dem), ba3d *direct* `_verify_utc_timezone` (9bal ma yrajja3 el session l caller). Ken el verification t-fail, exception ye-propagate w el caller ma-ya5dech-ch session ghalt fi yaddih – *"rather than return a session that would silently corrupt every timestamp"* (docstring).
- `session.sparkContext.setLogLevel(settings.spark.log_level)`: yet7att ba3d el verification, mouch 9bal – 3la 5atr el log level (`"WARN"` default) ye5dem 3la logs mte3 Spark internal, machi 7aja m3allé9a m3a el correctness mte3 el session.
- `stop_spark`: `SparkSession.getActiveSession()` ye-check ken fama session active 9bal ma y7awel ye-stop-ha – `if session is not None` ye5alli `stop_spark()` ykoun idempotent (call-ha marat 3adda, bidoun erreur ken deja stopped walla ma-tbenetch 7atta marra).

Chapter 3

ingestion/ – mel raw NDJSON l bronze

Hedha el package fih 3 files: `schemas.py` (registry mte3 schemas explicit), `bronze_transform.py` (transforms pure, action-free), w `bronze_writer.py` (l'orchestration el 7a9i9iya li ta9ra, tparse, w tektb).

3.1 ingestion/schemas.py

```
The schema lives in ``config/schema/cowrie.py`` rather than ``src/``
because, like ``config/local.yaml``, it describes *this deployment's*
known data shape and should be editable without a code change. It is
discovered through the same config-directory mechanism as
``threatlake.common.config``, then loaded as a plain Python module by
file path.

ThreatLake AI registers four source types (suricata, cowrie, dionaea,
heralding) here. PFA is cowrie-only (see ARCHITECTURE.md), so
``SOURCE_TYPES`` has exactly one entry - but the loader mechanism itself
is unchanged, so adding a second source later is a one-line addition to
``SOURCE_TYPES``/``_MODULE_BY_SOURCE`` plus a new schema file, not a
redesign.
```

Soual (Jury)

3lech el schema (structure mte3 el JSON, kol field w type mte3ha) mahouch f `src/`, walakin f `config/schema/cowrie.py` – 7atta houwa fi el a5er file `.py` kifma ay module fi `src/`?
Réponse: el schema ye-describe *el deployment el actuel*, mouch el code logique. Ken cowrie (el honeypot nafsou) ye-log field jdid ba3d update, walla ye-drop field 9dim, el fix houwa tbeddel `config/schema/cowrie.py` – *configuration change*, machi code change (ma-lezmek-ch PR review mte3 `src/`, ma-lezmek-ch re-deploy el package). Nafs el falsafa mte3 `config/local.yaml`: "chnowa el shape/values el m3aynin l hedha el deployment" ye-b9a separate 3la "kifeh el logique te5dem".

3.1.1 SOURCE_TYPES w _MODULE_BY_SOURCE

```
SOURCE_TYPES: tuple[str, ...] = ("cowrie",)

_MODULE_BY_SOURCE = {
    "cowrie": "cowrie.py",
}
```

`SOURCE_TYPES` tuple b entry wa7da bark (`"cowrie"`) – hedhi el scope el PFA el mou9arrar (contra ThreatLake AI li 3andha 4: suricata, cowrie, dionaea, heralding). `_MODULE_BY_SOURCE` ye-map

kol source type l esm el file `.py` mte3ou fi `config/schema/`.

3.1.2 SchemaError

```python
class SchemaError(RuntimeError):
    """Raised when a source schema cannot be located or loaded."""
```

Exception custom o5ra, nafs el pattern (`RuntimeError` subclass) – kol package 3andou exception mte3ou el specifique.

3.1.3 _load_schema – dynamic module loading

```python
def _load_schema(source_type: str) -> StructType:
    try:
        filename = _MODULE_BY_SOURCE[source_type]
    except KeyError as exc:
        raise SchemaError(
            f"Unknown source type {source_type!r}. Expected one of: {', '.join(SOURCE_TYPES)}"
        ) from exc

    path = config_dir() / "schema" / filename
    if not path.is_file():
        raise SchemaError(f"Schema file not found for {source_type!r}: {path}")

    spec = importlib.util.spec_from_file_location(f"threatlake_schema_{source_type}", path)
    if spec is None or spec.loader is None:
        raise SchemaError(f"Could not load schema module from {path}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)

    try:
        schema: StructType = module.SCHEMA
    except AttributeError as exc:
        raise SchemaError(f"{path} does not define SCHEMA") from exc
    return schema
```

- `except KeyError as exc: raise SchemaError(...) from exc`: pattern "exception chaining" – `from exc` y5alli el traceback el original (`KeyError`) mzabet ma3 el exception el jdida (`SchemaError`), bech el debugging ye-b9a clair (ta3raf el `SchemaError` jeyya mel `KeyError` lookup, mouch min chay o5or).
- `importlib.util.spec_from_file_location(...)` + `module_from_spec` + `exec_module`: hedhi el "3 étapes" el standard mte3 Python bech t-load module mel path *directement* (bidoun ma yk-oun el file f `sys.path` walla `PYTHONPATH`). Mousta3mla hna 3la 5atr `config/schema/cowrie.py` mahouch jowa `src/` (mahouch "importable" normalement b `import threatlake....`).
- `f"threatlake_schema_source_type"`: esm arbitraire l module fi `sys.modules` internal – prefix `threatlake_schema_` y-avoid collision m3a modules o5rin f nafs el process.
- `module.SCHEMA`: el convention el mou9arrar – kol schema file (`cowrie.py`) lezem ykoun 3andou variable esmha `SCHEMA` (`StructType`). Ken el file ma-3andou-ch, `AttributeError` ye-catch w ye-convert l `SchemaError` b message clair (*"does not define SCHEMA"*), machi `AttributeError` generic li ma yban-ch mnin.

Analogy

`importlib.util.spec_from_file_location` kifha "load plugin mel path direct" – kif application desktop (Photoshop, VS Code) t-load plugin mel folder m3ayen (mouch mel "install location" el standard), Python hna ye-load `.py` file mel path arbitraire w ye-execute-ha kifha module 3adi, bidoun ma tzid-ha l `PYTHONPATH` walla t-install-ha kif package.

3.1.4 source_schema – el entry point el cached

```
@cache
def source_schema(source_type: str) -> StructType:
    """Return the explicit StructType for a source type.

    Cached: schema modules are loaded from disk once per process.
    """
    return _load_schema(source_type)
```

`@cache` (mel `functools`, equivalent l `@lru_cache(maxsize=None)`) 3la function b *argument* (`source_type`) – machi bidoun argument kif `get_settings()`. Kol `source_type` distinct (hna "cowrie" bark, walakin design ye5alli barcha) ye5dou entry separate fi el cache. Nafs el motivation kif `get_settings`: `_load_schema` ya3mel I/O (disk read + `exec_module`, li houwa expensive nisbiyan), w el function hedhi te-ncall mel barcha places (`bronze_transform.py` kol marra ye-parse walla ye-finalize).

3.2 ingestion/bronze_transform.py

```
Every function here is a plain column-expression transform - no
``.count()``, ``.collect()``, ``.first()``, or other Spark action, and no
I/O. Keeping this module action-free is what makes it safe to test
without a live SparkSession action being forced at import time, and keeps
the parse/quarantine-split logic in one place regardless of how many call
sites end up needing it.
```

Soual (Jury)

3lech "action-free" (bidoun `.count()`, `.collect()`...) houwa design constraint mou9asad, mouch just coincidence?
Réponse: Spark DataFrames ye5edmou b "lazy evaluation" – `.withColumn`, `.select`, `.filter` ma-ye-executiwch chay fi el lever, ye-bnou just "plan" (DAG). Bark `.count()`, `.collect()`, `.write`... ("actions") ye-forciw el execution 7a9i9iya. Ken `bronze_transform.py` kan fih actions, kol test unitaire li ye3ayet `parse_source` mathalan lezmou SparkSession *active* bech el test rou7ou ye5dem – slow tests, w tests li lezemhom Spark cluster (7atta local) bark bech ta3mel unit test 3la logique transform basic. Bidoun actions, el functions hedhoula ye5edmou 3la DataFrame "plan" bark, w ta9dar t-test-hom b DataFrames sghar bidoun overhead.

3.2.1 CORRUPT_COLUMN w METADATA_COLUMNS

```
CORRUPT_COLUMN = "_corrupt_record"

METADATA_COLUMNS = (
    "ingest_date",
    "source_type",
    "_ingested_at",
    "_source_file",
    "_record_hash",
    "_raw",
)
```

- `CORRUPT_COLUMN`: esm el column li Spark's PERMISSIVE JSON mode (chnou el pipeline testa3mel, nchoufouha ba3d) ye7ott fiha el raw text mte3 line li ma-parsat-ch, w `null` f kol chay o5or.
- `METADATA_COLUMNS`: 6 columns li kol row (bronze walla quarantine) 3andha, barra 3la el data mte3 el event nafsou – `ingest_date` (date mte3 el batch run), `source_type` (`"cowrie"` dima hna), `_ingested_at` (timestamp exact), `_source_file` (win jat el line – audit trail), `_record_hash` (hash mte3 el content – l dedup, kifma chefna f `bronze_writer.py`), w `_raw` (el original text, bidoun ma tetbeddel).

3.2.2 with_ingestion_metadata

```python
def with_ingestion_metadata(df: DataFrame, source_type: str, ingested_at: datetime) -> DataFrame:
    """Stamp every row with its source, ingestion instant, and content hash.

    ``ingested_at`` is a single Python-computed instant for the whole
    batch, not ``current_timestamp()`` per row - every row in one run
    should agree on one ``ingest_date``.
    """
    return (
        df.withColumn("source_type", F.lit(source_type))
        .withColumn("_ingested_at", F.lit(ingested_at))
        .withColumn("ingest_date", F.to_date(F.col("_ingested_at")))
        .withColumn("_record_hash", F.sha2(F.col("_raw"), 256))
    )
```

- `F.lit(ingested_at)` machi `F.current_timestamp()`: hedhi el tafsila el critique – `ingested_at` houwa Python `datetime` m7soub *marra wa7da* 9bal ma yetba3et l hedhi el function (mel `bronze_writer.write_bronze`), ba3d ye-embed-ha (`F.lit`) f kol row. Ken testa3mel `F.current_timestamp(` kol row (fi partitions mo5tlifa, executors mo5tlifin) ye5dou timestamp *légèrement* mo5tlef (milliseconds difference), w `ingest_date` (`to_date` mennou) momken ykoun mo5tlef bin rows mel nafs el batch ken el batch ye5dem 3la midnight boundary – bug rare walakin s3ib ye-debug.
- `F.sha2(F.col("_raw"), 256)`: SHA-256 hash mte3 el raw text – hedha houwa el `_record_hash` li y-mentioned f `bronze_writer.py`'s docstring l dedup: nafs el line ye-ingest marat 3adda ye5dou nafs el hash, w nafs el `event_id` akhir (silver-side), fa duplicates ma-ye-corrupt-ch el data even ken el file ye-ingest twice.

3.2.3 parse_source

```python
def parse_source(df: DataFrame, source_type: str) -> DataFrame:
    """Parse `_raw` against the source's schema, flagging unparseable JSON."""
    schema = source_schema(source_type)
    corrupt_field = StructField(CORRUPT_COLUMN, StringType(), True)
    schema_with_corrupt = StructType([*schema.fields, corrupt_field])
    options = {"mode": "PERMISSIVE", "columnNameOfCorruptRecord": CORRUPT_COLUMN}
    return df.withColumn("_parsed", F.from_json(F.col("_raw"), schema_with_corrupt, options))
```

- `StructType([*schema.fields, corrupt_field])`: y-extend el schema el original (mel `config/schema/cowr` b field wa7ed zeda (`_corrupt_record`) – hedha el pattern el Spark documented l `from_json` PERMISSIVE mode: el `columnNameOfCorruptRecord` lezem ykoun *field jowa el schema el m3atat*, mouch column separate.
- `"mode": "PERMISSIVE"`: 3nd JSON parse fail, Spark *ma-ye5rej-ch erreur* – ye7ott `null` f kol el fields el parsed w yzid el raw text f `_corrupt_record`. Hedhi el base mte3 el "quarantine" flow – bidoun PERMISSIVE, line JSON wa7da ghalta ye-crash el batch kamel.
- `F.from_json(F.col("_raw"), schema_with_corrupt, options)`: el parse el 7a9i9i – ye-ekhraj struct column `_parsed` (nested, b kol el fields mte3 el schema + `_corrupt_record`).

3.2.4 split_quarantine

```python
def split_quarantine(df: DataFrame) -> tuple[DataFrame, DataFrame]:
    """Separate rows whose raw text failed to parse as JSON at all."""
    malformed = F.col("_parsed").isNull() | F.col(f"_parsed.{CORRUPT_COLUMN}").isNotNull()
    return df.filter(~malformed), df.filter(malformed)
```

`malformed` houwa el condition eli t7att row f quarantine: sawa `_parsed` nafsou null (JSON invalide kamel, ma-nja7ch el parse kamel), sawa `_corrupt_record` mahouch null (PERMISSIVE mode 3allem eno el line ma-t9addarch tetmatch m3a el schema). `return df.filter(~malformed),`

`df.filter(malformed)`: yrajja3 tuple b 2 DataFrames – el awel (`good`) houwa el rows el sa7 (negation b `~`), el theni (`bad`) houwa el malformed.

3.2.5 finalize_bronze w finalize_quarantine

```python
def finalize_bronze(df: DataFrame, source_type: str) -> DataFrame:
    """Project to metadata + ``source_type``'s own populated struct column -
    the physical shape of the bronze table, and exactly what
    ``threatlake.transform.silver.cowrie.map_cowrie`` expects to read.
    """
    parsed_field_names = [f.name for f in source_schema(source_type).fields]
    populated = F.struct(*[F.col(f"_parsed.{name}").alias(name) for name in parsed_field_names])
    return df.select(*METADATA_COLUMNS, populated.alias(source_type))


def finalize_quarantine(df: DataFrame) -> DataFrame:
    return df.select(*METADATA_COLUMNS, F.col(f"_parsed.{CORRUPT_COLUMN}").alias("_error"))
```

- `finalize_bronze`: ye-project mel struct `_parsed` (li fih kol field mte3 el schema + `_corrupt_record`) l struct jdid *bidoun* `_corrupt_record` (3la 5atr rows el hnaya deja `good`, ma-yehtajouch el field hedha) – `populated.alias(source_type)` y-esm el struct `"cowrie"`, w el result akhir houwa `METADATA_COLUMNS` (6 columns) + `cowrie` (struct wa7ed).
- Hedhi el shape 7a9i9i mte3 bronze table – w kifma el docstring ye9oul, hedhi *exactement* el shape li `map_cowrie` (silver) yes-ta5eleha. Fi ThreatLake AI (4 sources), el bronze row lezemha struct columns l kol el 4 sources (3 minhom null 3la kol row), w function `expand_to_unified_bronze_shape` te3mel had el "reconstruction". Hna, b source wa7ed bark, hedhi el étape *ma-lezmech*: bronze row 3andha struct `cowrie` wa7ed bark, mahich nsé.
- `finalize_quarantine`: nafs el fikra walakin ansem – `METADATA_COLUMNS` + `_error` (el raw corrupt text, mel `_corrupt_record`).

3.3 ingestion/bronze_writer.py

Hedha el file houwa el orchestrateur – ya5ou el functions pure mel `bronze_transform.py` w y-combine-hom m3a I/O 7a9i9i (Spark read, Delta write, file move).

```
Processed-file tracking: the exact set of files present in the landing
directory is captured once, up front, and used for both the read and the
post-write move - never re-listed - so a file that lands mid-batch is
left alone for the next run rather than partially processed. After a
successful write, every file in that set is *moved* (not deleted) to
``<landing>/cowrie/_processed/<ingest_date>/``, preserving them as an
audit trail while making the landing directory empty for the next run.
This gives at-least-once, not exactly-once, ingestion: if the process
dies between the Delta write succeeding and the move completing, the
same file is re-ingested on the next run, producing duplicate bronze
rows. That duplicate is harmless downstream - each silver ``event_id`` is
a content hash of the bronze row (see
``threatlake.transform.silver.schema``), so re-ingesting the same file
twice produces the same event_id both times rather than two silver rows.
```

Soual (Jury)

3lech "at-least-once" (momken duplicate rows f bronze) houwa acceptable, mouch tri ye5edem "exactly-once" strict?
Réponse: exactly-once ingestion (distributed transaction bin file-move w Delta write) houwa complex barcha w s3ib ye-implement correctly bidoun external coordinator (2-phase commit walla kifha). El authors 5tarou design as-hal: at-least-once mel ingestion layer (duplicates possible, rare – bark ken el process ye-crash f moment exact bin el write w el move), w exactly-

once semantics *effective* mel dedup downstream b content-hash (`event_id`). Hedhi separation of concerns: ingestion layer ye-focus 3la robustness (never lose data), silver layer ye-focus 3la correctness (never double-count) – kol wa7ed 3andou el responsabilité mte3ou.

Analogy

Kifha kif ta5dem b receipt/checklist mta3 déménagement – t7ott kol box f "processed" bark ba3d ma tet2akked eli el contenu deja fi el 5ir el jdid (el Delta write). Ken el kahraba te9ta3 (crash) 9bal ma tzid el checkmark (el move), tel9a el box mazelet f 'landing' – el marra el jayya, terja3 t-check-ha (te5dem duplicate work, walakin ma-tfou-tha 9att).

3.3.1 _PROCESSED_SUBDIR w BronzeWriteResult

```
_PROCESSED_SUBDIR = "_processed"


@dataclass(frozen=True)
class BronzeWriteResult:
    """Row counts from a single bronze ingestion batch."""

    source_type: str
    written: int
    quarantined: int
```

`@dataclass(frozen=True)`: nafs el pattern immutable li chefna f `Settings` – result object simple, 3andou 3 fields bark, machi lezmou class kbira b methods. `frozen=True` ye5alli-ha immutable (consistent m3a el falsafa el 3amma mte3 el codebase: results/settings immutable, mutations explicit bark f DataFrame transformations).

3.3.2 _read_landing_lines

```
def _read_landing_lines(spark: SparkSession, file_paths: list[str]) -> DataFrame:
    """One row per line across exactly the given files."""
    return (
        spark.read.text(file_paths)
        .withColumnRenamed("value", "_raw")
        .withColumn("_source_file", F.input_file_name())
    )
```

- `spark.read.text(file_paths)`: mahouch `spark.read.json(...)` el direct – ya9ra kol line ki string bark (column esmha `value` b default), *bidoun* parsing. El parsing el 7a9i9i ye-sir ba3d, explicit, b `parse_source` (bech ye9dar el pipeline ye-handle malformed lines manuellement – `spark.read.json` el direct ye-drop walla ye-corrupt rows ghalta bidoun control precis 3la el behavior).
- `.withColumnRenamed("value", "_raw")`: esm clair (`_raw`) machi el default generic (`value`).
- `F.input_file_name()`: Spark function li terja3 el path mte3 el file li jabet minou kol row – audit trail (`_source_file` f `METADATA_COLUMNS`), useful l debugging (win jat data ghalta).

3.3.3 _move_to_processed

```
def _move_to_processed(
    spark: SparkSession, file_paths: list[str], landing_dir: str, ingest_date: str
) -> None:
    processed_dir = f"{landing_dir}/{_PROCESSED_SUBDIR}/{ingest_date}"
    for path in file_paths:
        filename = path.rsplit("/", 1)[-1]
        fs.move(spark, path, f"{processed_dir}/{filename}")
```

`path.rsplit("/", 1)[-1]`: ye5ou el esm mte3 el file bark (a5er segment ba3d a5er "/") mel path el kamel. `fs.move` (mel `common/fs.py`, li chefnah) ye3ayet-lha l kol file, wa7ed wa7ed – el destination `{landing}/_processed/{ingest_date}/{filename}`.

3.3.4 `write_bronze` – el orchestration el kamla

```python
def write_bronze(
    source_type: str, spark: SparkSession, settings: Settings | None = None
) -> BronzeWriteResult:
    """Ingest one batch from ``source_type``'s landing directory into bronze.

    Idempotent in effect, not in mechanism: once a file is moved to
    ``_processed/``, re-running this against the same landing directory
    finds no files and does no work (no Spark job is even triggered).
    """
    if source_type not in SOURCE_TYPES:
        expected = ", ".join(SOURCE_TYPES)
        raise SchemaError(f"Unknown source type {source_type!r}. Expected one of: {expected}")

    settings = settings or get_settings()
    landing_dir = landing_path(source_type, settings)

    file_paths = fs.list_files(spark, landing_dir)
    if not file_paths:
        return BronzeWriteResult(source_type=source_type, written=0, quarantined=0)

    ingested_at = datetime.now(UTC)
    ingest_date = ingested_at.date().isoformat()

    raw = _read_landing_lines(spark, file_paths)
    raw = with_ingestion_metadata(raw, source_type, ingested_at)
    parsed = parse_source(raw, source_type)
    good, bad = split_quarantine(parsed)

    bronze_df = finalize_bronze(good, source_type).cache()
    quarantine_df = finalize_quarantine(bad).cache()

    written = bronze_df.count()
    quarantined = quarantine_df.count()

    bronze_df.write.format("delta").mode("append").partitionBy("ingest_date").option(
        "mergeSchema", "true"
    ).save(bronze_path(source_type, settings))

    if quarantined:
        quarantine_df.write.format("delta").mode("append").partitionBy("ingest_date").option(
            "mergeSchema", "true"
        ).save(quarantine_path(source_type, settings))

    bronze_df.unpersist()
    quarantine_df.unpersist()

    _move_to_processed(spark, file_paths, landing_dir, ingest_date)

    return BronzeWriteResult(source_type=source_type, written=written, quarantined=quarantined)
```

- `if source_type not in SOURCE_TYPES`: guard clause l'awel – fail fast b message clair ken `source_type` ghalt (typo mathalan), 9bal ma yebda 7atta ye-touch Spark.
- `file_paths = fs.list_files(...)` *marra wa7da*, w `if not file_paths: return ...(written=0, quarantined=0)`: hedha el "empty landing = normal, not error" (kifma chefna f `fs.list_files`), w hedhi el step el critique el docstring ye-emphasize: el list mte3 files "captured once, up front" – file jdid li yjini *f nos* el batch run (concurrent write mel honeypot) ma-yetahdech l hedha el run, y-processa el marra el jayya.
- `ingested_at = datetime.now(UTC)`: instant wa7ed mel batch kamel, ye3addiwh l `with_ingestion_metadata` (kifma chefna, `F.lit` machi `current_timestamp`).

- Pipeline el transforms: `_read_landing_lines` → `with_ingestion_metadata` → `parse_source` → `split_quarantine` → `finalize_bronze`/`finalize_quarantine` – kol step houwa function pure mel `bronze_transform.py`, el orchestrateur ye-chain-hom bark.
- `.cache()` 3la `bronze_df` w `quarantine_df`: el DataFrames rahoum lazy – bidoun cache, kol `.count()` w kol `.write` y-recompute el plan el kamel mel awel (mel `spark.read.text` l fou9). `.cache()` y5alli Spark ye5azzen el result f memory ba3d el execution el oula, bech el `.count()` (li ye5dem action) w el `.write` (action o5ra) ma-y3awdouch nafs el shughol.
- `written = bronze_df.count()`: hedhi el action el oula li t-force el execution – terja3 el row count el 7a9i9i (mous-ta3mal f el `BronzeWriteResult`).
- `.write.format("delta").mode("append").partitionBy("ingest_date").option("mergeSchema", "true")`: `mode("append")`: bronze table ye-grow bark, ma-ye-overwrite-ch (contra gold tables, li ye-overwrite kif nchoufou ba3dein). `partitionBy("ingest_date")`: physical partitioning 3la el disque b date – ye5alli queries li ye-filter 3la date range (mathalan "el a5er 7 iyem") ma-y9raouch el table kamla. `mergeSchema=true`: ken schema mte3 cowrie yetbeddel (field jdid f version jdida mte3 cowrie), Delta ye9bal el evolution automatiquement, machi ye-crash 3la mismatch.
- `if quarantined:`: write mte3 quarantine table *conditionnel* – ken `quarantined == 0` (kol el lines valides), ma-fama-ch 7ajja te3mel write 3la table fargh.
- `.unpersist()`: yfalless el cache mel memory ba3d el writes – 3ks `.cache()`, bech ma-yeb9ach memory mou7tal bidoun 7ajja ba3d ma el data temmet el write.
- `_move_to_processed(...)` *fi el a5er*, ba3d el writes kamlin (bronze w quarantine) – ordre muhim: ken el write ye-fail, el files ye-b9aou f landing (ma-yet7arkouch), w el run el jayya ye3awed y7awel – exactement el "at-least-once" el docstring ye-describe.

Chapter 4

transform/silver/ – el schema unifié w el mapping mte3 cowrie

Hedhi el couche ta5ou bronze rows (raw-ish, source-specific: struct `cowrie` b fields mte3 cowrie nafsou) w t7awwel-hom l schema unifié wa7ed (`SILVER_EVENT_SCHEMA`) – 18 columns, common l kol source (walaw PFA 3andha source wa7ed bark: cowrie).

4.1 transform/silver/schema.py

Hedha el file houwa "contract" el kbir – kol mapper (hna, `cowrie.py` bark) lezem yentej rows li ye-match had el schema exactement.

4.1.1 Semantique mte3 kol field – mel docstring

Docstring mte3 el file ye-explique kol field b tafsil, w hedhi el fikra el kbira: hedha schema *original*, machi mou5ta mel benchmark external (kif MITRE ATT&CK walla CVE taxonomy).

```
event_id
    ``sha2(source_type || ':' || _record_hash, 256)``. A content hash, not a
    random UUID: re-processing the same bronze row (e.g. after the
    at-least-once re-ingestion gap documented in
    ``threatlake.ingestion.bronze_writer``) always produces the same
    event_id, which is what makes exact-duplicate dedup
    (``threatlake.transform.silver.dedup``) possible.
```

Soual (Jury)

3lech `event_id` houwa content hash (`sha2(...)`) mouch UUID random (`uuid4()` mathalan)?
Réponse: kifma chefna f `bronze_writer.py`, el ingestion houwa "at-least-once" – file momken ye-ingest twice (crash bin write w move). Ken `event_id` ykoun UUID random, kol re-ingestion tebni row jdida b `event_id` mo5telfa – duplicate 7a9i9i fi silver, w gold aggregations (attacker_profiles, attack_timeline) ye-count el attack marat 3adda. B content hash (`sha2(source_type || _record_hash)`), nafs el bronze row (nafs `_record_hash`, mel `with_ingestion_metadata` f `bronze_transform.py`) dima tentej nafs `event_id` – fa dedup ye-sir automatiquement (row jdid m3a `event_id` mawjoud deja ye-drop-ha dedup step).

```
attack_category
    A small, original, coarse taxonomy chosen for cross-source filtering, NOT
    derived from any external standard: recon, credential_access,
    command_execution, malware_delivery, network_intrusion, service_probe,
    service_start, session_summary, connection, network_flow, other.
```

```
severity
    Normalized to 1-5, 5 = most severe/critical, 1 = informational. Chosen
    deliberately opposite to Suricata's native scale, where 1 is the highest
    priority - seeing "severity 5" mean "worst" is the more common SOC/
    dashboard convention, so Suricata's severity is inverted on the way in
    (native 1 -> 5, 2 -> 3, 3 -> 1).
```

- `attack_category` 11-valeur taxonomy: 3andha 9rar design muhim – "original", machi standard external. El fikra: kol source (cowrie, w f ThreatLake AI el kbir: suricata, dionaea, heralding zeda) 3andou vocabulary mo5tlef kamel l events mte3ou – taxonomy mouwahhad ye5alli `WHERE attack_category = 'credential_access'` ye5dem cross-source bidoun ma ta3raf source-specific event names.
- `severity` 1–5, 5=worst: hedhi el convention SOC/dashboard el 3adiya (opposé mte3 Suricata native scale, li 1=highest – mentioned l context walaw PFA cowrie-only ma3andhach Suricata). Cowrie *ma3andouch* native severity concept 7atta wa7ad – el mapping (`_CATEGORY_SEVERITY` f `cowrie.py`) houwa "an original, documented, intentionally simple per-eventid ... judgment call".

```
raw_ref
    ``source_type || ':' || _record_hash`` - unhashed and directly usable to
    look up the exact bronze row and its full raw JSON:
    ``SELECT * FROM bronze.raw_events WHERE source_type = split(raw_ref, ':')[0]
    AND _record_hash = split(raw_ref, ':')[1]``.

Dropped from every source (available via raw_ref, not carried into silver):
free-text fields with no analytic structure beyond what `description`
already summarizes.
```

Analogy

`raw_ref` kifha "receipt number" 3nd magasin – el ticket el mou5tasar (silver row) ma-fihech kol details (kol item, kol prix), walakin fih receipt number li ye5allik terja3 l el original document (bronze row, w mennou `_raw` el JSON el kamel). Hedhi el fikra: silver ye5od bark el fields el importants l'analytics, walakin ma-yfout-ch chay – kol chay mawjoud, just "wara" (mch f el silver row nafsou).

4.1.2 SilverSchemaError w ATTACK_CATEGORIES

```python
class SilverSchemaError(RuntimeError):
    """Raised when mapped rows violate the unified schema's NOT NULL columns."""


ATTACK_CATEGORIES = frozenset(
    {
        "recon",
        "credential_access",
        "command_execution",
        "malware_delivery",
        "network_intrusion",
        "service_probe",
        "service_start",
        "session_summary",
        "connection",
        "network_flow",
        "other",
    }
)
```

`frozenset` machi `set` 3adi walla `tuple`: `frozenset` immutable (nafs falsafa: fi kol codebase, constants global lezmohom ykounou immutable) w set membership check (`"x" in ATTACK_CATEGORIES`)

houwa $O(1)$ (hash lookup) machi $O(n)$ (linear scan kif tuple/list) – mahich critique l 11 values bark, walakin el habit el correcte.

4.1.3 SILVER_EVENT_SCHEMA – el 18 fields

```
SILVER_EVENT_SCHEMA = StructType(
    [
        StructField("event_id", StringType(), False),
        StructField("event_time", TimestampType(), True),
        StructField("ingest_date", DateType(), False),
        StructField("source_type", StringType(), False),
        StructField("source_event_type", StringType(), True),
        StructField("src_ip", StringType(), True),
        StructField("src_port", IntegerType(), True),
        StructField("dst_ip", StringType(), True),
        StructField("dst_port", IntegerType(), True),
        StructField("protocol", StringType(), True),
        StructField("attack_category", StringType(), True),
        StructField("severity", IntegerType(), True),
        StructField("session_id", StringType(), True),
        StructField("credentials_attempted", BooleanType(), False),
        StructField("attempted_username", StringType(), True),
        StructField("attempted_password", StringType(), True),
        StructField("payload_hash", StringType(), True),
        StructField("description", StringType(), True),
        StructField("raw_ref", StringType(), False),
    ]
)
```

- El 3eme argument mte3 kol `StructField` houwa `nullable` (`True`/`False`). Bark 5 fields `False` (NOT NULL): `event_id`, `ingest_date`, `source_type`, `credentials_attempted`, `raw_ref` – kol el baa9i (`event_time`, `src_ip`, `attack_category`...) momken ykounou null.
- 3lech hedhoula 5 el 5amsa specifiquement NOT NULL: `event_id` w `raw_ref` ye-derive mel `_record_hash` (dima mawjoud, mel bronze metadata). `ingest_date` w `source_type` zeda mel bronze metadata (dima populated, kifma chefna f `with_ingestion_metadata`). `credentials_attempted` houwa boolean li lezem ykoun true/false explicit (*"always true/false, never null"* kifma el docstring te9oul) – ma-fama-ch "unknown" state.
- `event_time` *momken ykoun null*: houwa el field el akthar "delicate" nadhariyan (el UTC verification f `spark.py` ye-concern-ha direct), walakin ye-b9a nullable 3la 5atr timestamp mte3 source momken ykoun missing/unparseable – el row ye-b9a (mahich rejected), just bidoun confirmed event time.

4.1.4 conform_to_silver_schema

```
def conform_to_silver_schema(df: DataFrame) -> DataFrame:
    """Select and cast ``df`` to exactly :data:`SILVER_EVENT_SCHEMA` - column
    names, order, and types - then verify the schema's NOT NULL columns are
    actually non-null.

    That second step matters: ``.cast()`` only changes a column's *type*.
    Spark infers a DataFrame's nullability from the expression tree that
    produced it, not from a target StructType you select against - so
    without an explicit check, a mapper bug that leaves e.g. ``event_id``
    null would silently pass through instead of failing where it happened.
    """
    conformed = df.select(
        *[F.col(field.name).cast(field.dataType).alias(field.name) for field in SILVER_EVENT_SCHEMA]
    )
    _assert_not_null(conformed)
    return conformed
```

Soual (Jury)

Docstring ye9oul `.cast()` "only changes a column's type" – 3lech hedhi tafsila muhimma bezef bech tefhemha?
Réponse: fi Spark, DataFrame's nullability (`StructField.nullable`) ma-tji-ch mel target schema li testa3mlou f `.select()` – Spark ye-infer-ha mel *expression tree* li 5al9et el column. Ye3ni: ken mapper (`cowrie.py`) 3andou bug w y5alli `event_id` null l row wa7da (mathalan expression ghalta li momken terja3 null), `.cast(StringType())` *ma-y-forceç-ch* el column tkoun NOT NULL – houwa just ye-cast el *type*. Bidoun el check el explicite (`_assert_not_null`), el bug hedha ye3addi silently l gold tables, w tel9a event_id null f attacker_profiles walla attack_timeline ba3d se3at/iyem, s3ib ye-trace l source mte3ou. B el check explicite, el pipeline ye-fail *fi el moment el mapper* rah, b message clair (*"This is a mapper bug, not bad input data"*).

4.1.5 _assert_not_null

```python
def _assert_not_null(df: DataFrame) -> None:
    required = [field.name for field in SILVER_EVENT_SCHEMA if not field.nullable]
    if not required:
        return
    null_counts = df.select(
        *[F.sum(F.col(name).isNull().cast("int")).alias(name) for name in required]
    ).first()
    assert null_counts is not None # noqa: S101 - always exactly one row from an unconditional select
    violations = {name: null_counts[name] for name in required if null_counts[name]}
    if violations:
        raise SilverSchemaError(
            f"Silver rows violate NOT NULL columns: {violations}. This is a mapper "
            "bug, not bad input data - every mapper must always populate these columns."
        )
```

- `required = [...]`: ye5ou el 5 fields NOT NULL bark (mel `SILVER_EVENT_SCHEMA` li chefnah).
- `F.sum(F.col(name).isNull().cast("int")).alias(name)`: pattern classic l count nulls – `isNull()` yentej boolean column, `.cast("int")` y-convert-ha l 0/1, `F.sum` ye7seb el total. Kol el 5 counts ye-t7esbou f query wa7da (`.select()` b comprehension) machi 5 queries separate – efficient, action wa7da bark (`.first()`).
- `violations = {name: ... for name in required if null_counts[name]}`: dict comprehension b filter – bark el fields li 3andhom count > 0 (`if null_counts[name]`, Python truthy 3la int: 0 = False, ay chay o5or = True) yed5elou f `violations`.
- El erreur message ye-includi el dict `violations` nafsou (mathalan `{'event_id': 3}`) – ta3raf exactement chnowa field w kaddech row victime, machi just "something is null".

4.2 transform/silver/cowrie.py – el mapping el kamel

Hedha el file houwa el mapper el wa7ed el mou-esta3mel fi PFA (cowrie-only). Docstring ye-documente el mapping field-by-field:

```
Mapping (see threatlake.transform.silver.schema for shared field semantics):

  event_time cowrie.timestamp
  source_event_type cowrie.eventid (e.g. "cowrie.login.failed")
  src_ip/src_port cowrie.src_ip/src_port - attacker
  dst_ip/dst_port cowrie.dst_ip/dst_port - the honeypot. Only populated
                        on connection-context events (session.connect); null
                        elsewhere per Cowrie's own schema (see
                        config/schema/cowrie.py) - join on session_id to
                        recover it for a specific session's other events.
```

```
  protocol lower(cowrie.protocol) - "ssh"/"telnet"
  session_id cowrie.session
  credentials_attempted true for cowrie.login.success / cowrie.login.failed
  attempted_username/ cowrie.username/password on those two eventids.
    password Password auth only: pubkey-auth attempts carry
                        fingerprint/key instead, which are dropped here
                        (available via raw_ref).
  payload_hash cowrie.shasum on file_download/file_upload
  description cowrie.message (already human-written by Cowrie)

Dropped entirely (available via raw_ref -> bronze._raw): sensor, late,
realm, outfile/destfile/duplicate, ttylog, size, key/fingerprint.
```

Soual (Jury)

3lech `dst_ip/dst_port` null 3la mou3zam el events (bark `session.connect` 3andhom-ha), w el fix el mou9tara7 houwa "join 3la session_id"?
Réponse: cowrie's own schema (mel `config/schema/cowrie.py`) ye7ott `dst_ip/dst_port` bark f el event el awel mte3 kol session (`cowrie.session.connect`) – events el o5rin (login, command, file...) mte3 *nafs* el session ma-3andhomch-ha (redundant l cowrie logger nafsou, 5ali tekteb-ha kol marra). El mapper ma-y7awelch "reconstruct" walla "forward-fill" hedha el value – ye5alliha kifma jat (null). Ken analyst 7eb ya3raf el honeypot IP/port l event m3ayen (mathalan login.failed), lezemou `JOIN` 3la `session_id` m3a el `session.connect` event mte3 nafs el session. Hedha 9rar design: keep el data kif jatek (honest, machi inferred), w 5alli el "join logic" l el consumer/query, mouch tzid complexity fi el mapper l chay li mahouch always needed.

4.2.1 _LOGIN_EVENTS w _FILE_EVENTS

```
_LOGIN_EVENTS = ("cowrie.login.success", "cowrie.login.failed")
_FILE_EVENTS = ("cowrie.session.file_download", "cowrie.session.file_upload")
```

Zouz tuples sghar, kol wa7ed ye-group eventids li 3andhom nafs el "treatment" f el mapping (login events → credentials_attempted + username/password, file events → payload_hash).

4.2.2 _CATEGORY_SEVERITY – el jadwal el kamel

Hedha el "table" el user 7eb tekoun mfasra kamla – kol `eventid` mte3 cowrie mapped l `(attack_category, severity)`:

```
# eventid -> (attack_category, severity). severity: 5=critical .. 1=informational.
_CATEGORY_SEVERITY = {
    "cowrie.session.connect": ("connection", 1),
    "cowrie.login.success": ("credential_access", 5), # attacker got in
    "cowrie.login.failed": ("credential_access", 2),
    "cowrie.command.input": ("command_execution", 4),
    "cowrie.session.file_download": ("malware_delivery", 4),
    "cowrie.session.file_upload": ("malware_delivery", 4),
    "cowrie.session.closed": ("session_summary", 1),
    "cowrie.log.closed": ("session_summary", 1),
}
_DEFAULT_CATEGORY_SEVERITY = ("other", 2)
```

- `cowrie.session.connect` → (`connection`, 1): just TCP connection etabli, 7atta authentication ma-sartech ba3d – severity el a9al (1, informational). Category `connection` houwa el wahda el bark mte3ha specifique l "network-level" event (mahouch behavior mte3 el attacker ba3d).
- `cowrie.login.success` → (`credential_access`, 5): el event el akthar critique – attacker *da5al* el honeypot b succès. Comment inline (`# attacker got in`) ye-explique 3lech severity

5 (max) – login success houwa el signal el akthar actionable/dangereux f el system kamel.

- `cowrie.login.failed` → (`credential_access`, 2): nafs category (`credential_access`, 3la 5atr houwa zeda "attempt" 3la credentials) walakin severity ba3ida barcha (2, low-ish) – failed attempts homa el mou3zam mte3 el traffic 3la honeypot (barcha brute-force noise), fa severity ba3ida t5alli el dashboard ma-yeb9ach flooded b "alerts" l chay mtawa9a3.
- `cowrie.command.input` → (`command_execution`, 4): attacker deja da5al w rah ye5dem commands 3la el shell – severity 3alya (4) 3la 5atr hedha behavior actif w potentiellement destructif (attacker momken ye7awel install malware, exfiltrate data...).
- `cowrie.session.file_download` w `cowrie.session.file_upload` → (`malware_delivery`, 4): kol el 2 (upload w download) nafs el category w severity – el fikra: file transfer (fi ay direction) 3la honeypot houwa *presque dima* malware-related (attacker ye7ott payload, walla ye-download tool).
- `cowrie.session.closed` w `cowrie.log.closed` → (`session_summary`, 1): events "housekeeping" – el session temmet, ma-3andhomch-ch signal nouveau 3la el attacker behavior (el behavior deja captured mel events el sa-b9in), severity el a9al.
- `_DEFAULT_CATEGORY_SEVERITY = ("other", 2)`: fallback l eventid mahouch f el dict – category `"other"` (mawjouda f `ATTACK_CATEGORIES`) w severity metoseta (2). Hedhi ye5alli el mapper ma-y-crash-ch 3la eventid jdid (cowrie update, feature jdida) – el row ye-b9a mapped (bidoun ma tel9a exception), just b category generic 7atta tzid entry jdid f el dict.

Soual (Jury)

Kifeh 9arrart el severity mte3 kol eventid – (5, 2, 4, 4, 4, 1, 1)? Fama methodology 7a9i9iya walla judgment call?
Réponse: docstring mte3 el file el kbir (`schema.py`) ye9oul directly: *"The other three sources have no native severity concept at all; their mapping is an original, documented, intentionally simple per-eventid/connection-type judgment call"*. Ye3ni: mahiche derived men CVSS walla standard external – houwa 9rar design mou5tasar w documenté, mbni 3la "kaddech hedha el event ye3ni chay actionable/dangereux l analyst SOC". `login.success` (5, max) houwa el akthar objectivement critique (attacker got in). `command.input` w file transfer (4) homa "post-compromise activity". `login.failed` (2) low 3la 5atr houwa noise (barcha minha, mou3zam-hom automated bots). `connection/closed` (1) houwa metadata bark, machi signal behavior. El transparence houwa el 9ima hna: el comment inline w el docstring ye5alliw kol reader (jury mathalan) ye-verifier el logique w ye-conteste-ha ken 7eb, contra "black box scoring" bidoun rationale.

4.2.3 `_category_severity_columns` – kifeh el dict ye-b9a Spark expression

```python
def _category_severity_columns() -> tuple[Column, Column]:
    # Each eventid is mutually exclusive, so iteration order doesn't matter -
    # every branch below only ever overrides the still-default value.
    category = F.lit(_DEFAULT_CATEGORY_SEVERITY[0])
    severity = F.lit(_DEFAULT_CATEGORY_SEVERITY[1])
    for eventid, (cat, sev) in _CATEGORY_SEVERITY.items():
        is_match = F.col("cowrie.eventid") == eventid
        category = F.when(is_match, F.lit(cat)).otherwise(category)
        severity = F.when(is_match, F.lit(sev)).otherwise(severity)
    return category, severity
```

- El fikra el jdida hna: el dict Python (`_CATEGORY_SEVERITY`) *mahouch* 9adar ye-esta3mel direct f Spark expression (Spark ma-y3arraf-ch `dict.get()` 3la column mte3ou – lezem kol chay ykoun

`Column` expression). Fa el function hedhi t-convert el dict l *chain* mte3 `F.when(...).otherwise(...)` expressions.

- `category = F.lit(_DEFAULT...)`: yebda b el default (`"other"`) ki starting point.
- El loop: kol iteration, `category = F.when(is_match, F.lit(cat)).otherwise(category)` – hedhi *ma tbeddelch* el value fil-7in (Spark expressions immutable, lazy), houwa ye-build expression jdid *yelfef* el expression el 9dim (`otherwise(category)`). Ba3d el loop kamel, `category` houwa chain twil mte3 8 `when` clauses (wa7ed l kol eventid f `_CATEGORY_SEVERITY`), b el default f el a5er (a3ma9 nested `otherwise`).
- Comment (*"Each eventid is mutually exclusive, so iteration order doesn't matter"*): 3la 5atr kol row 3andha `eventid` wa7ed bark, bark `when` wa7ed momken y-match (el ba9i `is_match` houm False l nafs el row), fa el ordre eli el dict ye-iterate bih (Python 3.7+ dict ye-preserve insertion order, walakin hna ma-yehemch) ma-yathrouch 3la result el akhir.

Analogy

`F.when(...).otherwise(...)` chain houwa el equivalent Spark mte3 `if/elif/elif/.../else` statement el 3adi f Python – walakin *expression-based* (yentej value, machi statement li ye-execute). Nested `otherwise` kifha "nesting" mte3 conditions – kol wa7ed ye-check 9bal ma yerja3 l el "else" (el default walla el when el 9bel).

4.2.4 `map_cowrie` – el function el kamla

```python
def map_cowrie(bronze_df: DataFrame) -> DataFrame:
    """Map bronze rows with ‘‘source_type = 'cowrie'‘‘ to the unified silver model."""
    df = bronze_df.filter((F.col("source_type") == "cowrie") & F.col("cowrie").isNotNull())

    is_login_event = F.col("cowrie.eventid").isin(*_LOGIN_EVENTS)
    is_file_event = F.col("cowrie.eventid").isin(*_FILE_EVENTS)
    attack_category, severity = _category_severity_columns()

    mapped = df.select(
        F.sha2(F.concat(F.col("source_type"), F.lit(":"), F.col("_record_hash")), 256).alias(
            "event_id"
        ),
        F.to_timestamp(F.col("cowrie.timestamp"), "yyyy-MM-dd'T'HH:mm:ss.SSSSSS'Z'").alias(
            "event_time"
        ),
        F.col("ingest_date").alias("ingest_date"),
        F.col("source_type").alias("source_type"),
        F.col("cowrie.eventid").alias("source_event_type"),
        F.col("cowrie.src_ip").alias("src_ip"),
        F.col("cowrie.src_port").alias("src_port"),
        F.col("cowrie.dst_ip").alias("dst_ip"),
        F.col("cowrie.dst_port").alias("dst_port"),
        F.lower(F.col("cowrie.protocol")).alias("protocol"),
        attack_category.alias("attack_category"),
        severity.alias("severity"),
        F.col("cowrie.session").alias("session_id"),
        is_login_event.alias("credentials_attempted"),
        F.when(is_login_event, F.col("cowrie.username")).alias("attempted_username"),
        F.when(is_login_event, F.col("cowrie.password")).alias("attempted_password"),
        F.when(is_file_event, F.col("cowrie.shasum")).alias("payload_hash"),
        F.col("cowrie.message").alias("description"),
        F.concat(F.col("source_type"), F.lit(":"), F.col("_record_hash")).alias("raw_ref"),
    )
    return conform_to_silver_schema(mapped)
```

- `df = bronze_df.filter((F.col("source_type") == "cowrie") & F.col("cowrie").isNotNull())`: double filter – `source_type == "cowrie"` (defensive, walaw PFA cowrie-only, el function te-check-ha explicit) w `cowrie IS NOT NULL` (row lezemha el struct `cowrie` populated, machi struct fargh/null).

- `is_login_event` w `is_file_event`: 2 boolean column expressions (`.isin(*tuple)`) mou7sbin *marra wa7da*, w mous-ta3mlin barcha marat ba3d (f `credentials_attempted`, `attempted_username`, `attempted_password`, `payload_hash`) – DRY (Don't Repeat Yourself), machi `F.col("cowrie.eventid").is` mekrar f kol place.
- `event_id`: `sha2(concat(source_type, ":", _record_hash))` – exactement kifma el docstring mte3 `schema.py` 9al.
- `event_time`: `F.to_timestamp(..., "yyyy-MM-dd'T'HH:mm:ss.SSSSSS'Z'")` – format string explicite (ISO 8601 b microseconds w `Z` literal l UTC marker), machi `F.to_timestamp` bidoun format (li y3awel 3la inference automatique, momken tefchel walla ta5ou format ghalt).
- `dst_ip`/`dst_port`: direct mel struct, *bidoun* ay "fill" walla "forward" logic (kifma et9echna fi el Soual box el fou9).
- `protocol`: `F.lower(...)` – normalisation basic (cowrie momken ye-log-ha b capitalisation mo5tlifa, "SSH" vs "ssh").
- `attempted_username`/`attempted_password`: `F.when(is_login_event, ...)` *bidoun* `.otherwise(...)` – ken `is_login_event` False, `F.when` bidoun otherwise yrajja3 `null` automatiquement (default behavior mte3 Spark). Hedhi el tari9a el as-hal ye-express "populate bark 3la certain rows, null 3la el baa9i" bidoun ma tekteb `.otherwise(F.lit(None))` explicite.
- `raw_ref`: nafs el concat expression kif `event_id` walakin *bidoun* hash (`sha2`) – unhashed, direct usable l lookup (kifma el docstring te-explique b el SQL example).
- `return conform_to_silver_schema(mapped)`: el a5er step – ye-verify eli el result ye-match `SILVER_EVENT_SCHEMA` exactement (order, types, NOT NULL constraints).

Chapter 5

transform/gold/ – el 2 tables el akhrin

Gold houwa el couche el a5ira – aggregations ready l'API/dashboard. PFA 3andha 2 gold tables bark: `attacker_profiles` (wa7ed l kol IP) w `attack_timeline` (par heure). Kol wa7da mennhom mbniya 3la `writer.py` el mouchtarek.

5.1 transform/gold/writer.py

```
Every gold table in this package is a full recompute from silver on each
run - not an incremental/merge update. ``mode("overwrite")`` makes reruns
idempotent by construction: the previous version of the table is entirely
replaced by the newly computed one, so re-running against unchanged silver
data produces a byte-identical table, and re-running after new silver data
arrives produces the correct up-to-date aggregate with no leftover or
duplicate rows from a prior run.

This trades incremental-update efficiency for simplicity and unconditional
correctness. Reasonable at ThreatLake's current data volumes; a production
system at much larger scale would likely partition-scope these overwrites
(``replaceWhere``) or move to MERGE-based incremental updates instead of
recomputing full history on every run.
```

Soual (Jury)

3lech "full recompute" (overwrite kamel) mouch "incremental update" (merge bark el data el jdida)? Mahouch wasteful, especially ken silver table tkabber?
Réponse: el docstring nafsou honest 3la had el trade-off: *"trades incremental-update efficiency for simplicity and unconditional correctness"*. El sabab el 7a9i9i (mel `attacker_profiles.py`'s docstring): aggregate metrics ki `first_seen` w `top_credentials_tried` momken ye-t2ather b *ay* event jdid l nafs el IP, *f ay wa9t* fi el history – machi bark events jdida. Ye3ni ma-fama-ch "natural incremental partition boundary" – event jdid l IP deja mawjoud momken ye-jib-lou credential jdid li ye-changi el top-5 ranking mte3ou kamel, walla event b timestamp a9dam (edge case walakin possible) ye-changi `first_seen`. Full recompute houwa el simplest way tguarantee correctness *unconditional*. El docstring ye9oul explicitement: had el trade-off ye5dem "at ThreatLake's current data volumes" – machi claim eli hedha el approach ye-scale bidoun limite; solution production akbar (`replaceWhere` partition-scoped, walla MERGE incremental) houma "future extension" mou3trafin, machi hidden limitation.

5.1.1 write_gold_table

```
def write_gold_table(
    df: DataFrame,
    table: str,
```

```
    settings: Settings | None = None,
    partition_by: list[str] | None = None,
) -> None:
    """Fully overwrite the gold Delta table ``table`` with ``df``."""
    settings = settings or get_settings()
    writer = df.write.format("delta").mode("overwrite").option("overwriteSchema", "true")
    if partition_by:
        writer = writer.partitionBy(*partition_by)
    writer.save(gold_path(table, settings))
```

- `partition_by: list[str] | None = None`: optional, machi kol gold table t7eb partitioning. `attacker_profiles` (grain wa7ed l kol IP, mahich time-series) ma ye5dmech b partitioning (ye3ayet-lha bidoun `partition_by`), `attack_timeline` (b `hour`/`date`) ye5dem `partition_by=["date"]`.
- `.option("overwriteSchema", "true")`: ye5alli el schema yetbeddel bin runs (mathalan ken tzid column jdid l aggregate) *bidoun* ma yehtej drop-and-recreate manuel mte3 el table – Delta b default ye-reject overwrite ken el schema differ, hedhi el option ye-override el behavior el hedha.
- `if partition_by: writer = writer.partitionBy(*partition_by)`: pattern "builder" – `writer` object ye-accumulate config (`.format`, `.mode`, `.option`, w conditionnellement `.partitionBy`) 9bal el `.save()` el akhir, li houwa el action el 7a9i9i.

5.2 transform/gold/attacker_profiles.py

```
GRAIN: one row per ``src_ip``. Recomputed in full from the entire silver
table on each run (see ``threatlake.transform.gold.writer``) - an
attacker's ``first_seen``, top credentials, etc. can all be affected by
any new event for that IP anywhere in history, so there is no natural
incremental partition boundary for this table.
```

- `GRAIN`: kelma el data-engineering el standard l "chnowa ye-identifier row wa7da f el table" – hna, `src_ip` wa7ed (kol IP 3andou row wa7da bark, machi wa7ed l kol event).

5.2.1 Kol column, w el 3lech mte3ou

```
Columns:
  src_ip key.
  first_seen / last_seen min/max(event_time).
  total_events count(*).
  distinct_ports_hit count(distinct dst_port).
  distinct_honeypots_hit count(distinct source_type). Always 1 in
                          PFA (cowrie-only, see ARCHITECTURE.md) -
                          kept as a real column rather than dropped,
                          since it is exactly what makes this table's
                          shape unchanged if a second source is added
                          later.
  top_credentials_tried array of up to 5 ``{username, password,
                          count}`` structs, this IP's most frequent
                          credential pairs, ranked by count desc. Only
                          counts rows with both attempted_username AND
                          attempted_password non-null. Empty array
                          (not null) for an IP with no credential
                          attempts.
  geo ``{country, city, latitude, longitude, asn,
                          asn_org}``, looked up ONCE per distinct src_ip
                          via a GeoEnricher, not once per event.
```

Soual (Jury)

3lech `distinct_honeypots_hit` ye-b9a column real f PFA, walaw dima 1 (source wa7ed bark: cowrie)? Mahouch redundant/wasteful?
Réponse: mel docstring direct – houwa *"exactly what makes this table's shape unchanged if a second source is added later"*. Ye3ni: 9rar design forward-looking – ken tzid source jdid (dionaea mathalan) l PFA fi el moustaqbal, `attacker_profiles` table *ma-tehtejch* migration walla schema change 7atta wa7ad – el column deja mawjoud, just el values ye-badlou min 1 l 1-4. Hedha ye5alli el schema stable 3ber evolution mte3 el scope, machi tzid columns ba3d kol source jdid.

Soual (Jury)

3lech `top_credentials_tried` array fargha (`[]`), mouch `null`, l IP li ma-3andou-ch 7atta credential attempt wa7ed?
Réponse: mel docstring: *"Empty array (not null) for an IP with no credential attempts"*. Hedha 9rar API/consumer-friendly – ken el column tkoun `null`, kol consumer (dashboard, API endpoint) lezmou special-case `if credentials is not None else []` 9bal ma ye-iterate 3liha. B empty array dima, el consumer ye9dar ye-loop 3la `top_credentials_tried` mba-cher (`for c in profile.top_credentials_tried`) bidoun 7atta null-check – l IP li 3andou 0 attempts, el loop just ma-ye-iterate 3la 7atta wa7ad, bidoun erreur.

5.2.2 _TOP_CREDENTIALS_LIMIT w _EMPTY_CREDENTIALS_TYPE

```
_TOP_CREDENTIALS_LIMIT = 5
_EMPTY_CREDENTIALS_TYPE = "array<struct<username:string,password:string,count:bigint>>"
```

`_EMPTY_CREDENTIALS_TYPE`: string mte3 Spark SQL DDL type (machi Python type) – mous-ta3mla ba3dein f `F.array().cast(...)` bech t5al9 array fargh *b el type el correct exactement* (struct b 3 fields specifiques), machi array fargh generic (li Spark momken ye-infer-ha b type ghalt walla `void` type).

5.2.3 _top_credentials_by_ip – top-N par groupe

```
def _top_credentials_by_ip(df: DataFrame) -> DataFrame:
    """One row per src_ip: its top _TOP_CREDENTIALS_LIMIT (username, password) pairs by count."""
    creds = df.filter(
        F.col("attempted_username").isNotNull() & F.col("attempted_password").isNotNull()
    )
    counted = creds.groupBy("src_ip", "attempted_username", "attempted_password").agg(
        F.count(F.lit(1)).alias("cred_count")
    )
    ranked = counted.withColumn(
        "rank",
        F.row_number().over(
            Window.partitionBy("src_ip").orderBy(
                F.col("cred_count").desc(), F.col("attempted_username")
            )
        ),
    ).filter(F.col("rank") <= _TOP_CREDENTIALS_LIMIT)

    return (
        ranked.groupBy("src_ip")
        .agg(
            F.sort_array(
                F.collect_list(F.struct("cred_count", "attempted_username", "attempted_password")),
                asc=False,
            ).alias("_ranked")
        )
        .withColumn(
            "top_credentials_tried",
            F.transform(
```

```
            F.col("_ranked"),
            lambda s: F.struct(
                s["attempted_username"].alias("username"),
                s["attempted_password"].alias("password"),
                s["cred_count"].alias("count"),
            ),
        ),
    )
    .drop("_ranked")
)
```

- `creds = df.filter(... isNotNull() & ... isNotNull())`: filter l'awel – bark rows b credentials 7a9i9iyin (zouz mel 2: username *w* password mawjoudin).
- `counted = creds.groupBy("src_ip", "attempted_username", "attempted_password").agg(F.count(.` group by 3 columns (IP + username + password) – yentej "kaddech marra hedha el IP 7awel had el credential pair exactement".
- El "top-N per group" pattern – hedha el pattern el classic Spark l "top N par IP": `F.row_number().over(Wind` y7ott rank (1, 2, 3...) l kol credential pair *jowa* kol IP (partition), ordoné b `cred_count desc` (el akthar frequent el awel). `F.col("attempted_username")` ki tie-breaker (secondary sort key) ye5alli el result deterministe ken zouz credentials 3andhom nafs el count exactement (bidoun-ha, Spark momken yerja3 ordre mo5tlef bin runs 3la nafs el data, 3la 5atr distributed execution).
- `.filter(F.col("rank") <= _TOP_CREDENTIALS_LIMIT)`: ye5alli bark el top-5 (rank 1 l 5) l kol IP.
- `F.collect_list(F.struct(...))` + `F.sort_array(..., asc=False)`: ba3d ma el filter ye5alli el top-5 bark, el `groupBy("src_ip")` el theni ye-collect-hom f array wa7ed (`collect_list`), w `sort_array` ye-re-sort-hom (defensive – `collect_list` ma-guarantee-ch ordre stable 3la distributed data, fa el sort el a5er hna houwa el guarantee el 7a9i9i eli el array output ordoné b `cred_count desc`).
- `F.transform(F.col("_ranked"), lambda s: F.struct(...))`: `F.transform` houwa el equivalent Spark mte3 Python's `map()` 3la array column – ye-apply lambda 3la kol element mte3 el array (`s`), y-rename el fields (`cred_count` → `count`) l el shape el akhir el mou9arrar (`username`, `password`, `count`).
- `.drop("_ranked")`: yne99i el column intermediate (esm b underscore, convention l "internal/temp column") ba3d ma `top_credentials_tried` tetbena mennha.

Analogy

El pattern "row_number over Window.partitionBy(...).orderBy(...), ba3d filter rank $\leq$ N" houwa el "top-N per group" el classic f SQL/Spark – kifha t7eb ta5ou "top 3 étudiants f kol classe": partitionBy("classe"), orderBy("note" desc), row_number, w ba3d filter rank ≤ 3. Nafs el fikra hna, walakin "classe" = `src_ip` w "note" = `cred_count`.

5.2.4 build_attacker_profiles

```
def build_attacker_profiles(
    silver_df: DataFrame,
    spark: SparkSession,
    geo_enricher: GeoEnricher | None = None,
) -> DataFrame:
    """Aggregate ``silver_df`` into the one-row-per-src_ip grain described above."""
    base = silver_df.filter(F.col("src_ip").isNotNull())

    profiles = base.groupBy("src_ip").agg(
```

```
        F.min("event_time").alias("first_seen"),
        F.max("event_time").alias("last_seen"),
        F.count(F.lit(1)).alias("total_events"),
        F.countDistinct("dst_port").alias("distinct_ports_hit"),
        F.countDistinct("source_type").alias("distinct_honeypots_hit"),
        F.sort_array(F.collect_set("attack_category")).alias("attack_categories"),
    )

    profiles = profiles.join(_top_credentials_by_ip(base), on="src_ip", how="left").withColumn(
        "top_credentials_tried",
        F.coalesce(F.col("top_credentials_tried"), F.array().cast(_EMPTY_CREDENTIALS_TYPE)),
    )
```

- `base = silver_df.filter(F.col("src_ip").isNotNull())`: filter l'awel – IP null (event bidoun source IP m3aruf) ma-y9darch yetgroupé (`groupBy("src_ip")` 3la null ye5dem, walakin yentej row `"unknown"` li mahouch useful/meaningful).
- `groupBy("src_ip").agg(...)`: el aggregation el asasi – `F.min`/`F.max("event_time")` l first/last seen, `F.count(F.lit(1))` l total events (`F.lit(1)` machi `F.col("x")` bech el count ma-y-affecté-ch b nulls f column specifique), `F.countDistinct` l distinct ports/honeypots, w `F.sort_array(F.collect_set(...))` l categories (distinct + sorted, deterministe).
- `.join(_top_credentials_by_ip(base), on="src_ip", how="left")`: LEFT join, machi INNER – 3la 5atr IP momken ma-3andouch 7atta credential attempt (kol events mte3ou homa connections walla commands bark), w lezem yeb9a f el result (b `top_credentials_tried = null` 9bal el coalesce).
- `F.coalesce(F.col("top_credentials_tried"), F.array().cast(_EMPTY_CREDENTIALS_TYPE))`: hna el "null → empty array" conversion el discuté f el Soual box el fou9 – `coalesce` yrajja3 l'awel valeur non-null, fa IP bidoun credentials (null mel LEFT join) ye5dou empty array el typed correctement.

5.2.5 geo struct – b enricher walla bidoun

```
    if geo_enricher is not None:
        profiles = enrich_geo(profiles, spark, geo_enricher, ip_column="src_ip")
        profiles = profiles.withColumn(
            "geo",
            F.struct(
                F.col("geo_country").alias("country"),
                F.col("geo_city").alias("city"),
                F.col("geo_latitude").alias("latitude"),
                F.col("geo_longitude").alias("longitude"),
                F.col("geo_asn").alias("asn"),
                F.col("geo_asn_org").alias("asn_org"),
            ),
        ).drop("geo_country", "geo_city", "geo_latitude", "geo_longitude", "geo_asn", "geo_asn_org")
    else:
        # A present struct with null fields, not a null struct: this must
        # match the shape enrich_geo itself produces for an IP it can't
        # resolve, so `geo.latitude` etc. is always safe to select
        # downstream regardless of whether a GeoEnricher was supplied.
        profiles = profiles.withColumn(
            "geo",
            F.struct(
                F.lit(None).cast("string").alias("country"),
                F.lit(None).cast("string").alias("city"),
                F.lit(None).cast("double").alias("latitude"),
                F.lit(None).cast("double").alias("longitude"),
                F.lit(None).cast("int").alias("asn"),
                F.lit(None).cast("string").alias("asn_org"),
            ),
        )
```

Soual (Jury)

Comment el code ye9oul "must match the shape enrich_geo itself produces" – 3lech had el matching exact houwa lezem, machi just "good practice"?
Réponse: ken `geo_enricher` houwa `None`, el code ma-y-3addich `geo` column simplement (`null` kamel) – houwa yebni struct `present` b kol el 6 fields `null` *jowa*. El sabab: kol consumer downstream (API, dashboard) ye5dem b syntax `profile.geo.latitude` walla `profile["geo"]["latitude"]`. Ken `geo` kan `null` (machi struct b fields null), had el access ye-crash (`None` ma-3andou-ch attribute `.latitude`) – consumer lezmou special-case 9bal ma ya3mel access. B struct present dima (b nulls jowa ken ma fama-ch geo data), `profile.geo.latitude` dima "safe" syntax-wise, sawa el IP 3andou geo data 7a9i9i sawa la – el difference bark houwa el value (coordonnée 7a9i9iya vs `null`), machi el "shape" nafsou.

Analogy

Hedhi kifha template mte3 formulaire fargh – 7atta ken ma-3andekech information ("city": unknown), el formulaire nafsou ye-b9a m3andou kol el champs (name, city, country...), just mel-i-in b "N/A" walla fargh. Ma tsawwarch formulaire kamel yb9a "missing" just 3la 5atr wahda mel champs ma3andekech-ha – structure el formulaire dima consistante.

5.2.6 write_attacker_profiles

```python
def write_attacker_profiles(
    spark: SparkSession,
    settings: Settings | None = None,
    geo_enricher: GeoEnricher | None = None,
) -> DataFrame:
    """Read silver, build attacker_profiles, and idempotently overwrite the gold table."""
    silver_df = spark.read.format("delta").load(silver_path("events", settings))
    profiles = build_attacker_profiles(silver_df, spark, geo_enricher)
    write_gold_table(profiles, "attacker_profiles", settings)
    return profiles
```

3 steps clairs: `spark.read.format("delta").load(...)` (ta9ra el silver table kamla mel disque), `build_attacker_profiles(...)` (el aggregation pure, li chefna kamla fou9), w `write_gold_table(...)` (el write idempotent, mel `writer.py`). `return profiles`: yrajja3 el DataFrame zeda (machi `None`) – useful l tests (verify el content bidoun ma tre-read mel disque) w l orchestration scripts (`run_pipeline.py`) li 7ebbin y-print row counts mba-cher.

5.3 transform/gold/attack_timeline.py

```
GRAIN: one row per ``(hour, attack_category, dst_port)``. ``hour`` is
``event_time`` truncated to the top of the hour (UTC - every timestamp in
this project is).

This is the finest-grained time-series gold table: rolling up to
``(hour, attack_category)`` or ``(hour, dst_port)`` alone is a single
``groupBy`` away from this table. The reverse - recovering the port
breakdown from a category-only rollup - is not, so this grain is kept rather
than pre-aggregating a dimension away.

``dst_port`` is nullable and kept that way: rows for application-layer
events with no destination context (e.g. a Cowrie command-input event) group
together under a null port rather than being dropped or assigned an
invented sentinel value.
```

Soual (Jury)

3lech el grain el m5tar houwa (hour, category, port), mouch category bark walla port bark – 3lech "finest-grained" el ekhtiyar?
Réponse: mel docstring direct – *"rolling up … is a single groupBy away from this table. The reverse … is not"*. Hedha principe important f data modeling: dima 5tar el grain *el akthar fine* li reasonable, 3la 5atr ta9dar dima "t-aggregate l fou9" (rollup, groupBy sghir) mel grain fine, walakin ma-t9darch "t-disaggregate l ta7t" (mel grain gros l fine) 3la 5atr el information deja tlefet f el aggregation el oula. Ken el table 3andha bark `(hour, category)`, ma-3andekch tari9a terja3 l breakdown mte3 ports mel data hedhi – lezmek te5dem query jdid mel silver mel awel. B `(hour, category, port)` el fine grain, kol dashboard query (category bark, port bark, walla el zouz) ye5dem b groupBy basic 3la had el table wa7da.

5.3.1 build_attack_timeline

```python
def build_attack_timeline(silver_df: DataFrame) -> DataFrame:
    """Aggregate ``silver_df`` into the hourly (category, port) grain described above."""
    base = silver_df.filter(F.col("event_time").isNotNull())

    timeline = base.groupBy(
        F.date_trunc("hour", F.col("event_time")).alias("hour"),
        F.col("attack_category"),
        F.col("dst_port"),
    ).agg(
        F.count(F.lit(1)).alias("event_count"),
        F.countDistinct("src_ip").alias("distinct_src_ip_count"),
    )

    return timeline.withColumn("date", F.to_date(F.col("hour"))).select(
        "date", "hour", "attack_category", "dst_port", "event_count", "distinct_src_ip_count"
    )
```

- `base = silver_df.filter(F.col("event_time").isNotNull())`: filter l'awel – row bidoun `event_time` confirmé (chefna f `schema.py` eli `event_time` nullable) ma-t9darch tetgroupé b heure, fa te5rej mel timeline hedhi (mahich lost – raw_ref dima ye5allik terja3-lha).
- `F.date_trunc("hour", F.col("event_time"))`: ye-truncate el timestamp l a3la sa3a (math-alan `14:37:22` → `14:00:00`) – hedhi el "bucketing" el time-series el standard.
- `groupBy(hour, attack_category, dst_port).agg(...)`: el grain el 3 dimensions, kifma et9echna.
- `F.count(F.lit(1))`: total events fi had el bucket. `F.countDistinct("src_ip")`: kaddech IP mo5telfa 7awlou f had el bucket (useful l "houwa attacker wa7ed kbir walla barcha IPs mo5tlifin?").
- `.withColumn("date", F.to_date(F.col("hour")))`: yzid column `date` zeda (bidoun heure) – houwa el partition column el mous-ta3mla f `write_attack_timeline` (`partition_by=["date"]`) – partitioning b `date` (machi `hour`, li ykoun barcha partitions sghar barcha) houwa el balance el pratique bin "efficient date-range queries" w "not too many tiny partitions".

5.3.2 write_attack_timeline

```python
def write_attack_timeline(spark: SparkSession, settings: Settings | None = None) -> DataFrame:
    """Read silver, build attack_timeline, and idempotently overwrite the gold table."""
    silver_df = spark.read.format("delta").load(silver_path("events", settings))
    timeline = build_attack_timeline(silver_df)
    write_gold_table(timeline, "attack_timeline", settings, partition_by=["date"])
    return timeline
```

Nafs el pattern kif `write_attacker_profiles` – read, build, write. El difference el wa7ida: `partition_by=["date"]` ye3addi explicite hna (contra `attacker_profiles` li ma3andou-ch partitioning – grain mte3ou mahouch time-series).

Chapter 6

ml/ – el 2 detectors

PFA 3andha 2 detectors ye5edmou b3adhom (mouch wa7ed instead el theni): IsolationForest (unsupervised, trained 3la silver features) w rule fixe (port scan). Kol wa7ed 3andou strengths/limitations mo5tlifin, w `score_events.py` y-combine-hom.

6.1 ml/features.py

```
Every feature here is TRAILING-WINDOW: for a given event, "how much
activity has this src_ip generated in the ``window_seconds`` before (and
including) this event?" - not a whole-batch total. A whole-batch total
can't tell "20 ports touched in one burst" from "20 ports touched over a
week"; a trailing window can, and it lets the exact same feature
computation run unchanged whether scoring one hour of silver data or one
year of it.

Implemented with a single Spark window function per feature -
``Window.partitionBy("src_ip").orderBy(event_time_unix).rangeBetween(-window_seconds, 0)`` -
no join, no self-union, no UDF: every row's window is computed directly
against that same src_ip's other rows, ordered by time.
```

Analogy

"Trailing window" kifha t-check-i el historique mte3 chkoun li da5al l immeuble f a5er 60 secondes, machi el historique *kamel* mel bidaya mte3 el yaum. Kol marra 7ad ye-swipe badge mte3ou (event jdid), ta3mel "combien de personnes ont badgé dans les 60 dernières secondes?" – mahiche "combien depuis ce matin?". Hedhi el window ye5allik ta3rf "burst" (barcha activité f wa9t 9sir) contra activité normale mfarr9a 3la yaum kamel, 7atta ken el 2 3andhom *nafs el total* 3la el yaum kamel.

Soual (Jury)

3lech IsolationForest lezmou trailing-window features (mahouch raw event fields direct), w 3lech window wa7ed (60s) mouch barcha windows (1min, 5min, 1h...) kif barcha systems o5rin?
Réponse: IsolationForest (w rule fixe zeda) ye7taj signal 3la "*comportement*" el attacker, machi 3la event wa7ed b rou7ou. Event wa7ed (`login.failed` mathalan) houwa normal 7atta – barcha bots automated ye3mlou login attempts 3ala kol honeypot. Chnowa *anormal* houwa el *fréquence* – 20 login attempts f 60 secondes mel nafs el IP houwa signal brute-force, 20 login attempts mfarr9a 3la yaum kamel houwa noise 3adi. Window wa7da bark (60s, `DEFAULT_WINDOW_SECONDS`) houwa scope minimal 3adéquat: PFA 3andha 2 attack shapes bark ye7taj ye-detecter-hom (brute-force burst, port scan burst), w el 2 homa "burst" patterns

short-term nisbiyan. Barcha windows (multi-scale) yzid complexity (barcha features, barcha training dimensions) bidoun benefit clair l had el scope el sghir.

6.1.1 DEFAULT_WINDOW_SECONDS w ANOMALY_FEATURE_NAMES

```python
DEFAULT_WINDOW_SECONDS = 60

ANOMALY_FEATURE_NAMES: tuple[str, ...] = (
    "events_by_src_ip_in_window",
    "distinct_dst_ports_touched_by_src_ip_in_window",
    "login_attempts_by_src_ip_in_window",
    "is_login_attempt",
)
```

- Comment fou9 `ANOMALY_FEATURE_NAMES` (mel code el 7a9i9i) ye9oul clair: *"Order matters: scikit-learn has no column-name concept, it trusts positional order"* – hedhi el tafsila el critique. scikit-learn's `IsolationForest.fit()` ta5ou `numpy` array (walla pandas DataFrame, mel-i converti l array internally) – ma-3andou-ch concept mte3 "esm el column", bark position. Ken el features t-passiw f ordre mo5tlef bin training (`train_anomaly.py`) w scoring (`score_events.py`), el model ye-interpret feature 1 kifha feature 2 – *silently ghalt*, bidoun 7atta erreur.
- `ANOMALY_FEATURE_NAMES` houwa el "single source of truth" l had el ordre – kol wa7ed mel 2 modules (`train_anomaly.py`, `score_events.py`) ye-import-ha, machi ye-hardcode-ha separately.
- 4 features, kol wa7da 3andha role clair (mel comment el kbir): `events_by_src_ip_in_window` w `login_attempts_by_src_ip_in_window` l brute-force (barcha events/logins f wa9t 9sir), `distinct_dst_ports_touched_by_src_ip_in_window` l port scan (barcha ports *mo5tlifin* f wa9t 9sir, machi nafs el port marat 3adda), w `is_login_attempt` (per-event, machi per-window) y-refine el 2 patterns.

6.1.2 FeatureValidationError

```python
class FeatureValidationError(RuntimeError):
    """Raised when a feature DataFrame's columns don't match what a model expects."""
```

Exception custom o5ra, nafs el pattern el mou3tad fi kol el codebase.

6.1.3 build_features – el core

```python
def build_features(
    silver_df: DataFrame, window_seconds: int = DEFAULT_WINDOW_SECONDS
) -> DataFrame:
    """Compute :data:`ANOMALY_FEATURE_NAMES` for every silver event that has
    both a src_ip and an event_time - the two columns every feature here
    is computed relative to. Rows missing either can't be attributed to an
    attacker's trailing activity and are dropped.

    Keeps ``event_id`` and ``src_ip`` alongside the features so callers can
    join scores back to the events they were computed for, and so
    ``threatlake.ml.rules.port_scan_rule`` can be applied per-row without a
    second pass over silver.
    """
    base = silver_df.filter(F.col("src_ip").isNotNull() & F.col("event_time").isNotNull())

    event_time_unix = F.unix_timestamp(F.col("event_time"))
    trailing_window = (
        Window.partitionBy("src_ip").orderBy(event_time_unix).rangeBetween(-window_seconds, 0)
    )
    # credentials_attempted is a NOT NULL boolean column (see
```

```
    # threatlake.transform.silver.schema) - always true/false, safe to
    # cast straight to 0/1.
    is_login_attempt = F.col("credentials_attempted").cast("int")

    return base.select(
        "event_id",
        "src_ip",
        F.count(F.lit(1)).over(trailing_window).alias("events_by_src_ip_in_window"),
        # collect_set (used here as a window aggregate) ignores nulls, so
        # events with no dst_port (e.g. a cowrie.command.input event) simply
        # don't contribute a port to the count, rather than counting as a
        # fake "null port".
        F.size(F.collect_set("dst_port").over(trailing_window)).alias(
            "distinct_dst_ports_touched_by_src_ip_in_window"
        ),
        F.sum(is_login_attempt).over(trailing_window).alias("login_attempts_by_src_ip_in_window"),
        is_login_attempt.alias("is_login_attempt"),
    )
```

- `base = silver_df.filter(... src_ip isNotNull & event_time isNotNull)`: filter l'awel – kifma el docstring te9oul, kol feature hna mou7sba *relative* l `src_ip` (partition key) w `event_time` (order key), fa row bidoun 7ad mel 2 ma-t9darch tetgroupé fi window 7a9i9i.
- `event_time_unix = F.unix_timestamp(F.col("event_time"))`: Spark's `rangeBetween` (contra `rowsBetween`) ye5dem 3la *values numériques* (secondes), mouch 3la row-count – lezmou timestamp mou7awel l Unix epoch seconds (integer) bech `-window_seconds` l `0` ykoun range clair (secondes, machi rows).
- `Window.partitionBy("src_ip").orderBy(event_time_unix).rangeBetween(-window_seconds, 0)`: el window definition el kbira – `partitionBy("src_ip")` (kol IP 3andou window separate mel IPs el o5rin), `orderBy(event_time_unix)` (rows ordoné b wa9t jowa kol partition), `rangeBetween(-window_seconds, 0)` (mel `window_seconds` 9bal el row el actuel, l el row nafsou inclus – *"trailing", including current"*).
- `is_login_attempt = F.col("credentials_attempted").cast("int")`: comment inline ye-explique el confidence hna – `credentials_attempted` houwa NOT NULL boolean (mel `SILVER_EVENT_SCHEMA` li chefna), fa `.cast("int")` dima ye5dem b confidence (0 walla 1, 9att `null`).
- `F.count(F.lit(1)).over(trailing_window)`: window aggregate *machi* groupBy aggregate – `.over(...)` y-apply el aggregation 3la kol row *bidoun ma tne99es* el row count (contra `groupBy` li y-collapse el rows). Kol row t5rej b `events_by_src_ip_in_window` mte3ha el 5assa, based 3la el window mte3ha.
- `F.size(F.collect_set("dst_port").over(trailing_window))`: `collect_set` (window aggregate) ye5ammel el *distinct* values mte3 `dst_port` jowa el window f set, `F.size` ye7seb el cardinality. Comment inline ye-explique el edge-case: `collect_set` ignore nulls automatiquement (behavior mte3 Spark), fa events bidoun `dst_port` (command.input mathalan) ma-y-contribuwch port fake l el count.
- `F.sum(is_login_attempt).over(trailing_window)`: sum mte3 el 0/1 flag 3la el window – "kaddech login attempt f had el window".
- `is_login_attempt.alias("is_login_attempt")`: hedhi el feature el wahida el *mahiche* window aggregate – houwa el value mte3 el row nafsou (per-event, machi trailing).

6.1.4 validate_feature_frame

```
def validate_feature_frame(
    columns: list[str], expected: tuple[str, ...] = ANOMALY_FEATURE_NAMES
) -> None:
    """Raise if ``columns`` is missing any column in ``expected``.
```

```
    Guards the model-input boundary: threatlake.ml.train_anomaly and
    threatlake.ml.score_events both call this right before handing a
    pandas frame to scikit-learn, so a feature-computation bug surfaces as
    an immediate, specific error instead of a silently wrong model.
    """
    missing = [name for name in expected if name not in columns]
    if missing:
        raise FeatureValidationError(
            f"Feature frame is missing expected columns: {missing}. Got: {columns}."
        )
```

Nafs el falsafa el "fail loud, fail early" li nchoufouha 3ber el codebase kamel (kif `_assert_not_null` f `silver/schema.py`) – guard clause mou3ayen 9bal el boundary el critique (hna: passing data l scikit-learn), machi assumption implicite.

6.2 ml/rules.py

```
WHY A RULE, GIVEN A TRAINED MODEL ALREADY EXISTS: threatlake.ml.train_anomaly's
"KNOWN LIMITATION" section documents a specific, verified failure mode - on the
project's injection test corpus, IsolationForest's ``contamination`` parameter
fixes a hard budget on how many events can be flagged per batch, and a more
extreme pattern (credential brute-force) crowded out a less extreme but still
real one (port scan) for most of that budget, even after
threatlake.ml.features.ANOMALY_FEATURE_NAMES gave the model a fan-out feature
that separates port scans from normal traffic almost perfectly on its own.
A fixed threshold rule has no such budget: it fires per-event, independent of
what else is in the same batch, so it isn't subject to that crowding-out
failure mode. It is also not a replacement for the model - it has no answer
for anomaly patterns nobody has written a rule for yet, which is the entire
reason threatlake.ml.train_anomaly exists.
```

Soual (Jury)

Hedhi el Soual el akthar muhimma f el chapter el ML kamel: 3lech zouz detectors (ML + rule) mahouch wa7ed bark? Mahouch redundant?
Réponse: docstring ye-documente failure mode *verified* 7a9i9i, machi theoretical – 3la corpus test (injection corpus) mte3 el project, IsolationForest's `contamination` parameter (mathalan 0.02, ye3ni "2% mel events houma anomalies") ye7ott *hard budget*: ken el batch 3andou brute-force pattern (extreme, barcha events "weird" f wa9t 9sir) w port-scan pattern (real, walakin less extreme relatively), el brute-force ye5ou mou3zam el "budget" mte3 el 2%, w el port-scan ye-b9a *under-flagged* – *7atta ken* el model 3andou feature (`distinct_dst_ports_touched_by_src_ip_in_window`) li ye-separate el 2 patterns clairement. Hedha houwa el "crowding-out" mentioned. Rule fixe (threshold-based) ma-3andou-ch had el budget – ye-fire per-event, independamment 3la chnowa fama f el batch el nafsou. Fa el 2 detectors 3andhom *failure modes mo5tlifa*: model ye-cover patterns *ma-3arafhomch-ch* 7ad w ma-kteb-lhomch rule (el "unknown unknowns"), rule ye-cover el pattern el m3aruf (port scan) b guarantee (ma-ye-crowd-out-ch). Kombinaison-hom ye5alli coverage a7sen mel 7ad mennhom bark.

Analogy

Kifha kif security team 3andha zouz layers: camera b AI detection (ye-learn patterns "weird" automatiquement, walakin momken yekhtar el akthar dramatic event ken 3andou "alert budget" limité) + alarm fixe 3la sensor precis (mathalan "ken 3adad el bebs el maftou7in $>$ 5 f nafs el wa9t, do99" – simple, deterministe, ma-yetathrouch b chnowa el AI rah ta3mel f nafs el moment). El 2 sawa ye5allik ma-tfoutch 7ata pattern, sawa el AI "distracted" b chay o5or akthar dramatique.

6.2.1 PORT_SCAN_DISTINCT_PORT_THRESHOLD

```
#: Chosen from the observed separation in the project's injection test
#: corpus (threatlake.ml.train_anomaly's KNOWN LIMITATION section, same
#: corpus, same 60s window): every one of the 565 ordinary events in that
#: corpus has this feature exactly equal to 1 (including brute-force events,
#: which hammer a single port repeatedly); every injected port-scan event
#: reaches up to 8. 5 is a threshold with headroom on both sides of that
#: gap - well above the "1" normal traffic never exceeds, well below the
#: observed "8" ceiling of an actual scan - rather than a value fit exactly
#: to this dataset (e.g. 2, which would perfectly split 1 from 8 here but
#: is a description of this specific corpus, not a general claim about what
#: "many distinct ports" means for honeypot traffic).
PORT_SCAN_DISTINCT_PORT_THRESHOLD = 5
```

Soual (Jury)

3lech 5 el threshold, mouch 2 (li "y-split perfectement" el 2 groupes 1 vs 8 fi el corpus mte3 test)?
Réponse: hedha el comment el akthar transparent f el codebase kamel 3la "kifeh 9rart 3la magic number" – w houwa answer direct l had el soual nafsou. Ken el threshold ykoun 2 (li y-split "perfectement" el data el observed), el number ykoun *overfit* 3la el corpus el test el specifique – mahouch generalizable l data jdida (real honeypot traffic, li momken 3andou distribution mo5talfa chwaya). B 5 (headroom fi el zouz jihat: 5 3la 5atr "well above the 1 normal traffic never exceeds" w "well below the observed 8 ceiling"), el threshold houwa *robust* l variation naturelle f el data, machi calibré exactement 3la data particulière. El comment ye9oul explicitement: 2 "is a description of this specific corpus, not a general claim" – distinction muhimma bin "chnowa el data hedhi t9oul" w "chnowa el rule el general el correcte".

6.2.2 port_scan_rule

```
def port_scan_rule(
    distinct_dst_ports_touched: int, threshold: int = PORT_SCAN_DISTINCT_PORT_THRESHOLD
) -> bool:
    """True if a src_ip touched more than ``threshold`` distinct dst_ports in
    the trailing window - see :data:`PORT_SCAN_DISTINCT_PORT_THRESHOLD` for
    where the default comes from.
    """
    return distinct_dst_ports_touched > threshold
```

Function pure, ligne wa7da – takes `distinct_dst_ports_touched` (el feature el mou7sba f `build_features`) w yerja3 boolean. Simplicité intentionnelle: el "rule" kamla houwa comparison wa7da, deterministe, predictable, testable b ligne wa7da (`assert port_scan_rule(8) is True`) – 3ks el model ML li houwa "black box" nisbiyan.

6.3 ml/train_anomaly.py

```
Live honeypot data has no ground-truth label - there is no analyst
sitting behind cowrie tagging each session "attack" or "not attack" - so
this module never fits a supervised model and never reports a supervised
accuracy/F1 figure. IsolationForest is unsupervised: it scores how easily
a point is isolated by random feature splits, under the assumption that
anomalies are "few and different" and so isolate in fewer splits than
normal points.
```

Analogy

IsolationForest kifha t7eb tal9a "el chakhs el ghalt" f foule mel 3ibad bidoun ma ta3raf mel-9bal chkoun houwa – houwa el chakhs eli "as-hal te-isole-h" (ta9dar te-separate-h mel groupe b su2al wa7ed walla zouz: "houwa lébès a7mar?", "houwa twil?"...), 3la 5atr houwa *different* mel majorité. El 3ibad el "normal" (mcheb-hin ba3dhom) ye7tajou barcha su2al (splits) bech te-isole wa7ed mennhom mel groupe, 3la 5atr homa mtabhin barcha m3a ba3dhom. Hedha houwa el fikra: anomaly = as-hal ye-isole = 9rib mel root mte3 el decision tree.

Soual (Jury)

3lech IsolationForest *unsupervised* (ma ye5demch b labels) houwa el ekhtiyar el m3a9oul hna, w mouch supervised classifier (mathalan XGBoost 3la labeled attacks)?
Réponse: docstring ye9olha directly – *"Live honeypot data has no ground-truth label"*. Ma-fama-ch analyst 7a9i9i 9a3ed ye-tag kol session "attack" walla "not attack" f real-time – el honeypot ye-log kol chay li yousal-lou (kollou "attack" f sense generic, honeypot ma3andou-ch "legitimate users" 7a9i9iyin). Fa training supervised classifier lezmou labels 7a9i9iyin – w bidoun-hom, ay label momken tzid-ha (mathalan "heuristic labeling" 3la eventid) ye-b9a biased/circular (ta3mel model ye-learn el heuristic nafsou, machi signal jdid). IsolationForest ye5dem bidoun labels 7atta wa7ed – assumption mte3ou el wahid houwa "anomalies are few and different", w hedhi assumption reasonable l honeypot traffic (mou3zam el traffic houwa noise/bots automated mtabhin, el attacks 7a9i9iyin/sophistiqués homa rare w mo5talfin).

```
PERSISTENCE: a plain joblib file (``settings.ml.model_path``), not an
MLflow model registry. ThreatLake AI (the full project this is a subset
of) tracks every run through MLflow - parameters, metrics, a registered
model with staged versions - which is real, useful machinery for a system
with a retraining schedule and multiple models competing for a
"Staging"/"Production" slot. PFA trains one model, once, for one batch
pipeline run; a single file scikit-learn's own ``joblib.dump``/``load``
round-trips is the whole story, and is what a from-scratch reader can
open and reason about without also learning MLflow's tracking/registry
API.
```

Soual (Jury)

3lech joblib mouch MLflow, walaw MLflow houwa el "standard" industriel l ML model management?
Réponse: 9rar mou9asad, w el docstring transparent 3la el trade-off (kifma etl9inha f zouz places o5rin f hedha el document): MLflow ye5dem real value ken 3andek *retraining schedule* (train kol yaum/jom3a, compare versions) w *multiple models* li ye-compétionw 3la "Staging"/"Production" slot (deployment strategy). PFA 3andha use-case as-hal bezef: train *marra wa7da*, l *batch pipeline run wa7ed*. MLflow f had el context ye-zid complexity (tracking DB, registry API, concepts jdida l fahmemhom) bidoun benefit 7a9i9i – persistence file basic (joblib) ye5dem exactement nafs el chay el mou7taj (save model, load-ha ba3dein l scoring), w kol wa7ed ya9ra el code (jury mathalan) ye9dar yefhem kif ye5dem bidoun ma yet3allem MLflow API l'awel. Hedha exemple 7a9i9i mte3 "YAGNI" (You Aren't Gonna Need It) applied bien – simplicité proportionnelle l el scope el 7a9i9i.

6.3.1 _RANDOM_STATE w AnomalyTrainingResult

```
_RANDOM_STATE = 42


@dataclass(frozen=True)
class AnomalyTrainingResult:
```

```
    """Outcome of one IsolationForest fit."""

    n_events: int
    n_flagged: int
    contamination: float
    model_path: str

    def summary(self) -> str:
        rate = self.n_flagged / self.n_events if self.n_events else 0.0
        return (
            f"events={self.n_events} flagged={self.n_flagged} "
            f"({rate:.1%}) contamination_param={self.contamination} "
            f"model_path={self.model_path}"
        )
```

- `_RANDOM_STATE = 42`: seed fixe (magic number el classique f el ML world, "the answer to life, the universe, and everything"). IsolationForest ye5dem b random splits internally – seed fixe ye5alli el training *reproductible*: nafs el data → nafs el model exactement, kol marra. Bidounha, kol training run yentej model chwaya mo5telef, w tests/debugging ye-b9aou s3ib (*"kifeh el bug hedha, el model deja mo5telef bin runs?"*).
- `AnomalyTrainingResult`: nafs pattern dataclass frozen kif `BronzeWriteResult`. `summary()` method: yentej string human-readable, useful l orchestration scripts (`run_pipeline.py`) li 7ebbin y-print result 3la terminal bidoun ma yeb-nou format-ha manuellement kol marra.
- `rate = self.n_flagged / self.n_events if self.n_events else 0.0`: guard 3la division-by-zero – ken `n_events == 0` (edge case, silver table fargh mathalan), `rate` ye-b9a 0.0 machi `ZeroDivisionError`.

6.3.2 `train_anomaly` – el function el kamla

```
def train_anomaly(
    spark: SparkSession,
    settings: Settings | None = None,
    contamination: float = 0.02,
    n_estimators: int = 200,
    window_seconds: int = 60,
) -> AnomalyTrainingResult:
    """Fit IsolationForest on the current silver table's feature space and
    save it to ``settings.ml.model_path``.

    ``contamination`` is IsolationForest's own prior on the expected
    anomaly fraction, not a measured rate - it directly controls how many
    points get flagged, which is why it is reported alongside the result
    rather than presented as a finding.
    """
    settings = settings or get_settings()

    silver = spark.read.format("delta").load(silver_path("events", settings))
    feature_df = build_features(silver, window_seconds=window_seconds)
    validate_feature_frame(feature_df.columns, expected=ANOMALY_FEATURE_NAMES)

    pdf: pd.DataFrame = feature_df.toPandas()
    if pdf.empty:
        raise ValueError("Silver table produced zero feature rows - nothing to train on.")

    features_only = pdf[list(ANOMALY_FEATURE_NAMES)]

    model = IsolationForest(
        n_estimators=n_estimators,
        contamination=contamination,
        random_state=_RANDOM_STATE,
        n_jobs=-1,
    )
    model.fit(features_only)
    predictions = model.predict(features_only) # -1 = anomaly, 1 = normal
    n_flagged = int((predictions == -1).sum())

```

```
    model_path = Path(settings.ml.model_path).expanduser().resolve()
    model_path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, model_path)

    return AnomalyTrainingResult(
        n_events=len(pdf),
        n_flagged=n_flagged,
        contamination=contamination,
        model_path=str(model_path),
    )
```

Soual (Jury)

Docstring ye9oul `contamination` "is IsolationForest's own prior ... not a measured rate". Chnowa el difference, w 3lech muhimma?
Réponse: `contamination=0.02` ma-ye9olch "measured-na eli 2% mel data houma attacks 7a9i9iyin" – houwa *input* l el model, machi *output* mennou. Houwa ye9oul l IsolationForest: "esta3mel had el prior 9bal ma t9arrar el decision boundary (threshold) mel anomaly scores continues l flag/no-flag binaire". Ken thebdel `contamination` l 0.10 (10%), el *nafs el data exactement* ye5rej 5 marat akthar "flagged" events – mahouch 3la 5atr el data tbeddlet, houwa 3la 5atr threshold-na tbeddel. Hedhi el sabab el `contamination` yetrapporta "alongside the result rather than presented as a finding" (f `AnomalyTrainingResult.contamination`) – lezem ykoun clair l reader eli `n_flagged` houwa function mte3 `contamination` el mou5tar, machi discovery mte3 "kaddech attacks 7a9i9i fama".

- `spark.read.format("delta").load(silver_path("events", settings))` → `build_features` → `validate_feature_frame`: el pipeline el mou3tad – read, transform, validate 9bal ma t3addi l model.
- `pdf: pd.DataFrame = feature_df.toPandas()`: hna houwa el moment el critique win Spark DataFrame (distributed) ye-collect l pandas DataFrame (single-machine, memory). scikit-learn ma-ye5demch 3la Spark DataFrames – lezmou numpy/pandas. `.toPandas()` houwa action (kif `.collect()`) – l had el reason, el feature computation lezmou ykoun "reasonable" f size (silver events, machi bronze raw kamel).
- `if pdf.empty: raise ValueError(...)`: guard clause – IsolationForest ma-y9darch ye-train 3la 0 samples, fa el erreur el explicite (message clair "nothing to train on") ahsen mel exception generic mte3 scikit-learn eli momken ma-tbanach clear l reader.
- `features_only = pdf[list(ANOMALY_FEATURE_NAMES)]`: select el 4 feature columns bark (bidoun `event_id`, `src_ip` li deja f `pdf` mel `build_features`) – `list(...)` 3la tuple 3la 5atr pandas `.[]` indexing ye5taj list (mahouch ye9bel tuple direct fi kol el versions).
- `IsolationForest(n_estimators=200, contamination=contamination, random_state=_RANDOM_STATE, n_jobs=-1)`: `n_estimators=200` (kaddech "trees" fi el forest – akthar trees, akthar stable walakin akthar compute). `n_jobs=-1`: esta3mel kol el cores disponibles (parallelism 3la level mte3 scikit-learn, machi Spark – houwa training local 3la machine wa7da b had el noqta).
- `model.predict(features_only)`: `-1` = anomaly, `1` = normal (convention mte3 scikit-learn el standard, comment inline ye3awed-ha bech ma-tensach). `n_flagged = int((predictions == -1).sum())`: count el `-1`s.
- `model_path.parent.mkdir(parents=True, exist_ok=True)`: ye5al9 el parent directory ken ma-mawjoud-ch (nafs pattern kif `fs.move`) – `exist_ok=True` ye5alli ma-yfail-ch ken el directory deja mawjoud (idempotent).
- `joblib.dump(model, model_path)`: el persistence el 7a9i9iya – `joblib` houwa serialization

library optimisé l objects mel numpy/scikit-learn (ahsen men `pickle` el standard l had el use-case, ye5dem ahsen m3a arrays kbar).

6.4 ml/score_events.py

```
COMBINED ANOMALY DETECTION: an event is flagged (``is_anomaly=True``) if
EITHER detector fires. The two are deliberately NOT merged into
one opaque score: ``alert_source`` records exactly which one(s) fired
(``"rule"``, ``"ml"``, or ``"both"``), so a flagged event's cause is
always attributable rather than hidden behind a single number.

APPEND, NOT OVERWRITE: unlike ``attacker_profiles``/``attack_timeline``,
this table is not a full recompute - it is a growing log of scoring
results, one row per scored event. Re-running against a silver batch that
was already scored is a no-op, not a duplicate: this module reads back
the event_ids already present in ``ml_scores`` and excludes them before
scoring, so re-running the pipeline produces no new rows rather than
double-counting.
```

Soual (Jury)

3lech `alert_source` ("rule"/"ml"/"both") houwa design choice muhim, mouch just `is_anomaly` boolean wa7ed bark?
Réponse: kifma el docstring te9oul, el 2 detectors "deliberately NOT merged into one opaque score". Ken el output ykoun `is_anomaly` bark (bidoun `alert_source`), analyst SOC li ye-check el alert ma-3andou-ch idea 3lech el event flagged – houwa ML (pattern statistique li el model 3arf-ha mel training) walla rule (port scan explicite, deterministe)? Had el information critique l el analyst bech ya3raf kifeh ye-investigui (ML flag momken ykoun false positive rare, rule flag houwa deterministe w explicable direct). `alert_source` ye5alli kol alert *attributable* – traceable l el detector exact li fired-ha, machi black box.

6.4.1 SCORES_SCHEMA

```
SCORES_SCHEMA = StructType(
    [
        StructField("event_id", StringType(), False),
        StructField("scored_at", TimestampType(), False),
        StructField("anomaly_score", DoubleType(), False),
        StructField("is_anomaly", BooleanType(), False),
        StructField("alert_source", StringType(), True),
    ]
)
```

`alert_source` houwa el field el wahed nullable f had el schema – 3la 5atr event mahouch flagged (`is_anomaly=False`) ma-3andou-ch "source" mte3 alert (null valide – machich rule wala ML fired-ha).

6.4.2 ScoringResult

```
@dataclass(frozen=True)
class ScoringResult:
    """Outcome of one scoring run."""

    n_candidates: int
    n_already_scored: int
    n_newly_scored: int
    n_flagged: int

    def summary(self) -> str:
        return (
            f"{self.n_candidates} events, {self.n_already_scored} already scored, "
            f"{self.n_newly_scored} newly scored, {self.n_flagged} flagged"
```

```
        )
```

Nafs pattern dataclass frozen + `summary()` kif `AnomalyTrainingResult`. 4 counters différents (candidates, already scored, newly scored, flagged) ye5alliw el observer ya3raf exactement chnowa sar f el run (mathalan "3andi 100 candidates, 80 deja scored mel run el 9bel, 20 jdida, 3 minhom flagged").

6.4.3 _already_scored_event_ids

```python
def _already_scored_event_ids(spark: SparkSession, settings: Settings) -> set[str]:
    try:
        existing = spark.read.format("delta").load(gold_path("ml_scores", settings))
    except Exception: # table not created yet is normal, not an error
        return set()
    rows = existing.select("event_id").distinct().collect()
    return {row["event_id"] for row in rows}
```

- `try/except Exception`: pattern "catch-all" (machi exception specifique) – comment inline ye9oul l'raison: *"table not created yet is normal, not an error"*. El run el awel mte3 el pipeline (7atta wa9t ma `ml_scores` ma-t5al9et-ch 3ad), `spark.read.format("delta").load(...)` 3la path li ma-fihech Delta table ye-throw exception (Delta-specific, kif `AnalysisException`) – had el except el generic ye-catch-ha w yerja3 set fargh (*"ma-fama-ch events scored 9bal, kolchay candidate"*).
- `.select("event_id").distinct().collect()`: ya9ra bark el column `event_id` (machi el table kamla – efficient, w Delta/Parquet ye5edmou b columnar storage fa had el projection ye9lel el I/O barcha), `.distinct()` (walaw `event_id` lezmou unique mel design, defensive), `.collect()` (action – el set el akhir lezmou ykoun f Python memory bech t9dar t-esta3mel-ha b `.isin()` ba3dein).
- `return {row["event_id"] for row in rows}`: set comprehension – $O(1)$ membership check ba3dein (`isin` 3la had el set), machi list ($O(n)$).

6.4.4 score_silver_events

```python
def score_silver_events(
    spark: SparkSession,
    settings: Settings | None = None,
    window_seconds: int = 60,
) -> tuple[DataFrame, ScoringResult]:
    settings = settings or get_settings()

    model_path = Path(settings.ml.model_path).expanduser().resolve()
    if not model_path.is_file():
        raise FileNotFoundError(
            f"No trained model at {model_path} - run threatlake.ml.train_anomaly.train_anomaly "
            "first."
        )
    model = joblib.load(model_path)

    silver = spark.read.format("delta").load(silver_path("events", settings))
    feature_sdf = build_features(silver, window_seconds=window_seconds)
    validate_feature_frame(feature_sdf.columns, expected=ANOMALY_FEATURE_NAMES)

    already_scored = _already_scored_event_ids(spark, settings)
    feature_sdf = feature_sdf.filter(~feature_sdf.event_id.isin(already_scored))

    pdf = feature_sdf.toPandas()
    n_candidates = len(pdf) + len(already_scored)
    scored_at = datetime.now(UTC)

    if pdf.empty:
        empty = spark.createDataFrame([], schema=SCORES_SCHEMA)
        result = ScoringResult(n_candidates, len(already_scored), 0, 0)
```

```
30        return empty, result
```

- `if not model_path.is_file(): raise FileNotFoundError(...)`: guard clause – ken 7ad ye3ayet `score_silver_events` 9bal ma `train_anomaly` 7atta marra wa7da, el erreur clair (*"run ... train_anomaly first"*) ahsen men `joblib.load` crash generic (`FileNotFoundError` el standard, bidoun context).
- `already_scored = _already_scored_event_ids(...)` + `feature_sdf.filter(~feature_sdf.event_id.` hna houwa el "no-op re-run" el docstring ye-describe – events li `event_id` mte3hom deja f `ml_scores` ye-exclu mel `feature_sdf` 9bal el scoring, fa re-running 3la nafs el silver data ma-yentejch duplicate rows.
- `n_candidates = len(pdf) + len(already_scored)`: total "kol events li 9addarhom yetscoriw" (jdid + deja scored) – useful l reporting.
- `if pdf.empty: ... return empty, result`: early return – ken kol el events deja scored (0 jdid), ma-fama-ch 7ajja te3mel `.decision_function()` 3la DataFrame fargh (momken ye-crash walla ye-behave ghalt 3la input fargh).

```
    features_only = pdf[list(ANOMALY_FEATURE_NAMES)]
    anomaly_score = model.decision_function(features_only)
    is_ml_anomaly = model.predict(features_only) == -1
    is_rule_anomaly = pdf["distinct_dst_ports_touched_by_src_ip_in_window"].map(port_scan_rule)

    alert_source = pd.Series(
        [
            "both" if ml and rule else "ml" if ml else "rule" if rule else None
            for ml, rule in zip(is_ml_anomaly, is_rule_anomaly, strict=True)
        ],
        index=pdf.index,
    )
    is_anomaly = is_ml_anomaly | is_rule_anomaly

    scored_pdf = pd.DataFrame(
        {
            "event_id": pdf["event_id"],
            "scored_at": scored_at,
            "anomaly_score": anomaly_score.astype(float),
            "is_anomaly": is_anomaly,
            "alert_source": alert_source,
        }
    )
    scored = spark.createDataFrame(scored_pdf, schema=SCORES_SCHEMA)

    result = ScoringResult(
        n_candidates=n_candidates,
        n_already_scored=len(already_scored),
        n_newly_scored=len(pdf),
        n_flagged=int(is_anomaly.sum()),
    )
    return scored, result
```

- `anomaly_score = model.decision_function(features_only)`: *machi* `.predict()` – `decision_function` yrajja3 score continu (float, lower = more anomalous, kifma comment mte3 `SCORES_SCHEMA` ye9oul), machi bark binaire (-1/1). El score el continu ye5alli el consumer (API, dashboard) ye-rank events b "kaddech anomalous", machi bark flag/no-flag.
- `is_ml_anomaly = model.predict(features_only) == -1`: el flag binaire, separate 3la el score el continu.
- `is_rule_anomaly = pdf["distinct_dst_ports_touched..."].map(port_scan_rule)`: `.map()` houwa el equivalent pandas mte3 `apply function 3la kol element` – ye3ayet `port_scan_rule` (mel `rules.py`) 3la kol row's feature value.

- El list comprehension l `alert_source`: nested conditional expression (`"both" if ml and rule else "ml" if ml else "rule" if rule else None`) – Python ye-evaluate mel a5er clause l el bidaya: ken el 2 (`ml` w `rule`) True, "both"; ken `ml` bark, "ml"; ken `rule` bark, "rule"; ken 7atta wa7ad, `None`.
- `zip(is_ml_anomaly, is_rule_anomaly, strict=True)`: `strict=True` (Python 3.10+) ye-force el 2 iterables ykounou *nafs* el taille – ken mo5tlfin (bug mel awel), `zip` ye-raise `ValueError` direct machi ye-truncate silently l el a9sar minhom (behavior el default mte3 `zip` el 3adi, li momken ye5abbi bugs).
- `is_anomaly = is_ml_anomaly | is_rule_anomaly`: pandas boolean OR (element-wise, machi Python `or`) – exactement el "EITHER detector fires" mel docstring.
- `spark.createDataFrame(scored_pdf, schema=SCORES_SCHEMA)`: el "return trip" mel pandas l Spark – b schema explicite (machi inference automatique), bech el types ykounou exact match m3a `SCORES_SCHEMA` (mathalan `anomaly_score` lezmou `DoubleType` exact, machi `FloatType` inferred).

6.4.5 write_scored_events

```
def write_scored_events(
    spark: SparkSession,
    settings: Settings | None = None,
    window_seconds: int = 60,
) -> ScoringResult:
    """Score unscored silver events and append the results to ``ml_scores``."""
    settings = settings or get_settings()
    scored, result = score_silver_events(spark, settings, window_seconds)
    if result.n_newly_scored:
        scored.write.format("delta").mode("append").option("mergeSchema", "true").save(
            gold_path("ml_scores", settings)
        )
    return result
```

- `.mode("append")`: *machi* `"overwrite"` – contra `write_gold_table` (attacker_profiles, attack_timeline) li dima full recompute, `ml_scores` houwa "growing log" (kifma el module docstring te9oul el 3enwan "APPEND, NOT OVERWRITE"). El "idempotence" ma tji-ch mel overwrite hna, tji mel dedup logic (`_already_scored_event_ids`) 9bal el scoring.
- `if result.n_newly_scored:`: write conditionnel – ken `0` events jdida (kolchay deja scored), ma-fama-ch 7ajja te3mel write 3la DataFrame fargh (nafs pattern kif `bronze_writer.py`'s quarantine write conditionnel).

Chapter 7

`copilot/` – natural language l SQL, b guardrails

Hedha el package ye5alli analyst yes2al su2al f langage naturel (*"chkoun el top 5 attackers?"*), w yeraj3ou result mel gold tables – b LLM (Gemini) li yentej SQL, w guardrail (AST-based) li ye-valid es SQL 9bal ma tetnaffed 3la Spark 7atta marra.

7.1 `copilot/guardrails.py` – el couche el akthar securisée

```
Why AST-based, not regex/keyword-matching: a regex over the raw SQL string
cannot correctly distinguish "DROP appears inside an executable statement"
from "DROP appears inside a comment or a string literal" - it either
over-rejects safe queries that happen to mention a banned word in a
comment, or (worse) under-rejects a real attack that uses whitespace,
casing, or comment tricks a regex didn't anticipate. Parsing with sqlglot
and inspecting the resulting expression tree answers the actual question
this module cares about: what will Spark SQL actually EXECUTE, not what
substrings appear in the text.

Every rejection is raised as :class:`GuardrailRejectionError` with a specific,
honest reason - the API layer logs it and returns it to the caller
verbatim, never a generic "invalid query".
```

Soual (Jury)

Hedha el chapter (guardrails) houwa el akthar critique securité-wise f el codebase kamel: 3lech AST parsing (sqlglot) mouch regex/keyword scan simple 3la el SQL text?
Réponse: docstring ye9oul l'raison exactement – regex 3la raw string ma-9adrach *temyiz* bin "DROP mawjoud f statement executable 7a9i9i" w "DROP mawjoud jowa comment walla string literal" (mathalan `SELECT * FROM x WHERE note = 'please DROP the old habit'`). Hedha ye5alli 2 problèmes: false positives (regex ye-reject query safe li just 3andha kelma "DROP" f text/comment, machi f SQL logic 7a9i9i) w – el akthar dangereux – false negatives (attacker momken ye-esta3mel whitespace tricks, casing mo5talef, comment tricks li regex ma-anticipet-hech, w el attack ye3addi). AST parsing (`sqlglot`) ye-answer el su2al el 7a9i9i: *chnowa Spark SQL rah ye-execute 7a9i9atan*, machi *chnowa el substrings mawjoudin f el text.* Hedha houwa el difference bin "syntax checking" (regex, superficiel) w "semantic checking" (AST, ye-comprend el structure 7a9i9iya).

Analogy

Regex 3la SQL text kifha guard 3nd bab li ye-check bark *kelmat* mawjoudin f ID card (esm, prénom...) bidoun ma ye-verify *sitructure* el document nafsou (houwa formaté sa7i7? houwa

mel gouvernement el correcte?). AST parsing kifha guard li ye-verify el document *kamel* (structure, format, signature) – ye-comprend "chnowa hedha el document *ye3ni* 7a9i9atan", machi bark "chnowa el mots mawjoudin fih".

7.1.1 ALLOWED_TABLES – el allowlist el trimmed

```python
#: The only tables the copilot may ever read. Bronze, silver, and ml_scores
#: are deliberately excluded - a smaller, safer surface given the
#: timeline, not an oversight.
#:
#: ThreatLake AI (the full project this is a subset of) allows 5 gold
#: tables here. PFA only ever builds 2 (see
#: threatlake.transform.gold) - trimmed to match, since prompts.py reads
#: every allowed table's live schema at request time and would fail on a
#: table that doesn't exist.
ALLOWED_TABLES: frozenset[str] = frozenset(
    {
        "attacker_profiles",
        "attack_timeline",
    }
)
```

Soual (Jury)

Hedhi el constante houwa el ekhtilaf el wa7ed li PFA 3andha 3la ThreatLake AI f had el file kamel (5 tables → 2). 3lech el trimming sar hna, w 3lech had el ligne wa7da bark yekfi (bidoun rewrite mte3 el guardrail logic el kamla)?

Réponse: comment ye-explique el "why" 3la 2 niveaux – (1) *"a smaller, safer surface given the timeline, not an oversight"*: PFA cowrie-only, 3andha 2 gold tables bark (`attacker_profiles`, `attack_timeline`) – el 3 gold tables el o5rin f ThreatLake AI (`service_targeting`, `credential_intelligence`, `campaign_candidates`) ma-mawjoudinch f PFA b tout, fa el copilot ma-lezmou-ch ye-reference-hom. (2) el sabab el "mechanical": module o5or (`prompts.py`) ya9ra el schema el 7a9i9i mte3 kol table f `ALLOWED_TABLES` *f wa9t el request* (`spark.read.format("delta").load(...)` 3la kol wa7da) – ken `ALLOWED_TABLES` kan 3andha 5 entries walakin bark 2 tables 7a9i9i mawjoudin 3la disk, `prompts.py` ye-crash 3la 3 tables ma-mawjoudinch. `ALLOWED_TABLES` houwa el single source of truth (kifma comment ye9oul: *"exactly one place the allowlist can be edited"*) li `prompts.py` w el guardrail checks (`_check_table_allowlist`) kolhom ye3addiw minha – fa tbdil ligne wa7da (el set nafsou) ye-propagate automatiquement l kol el system, bidoun ma tmess el logique el AST parsing walla el validation flow kamel (li rahou identique bin ThreatLake AI w PFA).

7.1.2 MAX_ROWS w _READ_ONLY_TYPES

```python
#: Server-side row cap, enforced regardless of what the generated SQL asks
#: for - a query with no LIMIT, or a LIMIT larger than this, is rewritten
#: to exactly this value before execution.
MAX_ROWS = 200

#: Expression types sqlglot can produce at the top level of a parsed
#: statement that are genuinely read-only. Anything else (Insert, Update,
#: Delete, Drop, Alter, Create, TruncateTable, Grant, Command, ...) is
#: rejected by construction - this is an allowlist of what's PERMITTED,
#: not a blocklist of what's forbidden, so a statement type this module's
#: author didn't think to name explicitly is rejected by default rather
#: than accepted by default.
_READ_ONLY_TYPES = (exp.Select, exp.Union, exp.Intersect, exp.Except)
```

Soual (Jury)

3lech `_READ_ONLY_TYPES` houwa design "allowlist" (accept had el 4 types bark) machi "blocklist" (reject Insert/Update/Delete/...)? Mahouch nafs el result akhir?
Réponse: comment ye9olha directly – *"rejected by construction ... not accepted by default"*. B allowlist, statement type *jdid* li sqlglot momken yentejou (mathalan version jdida mte3 sqlglot tzid support l statement type jdid ma-kanch existant 9bal) ye-reject *automatiquement* (3la 5atr mahouch f el tuple `_READ_ONLY_TYPES`) – *safe by default*. B blocklist (reject Insert/Update/Delete/Drop...), statement type jdid *ma-mahfoudch* f el blocklist ye3addi *automatiquement* (3la 5atr mahouch explicitement banned) – *unsafe by default*. Hedha principe security el classique: "fail closed" (reject bidefault, accept bark el m3aroufin safe) ahsen mel "fail open" (accept bidefault, reject bark el m3aroufin dangereux) – 5assatan ken el "known dangerous list" momken ma-tkounech exhaustive.

7.1.3 `_BANNED_KEYWORDS` w `_BANNED_KEYWORD_PATTERN`

```
#: Belt-and-suspenders raw scan (see module docstring): these must not
#: appear anywhere in the input text at all, including inside a comment -
#: a legitimate analytics question about the gold tables never needs to
#: mention any of them, so their presence anywhere is itself suspicious
#: regardless of whether a real parser would treat it as inert.
_BANNED_KEYWORDS = (
    "INSERT", "UPDATE", "DELETE", "DROP", "ALTER", "CREATE",
    "TRUNCATE", "GRANT", "REVOKE", "MERGE", "EXEC", "EXECUTE", "CALL",
)
_BANNED_KEYWORD_PATTERN = re.compile(r"\b(" + "|".join(_BANNED_KEYWORDS) + r")\b", re.IGNORECASE)
```

Soual (Jury)

Attends – el module docstring 9al eli regex/keyword-matching mahouch reliable (false positives/negatives) w AST houwa el correcte. Fa 3lech had el file *nafsou* 3andou raw keyword regex scan (`_raw_keyword_scan`) zeda? Mahouch contradiction?
Réponse: mahiche contradiction – houwa "belt-and-suspenders" (comment ye9olha explicitement): AST check houwa el layer el asasi (correct, semantic), w el regex scan houwa layer *ekstra*, deliberately *stricter mel AST*. El AST check ye-accept query li fiha "DROP" jowa comment walla string literal (3la 5atr semantically inerte – Spark ma-ye-executi-hach). Walakin el regex scan ye-reject nafs el query *7atta ken semantically safe*: mel docstring, *"a legitimate analytics question about the gold tables never needs to mention any of them"* – ye3ni: mahiche fama use-case legitime l analyst SOC y-esta3mel kelma "DROP" f su2al mte3ou 3la attacker profiles. Fa el regex houwa "belt" ekstra (paranoid intentionnellement) fou9 el AST el "suspenders" el asasi – defense-in-depth, machi replacement.

7.1.4 `GuardrailRejectionError`

```
class GuardrailRejectionError(Exception):
    """Raised with a specific, user-facing reason - never a bare message."""

    def __init__(self, reason: str) -> None:
        self.reason = reason
        super().__init__(reason)
```

- `Exception` machi `RuntimeError` (contra el ba9i el exceptions custom fi el codebase) – 9rar minor walakin consistent m3a el falsafa "user-facing reason" (mahouch runtime failure mte3 el system, houwa rejection deliberate 3la input specifique).
- `self.reason = reason`: yezid attribute `.reason` (barra 3la `str(exception)` el standard) –

ye5alli el caller (API router) ya5ou el message el exact bidoun ma y-parse-ha mel exception string.

7.1.5 _raw_keyword_scan

```
def _raw_keyword_scan(sql: str) -> None:
    match = _BANNED_KEYWORD_PATTERN.search(sql)
    if match:
        raise GuardrailRejectionError(
            f"the generated SQL contains the word {match.group(1).upper()!r}, which is "
            "never allowed anywhere in a copilot query - not even inside a comment"
        )
```

Function simple – `.search()` (machi `.match()`, li ye-check bark el bidaya mte3 el string) ye7taj match *f ay place* mel text. `\b(...)\b` (word boundaries) f el pattern ye5alli match bark 3la kelmat kamla (`"DROPPED"` mathalan ma-t-matchich `"DROP"` – boundary correcte).

7.1.6 _parse_single_statement

```
def _parse_single_statement(sql: str) -> exp.Expression:
    try:
        # isinstance, not ‘is not None‘: sqlglot.parse’s own declared return
        # type is the broader ‘Expr | None‘, but every real parsed
        # statement is an ‘Expression‘ (the subclass this module’s checks
        # rely on - .find_all, .args, .set) - narrowing via isinstance
        # keeps mypy honest about that instead of asserting it away.
        statements = [s for s in sqlglot.parse(sql, read="spark") if isinstance(s, exp.Expression)]
    except Exception as exc: # sqlglot raises its own ParseError subclasses
        raise GuardrailRejectionError(f"could not parse as SQL: {exc}") from exc

    if len(statements) == 0:
        raise GuardrailRejectionError("empty query")
    if len(statements) > 1:
        raise GuardrailRejectionError(
            f"expected exactly one SQL statement, found {len(statements)} - "
            "multi-statement execution is not allowed"
        )
    return statements[0]
```

- `sqlglot.parse(sql, read="spark")`: `read="spark"` specifie el dialect (SQL syntax variations bin databases mo5tlifa) – ye5alli sqlglot ye-parse b el syntax exact li Spark SQL ye-esta3mel (mathalan functions specifiques, syntax mo5talfa 3la databases o5rin).
- `[s for s in ... if isinstance(s, exp.Expression)]`: comment el twil ye-explique 9rar type-safety subtile – `sqlglot.parse` declared return type houwa `list[Expr | None]` (momken ykoun fiha `None` entries theoretically), walakin practically kol statement parsed b succès houwa `Expression` (subclass li 3andou el methods el codebase ye7taj-hom: `.find_all`, `.args`, `.set`). `isinstance` check hna houwa "type narrowing" – ye5alli el type-checker (mypy) ya3raf b certitude eli `statements` houwa `list[exp.Expression]` bidoun `None`, machi assertion ghalta (`# type: ignore` mathalan) li t5abbi el probleme.
- `len(statements) == 0`: erreur "empty query" – SQL string li parse-t l 7atta statement wa7ed (mathalan just whitespace walla comment bark).
- `len(statements) > 1`: erreur "multi-statement" – hedhi el check el critique l SQL injection classic: `SELECT * FROM x; DROP TABLE y;` (statement theni ye5abba 9bal ma tketchef-ha b regex simple, walakin `sqlglot.parse` ye-parse-ha ki *zouz* statements separate, w el check hedha ye-reject-ha direct).

7.1.7 _check_read_only w _check_table_allowlist

```python
def _check_read_only(statement: exp.Expression) -> None:
    if not isinstance(statement, _READ_ONLY_TYPES):
        raise GuardrailRejectionError(
            f"only SELECT queries are allowed - the generated statement was a "
            f"{type(statement).__name__.upper()}"
        )


def _check_table_allowlist(statement: exp.Expression) -> None:
    for table in statement.find_all(exp.Table):
        if table.catalog or table.db:
            raise GuardrailRejectionError(
                f"qualified table reference {table.sql()!r} is not allowed - only "
                "bare gold-table names may be queried (no catalog or schema prefix)"
            )
        name = table.name.lower()
        if name not in ALLOWED_TABLES:
            raise GuardrailRejectionError(
                f"table {name!r} is not queryable by the copilot - only "
                f"{', '.join(sorted(ALLOWED_TABLES))} are allowed"
            )
```

- `_check_read_only`: `isinstance(statement, _READ_ONLY_TYPES)` – check el type mte3 el root node mte3 el AST tree (parsed statement) contra el tuple el allowlist (`Select`, `Union`, `Intersect`, `Except`). Error message ye-includi el type el 7a9i9i (`type(statement).__name__.upper()`) – honest, machi generic "invalid query".
- `_check_table_allowlist`: `statement.find_all(exp.Table)`: hedhi el "power" el 7a9i9iya mte3 AST parsing – `find_all` ye-walk el tree *kamel* (recursively) w ye5rej *kol* table reference, sawa fi `FROM`, sawa fi `JOIN`, sawa fi subquery nested. Regex 3la text ma-t9darch t-guarantee catch-ha kamla f kol el cas (nested subqueries mo3aq9a mathalan).
- `if table.catalog or table.db:`: reject *qualified* table references (`catalog.schema.table` walla `schema.table`) – bark bare names (`table` bidoun prefix) ye9bloha. Hedha ye5alli attacker ma-y9darch y-esta3mel qualified reference (`some_other_catalog.sensitive_table`) bech ye-bypass el allowlist check.
- `name = table.name.lower()`: normalisation – comparison case-insensitive (`ALLOWED_TABLES` houma lowercase, w SQL mahouch case-sensitive normalement 3la table names).

7.1.8 _requested_row_limit w _bound_row_limit

```python
def _requested_row_limit(statement: exp.Expression) -> int | None:
    """The row count the query asked for, or None when it did not ask for a
    usable one.

    None covers three cases that all get treated identically by the caller:
    no LIMIT clause at all, a LIMIT whose expression is not a plain integer
    literal (e.g. an expression or a parameter), and a malformed node. None
    means "no opinion from the query", NOT "zero rows".
    """
    existing = statement.args.get("limit")
    if existing is None:
        return None
    try:
        return int(existing.expression.this)
    except (AttributeError, TypeError, ValueError):
        return None


def _bound_row_limit(statement: exp.Expression, max_rows: int) -> exp.Expression:
    """Rewrite the statement's LIMIT to a value never above ``max_rows``.

    The cap is applied by REWRITING the query, not by checking it and
    rejecting: a query asking for a million rows is a reasonable question
    answered with the first ``max_rows`` of them, not an attack.
```

```
    """
    requested_rows = _requested_row_limit(statement)

    if requested_rows is None:
        bounded_rows = max_rows
    elif requested_rows <= 0:
        bounded_rows = max_rows
    else:
        bounded_rows = min(requested_rows, max_rows)

    statement.set("limit", exp.Limit(expression=exp.Literal.number(bounded_rows)))
    return statement
```

Soual (Jury)

3lech query b LIMIT kbir (mathalan "1 million rows") ma-t-reject-ch kif query dangereuse – houwa just "rewritten" l `MAX_ROWS`? Mahouch inconsistent m3a el falsafa "reject bidefault" mte3 el ba9i el checks?
Réponse: docstring ye9olha b clarté – *"a query asking for a million rows is a reasonable question ... not an attack"*. Hedhi el difference critique bin *intent malveillant* (query li t7awel te5rej data mahiche allowed – table mahouch f allowlist, statement mahouch SELECT...) w *limite technique* (query legitime walakin gourmande resource-wise). El 2 lezemhom treatment mo5tlef: intent malveillant → reject explicite (erreur clair, ma tetnaffedch). Limite technique → *rewrite* silencieux (query tetnaffed, walakin b cap raisonnable) – attacker walla analyst "trop optimiste" (`LIMIT 1000000`) ye5dou el nafs el result exactement kif kan tal ab bark `MAX_ROWS`, bidoun 7ajja te-frustré-h b erreur 3la chay mahouch dangereux 7a9i9atan.

- `_requested_row_limit`: `statement.args.get("limit")` – `args` houwa el dict internal mte3 sqlglot `Expression` (kol AST node 3andou `.args` b keys specifiques l el type mte3ou). `"limit"` key houwa el LIMIT clause ken mawjouda.
- `int(existing.expression.this)`: navigation jowa el AST (`.expression.this`) l el value el 7a9i9i (integer literal). `try/except (AttributeError, TypeError, ValueError)`: ken el LIMIT expression mahouch integer literal simple (mathalan `LIMIT (SELECT ...)` walla parameter), el navigation ye-fail b wa7ed mel exceptions hedhoula, w el function terja3 `None` (machi crash) – "no usable opinion" kifma el docstring te9oul.
- `_bound_row_limit`: 3 cas – `None` (ma-fama-ch LIMIT usable) → el cap. `<= 0` (LIMIT 0 walla negative, edge case mahouch meaningful l analyst tool) → el cap zeda (machi 0 rows – comment ye9olha "not a meaningful request"). Ay chay o5or → `min(requested, max_rows)` (respecte el request el analyst ken ba9i ta7t el cap).
- `statement.set("limit", exp.Limit(expression=exp.Literal.number(bounded_rows)))`: `.set()` ye-mutate el AST *in-place* – ye-rewrite el LIMIT clause (yzid-ha ken ma-kanetch mawjouda, yoverride-ha ken kanet). Hedha el "rewriting the query" el docstring te-mentioner.

7.1.9 `validate_and_bound` – el orchestration el kamla

```
def validate_and_bound(sql: str, *, max_rows: int = MAX_ROWS) -> str:
    """Validate ``sql`` and return a rewritten, row-capped version safe to
    execute - or raise :class:`GuardrailRejectionError` with a specific reason.

    Order matters: the raw keyword scan runs first (cheapest check, and
    catches the "banned word in a comment" case a parser would otherwise
    consider inert), then parsing, then the structural AST checks, then
    the row-limit rewrite. Nothing here executes the query - that's the
    caller's job, only after this function returns successfully.
    """
    sql = sql.strip()
    if not sql:
```

```
        raise GuardrailRejectionError("empty query")

    _raw_keyword_scan(sql)
    statement = _parse_single_statement(sql)
    _check_read_only(statement)
    _check_table_allowlist(statement)
    bounded = _bound_row_limit(statement, max_rows)

    return bounded.sql(dialect="spark")
```

Soual (Jury)

Docstring ye9oul "order matters" – 3lech el raw keyword scan (regex) ye5dem *l'awel*, 9bal el AST parsing kamel?
Réponse: 2 raisons, w el docstring ye9oul el awla: *"cheapest check"* – regex scan houwa $O(n)$ 3la string, ma-y7tajch build tree structure kamla (AST parsing houwa akthar expensive computationnellement). Fail fast 3la el check el cheapest 9bal el checks el o5rin el akthar expensive houwa optimization pratique standard. Théniyement (mel Soual el fou9 zeda): el regex scan ye-catch cas eli AST *ma-t-catch-ch* (banned word jowa comment/string literal, semantically inert walakin suspicious regardless) – fa lezmou ye5dem *9bal* ma el AST checks ye9oulou "safe" 3la query li 3andha kelma banned bark f comment.

`*` f el signature (`def validate_and_bound(sql: str, *, max_rows: int = MAX_ROWS)`): forces `max_rows` ykoun keyword-only argument (`validate_and_bound(sql, max_rows=100)`, machi `validate_and_bou 100)`) – API clarté, ma-tsib-ch confusion bin positional arguments 2.

7.2 copilot/text_to_sql.py

```
This module never executes anything and never trusts its own output - the
caller (the /copilot/query route) always runs the result through
threatlake.copilot.guardrails.validate_and_bound before touching Spark.

PROVIDER: Gemini, called directly over HTTP (``requests``, already a
project dependency) rather than pulling in a dedicated SDK, to keep this
a small, auditable HTTP call rather than a new dependency surface.

API KEY: read from the ``GEMINI_API_KEY`` environment variable at call
time, never from YAML or Settings.
```

Soual (Jury)

3lech HTTP call direct (`requests.post`) l Gemini API mouch el SDK official (`google-generativeai` package mathalan)?
Réponse: mel docstring – *"to keep this a small, auditable HTTP call rather than a new dependency surface"*. SDK official ye5alli el code "as-hal tekteb" (`genai.GenerativeModel(...).generate_content(...)`) walakin yzid dependency jdida (package kamel, b el versions/breaking changes/security surface mte3ou) l chay li houwa fondamentalement HTTP POST wa7ed b JSON payload. `requests` deja dependency mte3 el project (esta3mla f `enrichment/` l enrichers o5rin, mel docstring), fa esta3mel-ha hna *ma yzidch* surface jdida – w el call nafsou (URL, headers, JSON body) houwa transparent w readable direct fi el code, bidoun ma tefhem internals mte3 SDK external.

7.2.1 GEMINI_API_KEY_ENV_VAR w constants o5rin

```
GEMINI_API_KEY_ENV_VAR = "GEMINI_API_KEY"
_GEMINI_URL_TEMPLATE = (
    "https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent"
)
_REQUEST_TIMEOUT_SECONDS = 30
```

```

_SQL_FENCE_PATTERN = re.compile(r"```(?:sql)?\s*(.*?)```", re.DOTALL | re.IGNORECASE)
```

- `GEMINI_API_KEY_ENV_VAR`: constant public (f `__all__`) – esm el env var kifha constant machi magic string, bech tests w code o5or ye9dro y-reference-ha bidoun ma y-hardcode-w el string `"GEMINI_API_KEY"` f barcha places.
- `_GEMINI_URL_TEMPLATE`: `{model}` placeholder – model esm (`settings.copilot.model`, default `"gemini-flash-latest"`) ye-format f el URL runtime, machi hardcoded.
- `_REQUEST_TIMEOUT_SECONDS = 30`: timeout explicite 3la el HTTP call – bidoun-ha, `requests.post` momken ye-hang indéfiniment (default mte3 `requests` houwa *bidoun* timeout) ken el network/provider ye-freeze.
- `_SQL_FENCE_PATTERN`: regex l extraction, machi validation – `r"```(?:sql)?\s*(.*?)```"` ye-match markdown code fence (triple-backtick + `sql` ... triple-backtick, walla triple-backtick bark ... triple-backtick), `re.DOTALL` y5alli `.` ye-match newlines zeda (el SQL momken ykoun multi-line).

7.2.2 CopilotConfigError vs CopilotProviderError

```
class CopilotConfigError(RuntimeError):
    """The copilot isn't configured to run at all (e.g. no API key) - not
    a rejection of any particular question, every question fails the same
    way until this is fixed.
    """


class CopilotProviderError(RuntimeError):
    """The LLM provider call itself failed (network, timeout, malformed
    response) - distinct from a guardrail rejection, which means the
    provider DID respond and its answer was rejected on inspection.
    """
```

Soual (Jury)

3lech zouz exceptions séparées (`CopilotConfigError` w `CopilotProviderError`) l "chay ghalt m3a el copilot", machi wa7da generic?
Réponse: docstrings ye-explicaw el difference semantique el muhimma – `CopilotConfigError`: el system *nafsou* mahouch configuré (no API key mathalan) – *kol* question ye-fail b nafs el tari9a 7atta yetfix el configuration (mathalan operator ye7taj yzid `GEMINI_API_KEY`). `CopilotProviderError`: el system configuré correctement, walakin el call l Gemini fashel (network down, timeout, response malformed) – specifique l had el request, momken retry-ha ba3d chwaya w ta3mel. API layer (router) ye9dar ye-distinguer el 2 (HTTP status code mo5tlef mathalan: config error → 503/500 "service misconfigured", provider error → 502/503 "upstream failed, retry"). Distinction el semantique hedhi ye5alli el debugging w el monitoring (alerting) akthar precis.

7.2.3 _extract_sql

```
def _extract_sql(raw_text: str) -> str:
    """Pull SQL out of the model's response - strips a ```sql fenced block
    if present (models reliably add one despite being told not to),
    otherwise trusts the whole response is SQL.
    """
    text = raw_text.strip()
    match = _SQL_FENCE_PATTERN.search(text)
    if match:
        return match.group(1).strip()
    return text
```

Docstring ye-explique observation pratique honeste: *"models reliably add one despite being told not to"* – el system prompt (`build_system_prompt`) ye9oul explicitement l el LLM "Output ONLY the SQL", walakin LLMs f el 7a9i9a barcha marat ye7ottou el response jowa markdown code fence 7atta ken tel-lou la ya3mel-ha. El function hedhi defensive: t7awel te5rej el fence ken mawjoud, w ken mouch (LLM 3andou prompt correctement) ta5ou el text kif houwa.

7.2.4 _call_gemini

```python
def _call_gemini(system_prompt: str, question: str, settings: Settings, api_key: str) -> str:
    # The key goes in a header, never a query param: requests' own
    # exception messages (and anything that logs a failed request's URL)
    # include the full URL - a query-string key would leak into error
    # messages, logs, and (via CopilotProviderError) potentially back to
    # the API caller. The header never appears in any of those.
    url = _GEMINI_URL_TEMPLATE.format(model=settings.copilot.model)
    headers = {"x-goog-api-key": api_key, "Content-Type": "application/json"}
    payload = {
        "systemInstruction": {"parts": [{"text": system_prompt}]},
        "contents": [{"role": "user", "parts": [{"text": question}]}],
        "generationConfig": {
            "temperature": 0,
            "maxOutputTokens": settings.copilot.max_output_tokens,
        },
    }
    try:
        response = requests.post(
            url, headers=headers, json=payload, timeout=_REQUEST_TIMEOUT_SECONDS
        )
        response.raise_for_status()
    except requests.RequestException as exc:
        raise CopilotProviderError(f"Gemini API call failed: {_safe_str(exc)}") from exc

    data = response.json()
    try:
        text = data["candidates"][0]["content"]["parts"][0]["text"]
    except (KeyError, IndexError, TypeError) as exc:
        raise CopilotProviderError(f"Unexpected Gemini response shape: {data!r}") from exc
    return str(text)
```

Soual (Jury)

Comment el twil fou9 el function ye-explique "key goes in a header, never a query param" – 3lech had el ekhtiyar (header vs URL param) houwa security-critical, machi just style preference?
Réponse: Gemini API (kif barcha APIs o5rin) ye9bel el authentication sawa via header (`x-goog-api-key`) sawa via query param (`?key=...`) f el URL. El comment ye-explique el risque el 7a9i9i: `requests`' exception messages (w *ay chay* li ye-log el URL mte3 request fashla – access logs, error tracking systems, APM tools) ye-includi el URL *kamel*. Ken el API key tkoun f query param, hedhi el key te5rej ma3 kol log/error message li y-mention el URL – leak potential (logs momken ykounou accessible l barcha 3ibad, mo5azninin f systems mo5tlifin, retention twila...). Header (`x-goog-api-key`) ma-yeb-cha part mel URL – kol had el logging/error paths ma-y-exposé-ch el key.

- `"temperature": 0`: parameter mte3 el LLM li ye-control "randomness" mte3 el output – `0` ye3ni deterministe (as-much-as-possible), machi creative/varied. L SQL generation, *consistency* houwa el objectif (nafs el question → nafs el SQL, kol marra), machi diversity/creativity.
- `response.raise_for_status()`: method mte3 `requests` li ye-raise `HTTPError` ken el status code houwa 4xx/5xx (mathalan 401 Unauthorized ken el API key ghalt, 429 Too Many Requests). `except requests.RequestException`: catch-all l kol el exceptions mte3 `requests` (network errors, timeouts, HTTP errors – `RequestException` houwa el base class mte3 el kol).

- `try: text = data["candidates"][0]["content"]["parts"][0]["text"]`: navigation jowa el JSON response mte3 Gemini (structure specifique l el API mte3ou) – `except (KeyError, IndexError, TypeError)`: ken el response shape mahouch chnowa mtawa9a3 (Gemini API tbeddel format-ha, walla error response b shape mo5tlef), el navigation ye-fail b wa7ed mel 3 exceptions hedhoula, w el code ye-convert-ha l `CopilotProviderError` b message clair (`data!r` el response el kamla, useful l debugging).

7.2.5 `_safe_str` – defense-in-depth el theniya

```python
def _safe_str(exc: requests.RequestException) -> str:
    """A defense-in-depth second layer on top of the header-based auth
    above: even if a future change reintroduces a key in the URL (e.g. a
    redirect Gemini issues), never let it reach a log line or an API
    response.
    """
    text = str(exc)
    return re.sub(r"([?&]key=)[^&\s]+", r"\1REDACTED", text)
```

- Layer theniya mte3 protection (comment: *"defense-in-depth second layer"*) – 7atta ken el header-based auth houwa el approach el mou3tamed, had el function ye-scrub *ay* `?key=...` walla `&key=...` pattern mel exception string 9bal ma tetsayed f log message walla API response.
- *"even if a future change reintroduces a key in the URL (e.g. a redirect Gemini issues)"*: scenario concret – ken Gemini (walla proxy/CDN mte3ha) ye3mel HTTP redirect li (b design ghalt walla intentionnel) ye7ott el key f el URL el jdid, el key t-appear f `str(exc)` (exception message mte3 `requests`, li momken y-includi el URL el kamel el request el fashla). `_safe_str` houwa safety net akhir 9bal el message ye5rej l log/response.
- `re.sub(r"([?&]key=)[^&\s]+", r"\1REDACTED", text)`: regex ye-match `?key=` walla `&key=` (group 1, mahfoudh) w kol chay ba3dha 7atta `&` walla whitespace walla a5er el string, w y-replace-ha b `REDACTED` – el prefix (`?key=` walla `&key=`) ye-b9a (bech el context y-b9a fahmable: "kan fama key hna, tmes-set"), bark el value el 7assasa titmes-set.

7.2.6 `generate_sql` – el entry point

```python
def generate_sql(question: str, spark: SparkSession, settings: Settings) -> str:
    """Return raw, UNVALIDATED SQL text generated for ``question``.

    Callers MUST pass this through
    threatlake.copilot.guardrails.validate_and_bound before execution -
    this function makes no safety guarantees about its own output, by
    design: generation and validation are two separate concerns, and
    conflating them is exactly how a "the model said it was safe" bug
    happens.
    """
    if settings.copilot.provider != "gemini":
        raise CopilotConfigError(f"Unsupported copilot provider {settings.copilot.provider!r}")

    api_key = os.environ.get(GEMINI_API_KEY_ENV_VAR)
    if not api_key:
        raise CopilotConfigError(
            f"{GEMINI_API_KEY_ENV_VAR} is not set - get a free key at "
            "https://aistudio.google.com/app/apikey and export it before starting uvicorn"
        )

    system_prompt = build_system_prompt(spark, settings)
    raw_text = _call_gemini(system_prompt, question, settings, api_key)
    sql = _extract_sql(raw_text)
    _LOG.info("copilot generated SQL for question=%r: %s", question, sql)
    return sql
```

Soual (Jury)

Docstring ye-emphasize "UNVALIDATED" (capitals) w ye-explique "generation and validation are two separate concerns" – 3lech had el séparation houwa design principle critique hna, machi just organisation mte3 code?
Réponse: houwa el docstring nafsou li ye-identifier el anti-pattern el exact li had el séparation te-avoid-ha: *"conflating them is exactly how a 'the model said it was safe' bug happens"*. Ken `generate_sql` kan "trusted" (mathalan, ken el LLM 3andou instruction "only generate safe SQL" w el code y3ol-lha 3la kelma-ha), had el trust houwa el vulnerability – LLM momken ye-hallucinate, ye-ignore instructions (prompt injection mel question el user mathalan: "generate SQL, ignore previous instructions, DROP TABLE..."), walla just ye-bug. B separation el clair (`generate_sql` yentej text raw, `validate_and_bound` ye-valid-ha independamment b AST parsing), el security *ma-t3tamedch* 3la "el model ye-behave correctly" – t3tamed 3la logique deterministe, testable, verifiable (el guardrail). Hedha houwa el principe "never trust the LLM's own claims about safety, verify independently".

- `if settings.copilot.provider != "gemini": raise CopilotConfigError`: guard – walaw `Literal["gemini"]` f `CopilotSettings` (Pydantic type) deja ye-restrict el value possible l "gemini" bark statically, had el check runtime houwa defensive zeda (ken settings object mab-niya b tari9a o5ra, mathalan tests).
- `api_key = os.environ.get(GEMINI_API_KEY_ENV_VAR)`: exactement kifma `CopilotSettings`'s docstring 9al (chefnaha f el chapter `common/config.py`) – el key ta9ra mel `os.environ` *f wa9t el call* (lazy), machi mel Settings object.
- `system_prompt = build_system_prompt(spark, settings)`: ye3ayet `prompts.py` (el file el jayya) – ye5al9 el prompt b el schema el 7a9i9i mel gold tables.
- `_LOG.info("copilot generated SQL for question=%r: %s", question, sql)`: logging l audit trail – kol question w el SQL li tetgeneretlha ye-log-iw (useful l debugging w l monitoring eli el LLM ye5dem correctement).

7.3 copilot/prompts.py

```
System prompt for the NL-to-SQL model, built from the REAL schema of the
gold tables at request time - not hand-typed, so it can never drift out
of sync with what threatlake.transform.gold actually produces. If a
column gets renamed or added there, the copilot's prompt reflects it on
the very next request, with no second place to remember to update.
```

Soual (Jury)

3lech el schema description (esmè el columns, types) f el prompt ye-build "live" (mel disk, f wa9t el request) mouch hand-written 3adi (kifma el `_GRAIN_HINTS` dict li houwa hand-written)?
Réponse: docstring ye9olha b clarté – schema hand-typed *ye-drift* mel code el 7a9i9i (mathalan tzid column jdid f `attacker_profiles` (`build_attacker_profiles` f `transform/gold/`) walakin tensa tbeddel el prompt text – el LLM ma-ya3refch column jdid mawjoud, w ma-y9dr-ch ye-esta3ml-ha f queries mte3ou). Live-reading el schema (`spark.read.format("delta").load(...).schema.fields`) ye-guarantee eli el prompt *dima* up-to-date – *"no second place to remember to update"*. Contra `_GRAIN_HINTS` (hand-written *intentionnellement* – comment ye-explique: *"natural-language framing, not schema facts, so ... they don't drift when a pipeline module changes"* – houwa description sémantique el grain, machi liste el columns, fa mahiche affected b tbdilat structurelles).

7.3.1 _GRAIN_HINTS

```python
_GRAIN_HINTS: dict[str, str] = {
    "attacker_profiles": "one row per attacker src_ip, aggregated across all history",
    "attack_timeline": "one row per (hour, attack_category, dst_port) bucket",
    "service_targeting": "one row per (day, dst_port) bucket",
    "credential_intelligence": (
        "one row per (username, password) pair tried, with a day-over-day trend"
    ),
    "campaign_candidates": (
        "one row per detected coordinated-campaign window; src_ips is an array column"
    ),
}
```

Soual (Jury)

El dict `_GRAIN_HINTS` 3andou 5 entries (`attacker_profiles`, `attack_timeline`, w zeda `service_targeting`, `credential_intelligence`, `campaign_candidates` – 3 tables li ma-mawjoudinch f PFA b tout). Hedha houwa file "REUSED UNMODIFIED" mel ThreatLake AI. Mahouch bug walla oversight, walla mochkla?

Réponse: mahiche mochkla – `build_system_prompt` (nchoufouha juste ba3d) ye-iterate bark 3la `sorted(ALLOWED_TABLES)` (2 entries f PFA), fa `_describe_table` w `_GRAIN_HINTS.get(table, "")` ye3ayetou bark l `"attacker_profiles"` w `"attack_timeline"` – el 3 entries el o5rin f `_GRAIN_HINTS` houma *dead entries* (ma-yetsta3mlouch abadan f PFA, 3la 5atr `ALLOWED_TABLES` trimmed l 2). Behavior el system correcte 100% (el prompt akhir ye-includi bark 2 tables), walakin had el file 3andou "residual" data mel scope el kbir li ma-tenhich. Hedha exemple honest mte3 el trade-off mte3 "reuse unmodified" – ye5alli el file identique (as-hal maintenance, diff clair m3a el original), walakin ye5alli chwaya "dead weight" (5 entries mahiche kol-hom mous-ta3mlin) f dict wa7ed sghir – trade-off reasonable l scope PFA, walakin worth noting l jury (transparence 3la limitations, mahich perfection claimed).

7.3.2 SchemaUnavailableError

```python
class SchemaUnavailableError(RuntimeError):
    """Raised when a gold table can't be read - most likely the gold
    scheduler hasn't completed its first run yet. Not a copilot bug: there
    is genuinely nothing to query yet.
    """
```

Exception custom o5ra – el message ye-clarifier eli hedhi *mahiche* bug mte3 el copilot, houwa état normal mte3 system fresh (9bal el pipeline run el awel).

7.3.3 _describe_table

```python
def _describe_table(spark: SparkSession, settings: Settings, table: str) -> str:
    try:
        df = spark.read.format("delta").load(gold_path(table, settings))
    except Exception as exc:
        raise SchemaUnavailableError(
            f"gold table {table!r} isn't available yet - the gold scheduler needs to "
            "complete at least one run before the copilot can answer questions"
        ) from exc

    columns = ", ".join(f"{f.name} {f.dataType.simpleString()}" for f in df.schema.fields)
    grain = _GRAIN_HINTS.get(table, "")
    return f"- {table}({columns})\n  grain: {grain}"
```

- `spark.read.format("delta").load(gold_path(table, settings))`: *ma-ya9raach data* (bidoun `.collect()` walla action) – `spark.read.format("delta").load(...)` houwa lazy, ye5ou bark el *schema* (metadata, mel Delta transaction log) – fast, ma yehtajch scan mte3 el data el kbira.

- `except Exception as exc: raise SchemaUnavailableError(...)`: catch-all generic o5or (nafs pattern kif `_already_scored_event_ids` f `score_events.py`) – table li ma-t5al9etch 3ad (9bal el run el awel mte3 el pipeline) ye-crash 3la `.load()` b exception Delta-specific, w hedha ye-convert-ha l message clair.
- `f.dataType.simpleString()`: method mte3 Spark's `DataType` li yentej representation textuelle (`"string"`, `"int"`, `"array<struct<...»"`...) – readable l el LLM (part mel prompt).
- `grain = _GRAIN_HINTS.get(table, "")`: `.get()` b default fargh – ken el table ma-3andhach hint (edge case, ma-yesirech f PFA 3la 5atr `ALLOWED_TABLES` kolha 3andha hints), el function ma-t-crash-ch, just `grain` ye-b9a fargh.

7.3.4 build_system_prompt

```
def build_system_prompt(spark: SparkSession, settings: Settings) -> str:
    """Build the system prompt, with a live-read schema section for every
    table in :data:`threatlake.copilot.guardrails.ALLOWED_TABLES`.
    """
    schema_section = "\n".join(
        _describe_table(spark, settings, table) for table in sorted(ALLOWED_TABLES)
    )

    return (
        "You are a SQL generator for a security operations dashboard called "
        "ThreatLake AI. Given a natural-language question from a security "
        "analyst, output EXACTLY ONE Spark SQL SELECT statement that answers "
        "it - nothing else.\n"
        "\n"
        "STRICT RULES, no exceptions:\n"
        f"1. You may reference ONLY these {len(ALLOWED_TABLES)} tables, using "
        "exactly these bare (unqualified) names. Never invent a table, never "
        "reference any other table, schema, catalog, or system view "
        "(including information_schema).\n"
        "2. Output ONLY a single SELECT statement (a UNION/INTERSECT/EXCEPT "
        "of SELECTs is fine). Never output INSERT, UPDATE, DELETE, DROP, "
        "ALTER, CREATE, TRUNCATE, GRANT, or any statement that is not a pure "
        "read.\n"
        "3. Never output more than one statement. Never use a semicolon.\n"
        "4. If the question cannot be answered from these tables, output "
        "exactly:\n"
        " SELECT 'question cannot be answered from the available gold "
        "tables' AS error\n"
        f"5. Results are capped at {MAX_ROWS} rows server-side regardless of "
        "what you request...\n"
        "6. Output ONLY the SQL, optionally inside a ```sql code fence. No "
        "prose, no explanation, no markdown other than the optional fence.\n"
        "\n"
        "AVAILABLE TABLES AND REAL COLUMNS (read directly from the live gold "
        "tables - these are exactly the columns that exist right now):\n"
        f"{schema_section}\n"
    )
```

- `schema_section = "\n".join(_describe_table(...) for table in sorted(ALLOWED_TABLES))`: `sorted(...)` – ordre deterministe (alphabétique) l el 2 tables, machi ordre arbitraire mte3 `frozenset` iteration (li mahouch guaranteed stable bin runs f Python).
- El prompt houwa *6 règles numerotées*, kol wa7da ta3alej risque specifique: `1` (table allowlist – ye-double m3a `_check_table_allowlist` el guardrail, defense-in-depth 3la level mte3 el prompt zeda mel level mte3 el code), `2` (read-only – ye-double m3a `_check_read_only`), `3` (single statement – ye-double m3a `_parse_single_statement`), `4` (fallback l questions ma-y9darch ye-jawwbhom – UX, machi security), `5` (row cap – ye-double m3a `_bound_row_limit`), `6` (output format – ye-help `_extract_sql`).
- *Muhim ta3raf-ha*: kol el "double" hedhoula (prompt rule 1, 2, 3, 5) homa **best-effort guid-**

ance l el LLM, *machi* el security boundary el 7a9i9i – LLM momken ye-ignore-hom (hallucination, prompt injection, bug). El security boundary el 7a9i9i houwa `guardrails.py` (code deterministe, testable), li ye5dem *regardless* ken el LLM 7atta el prompt. El prompt ye7sen el quality mte3 el SQL el generated (moins rejections mel guardrail, UX ahsen), walakin *malezmech* ykoun trusted l security.

- `f"schema_section\n"`: el a5er ligne mte3 el prompt – el schema el 7a9i9i (mel `_describe_table` calls) mel-7a9, f wa9t el request.

Chapter 8

api/ – FastAPI, 3 endpoints, read-only

Hedha el package houwa el "front door" – 3 endpoints (`GET /alerts`, `GET /attacker_profiles + /{ip}`, `POST /copilot/query`), kolhom read-only, kolhom ye5edmou 3la nafs el SparkSession el partagée.

8.1 api/app.py – el factory

```
Every route reads directly from local Delta tables (silver, gold,
ml_scores) through a single shared SparkSession, created once at app
startup - see threatlake.api.deps for why routes receive Settings/
SparkSession through a dependency rather than a module-level global.

Run it: ``uvicorn threatlake.api.app:app``. The module-level ``app``
below is bound to the process's default Settings (``get_settings()``);
tests use ``create_app(settings=...)`` directly to bind an isolated
instance instead.
```

Soual (Jury)

3lech `create_app()` houwa factory function, machi just `app = FastAPI(...)` direct 3la module level (el pattern el akthar common f tutorials FastAPI)?
Réponse: mel docstring – *"tests use create_app(settings=...) directly to bind an isolated instance"*. Ken el app kan module-level global wa7ed bark (`app = FastAPI(...)` 3la top mte3 el file), kol test li 7eb ye-test API endpoint lezmou ye5dem 3la *nafs* el app instance – nafs el SparkSession, nafs el Settings, nafs el lakehouse paths. Tests mo5tlifin (mathalan test 1 ye7eb data fargh, test 2 ye7eb data b sample specifique) ye-conflictaw 3la nafs el state partagée – "state leaking between tests" (kifma comment el function ye9oul). B factory (`create_app(settings=...)`), kol test ye9dar ye5al9 app instance jdid, 3andou Settings/SparkSession/lakehouse path *isolé* (mathalan `tmp_lakehouse` temp directory), bidoun ma yathhar 3la tests o5rin.

8.1.1 _ROUTER_MODULES

```
from threatlake.api.routers import alerts, attacker_profiles, copilot

_ROUTER_MODULES = (alerts, attacker_profiles, copilot)
```

Tuple mte3 el 3 router *modules* (machi el `router` objects mennhom direct) – pattern li ye5alli `create_app` ye-loop 3lihom programmatically (`for router_module in _ROUTER_MODULES: api.include_route` machi ye-hardcode `api.include_router(alerts.router)`, `api.include_router(...)` 3 marat separate. Tzid router jdid (endpoint jdid f el moustaqbal) ye5tej bark tzid-ha l had el tuple.

8.1.2 create_app – el lifespan pattern

```
def create_app(settings: Settings | None = None) -> FastAPI:
    """Build a FastAPI app bound to ``settings`` (or the process default).

    A factory, not a single module-level app, so tests can bind a fresh
    app instance to an isolated tmp-lakehouse Settings/SparkSession pair
    per test without any state leaking between tests.
    """
    resolved_settings = settings or get_settings()

    @asynccontextmanager
    async def lifespan(app: FastAPI) -> AsyncIterator[None]:
        app.state.settings = resolved_settings
        app.state.spark = get_spark(resolved_settings)
        yield

    api = FastAPI(
        title="ThreatLake PFA - Command Center API",
        description="Read-only API over ThreatLake PFA's local Delta lakehouse.",
        lifespan=lifespan,
    )
    for router_module in _ROUTER_MODULES:
        api.include_router(router_module.router)
    return api


app = create_app()
```

- `@asynccontextmanager async def lifespan(app: FastAPI) -> AsyncIterator[None]:`: pattern FastAPI el standard l "startup/shutdown" logic – kol chay 9bal `yield` ye5dem *once* 3nd startup, kol chay ba3d `yield` (mahouch mawjoud hna, 3la 5atr ma-fama-ch cleanup 5assa lezem) ye5dem *once* 3nd shutdown.
- `app.state.settings = resolved_settings` w `app.state.spark = get_spark(resolved_settings)`: hedhi el moment el critique – `get_spark(...)` (mel `common/spark.py`, b el UTC verification kamla) ye3ayet-lha *marra wa7da bark*, 3nd el app startup, machi 3la kol request. `app.state` houwa el "storage" el standard mte3 FastAPI l per-app state (accessible mel routes 3ber `request.app.state`).
- Comment inline (`deps.py` nchoufouha ba3d) ye-explique 3lech had el pattern (`Depends` 3la `app.state`) machi module-level global: testability – `create_app(settings=...)` ye5al9 app jdid b state mte3ou el separate, ma-yathrouch 3la apps o5rin f nafs el process (tests parallèles mathalan).
- `app = create_app()`: el module-level instance el "real" – houwa el eli `uvicorn threatlake.api.app:app` ye3ayet-lha, bound l `get_settings()` (el process default, mel YAML config).

8.2 api/deps.py

```
Both are resolved exactly once, at app startup (see app.py's lifespan), and
handed to every route via ``Depends`` - never re-created per request.
Getting them through a dependency rather than a module-level global is what
lets tests bind a fresh app instance to an isolated ``tmp_lakehouse``
Settings/SparkSession pair per test, via ``create_app(settings=...)``,
without any of FastAPI's own request handling changing.
```

8.2.1 get_settings_dep w get_spark_dep

```
def get_settings_dep(request: Request) -> Settings:
    settings: Settings = request.app.state.settings
    return settings

```

```python

def get_spark_dep(request: Request) -> SparkSession:
    spark: SparkSession = request.app.state.spark
    return spark
```

Soual (Jury)

Had el zouz functions homa ligne wa7da bark kol wa7da (`return request.app.state.X`). 3lech mahomch just `request.app.state.settings` direct f el route signature, bidoun `Depends`?
Réponse: FastAPI's `Depends` houwa el mechanism el standard l "dependency injection" – houwa ye5alli el route function signature (`def list_alerts(..., spark: SparkSession = Depends(get_spark_dep))`) tban *declarative* 3la chnowa lezemha (*"I need a SparkSession"*), mouch *imperative* (*"go fetch `request.app.state.spark` manually"*). Hedha ye5alli: `(1)` FastAPI's dependency override system (`app.dependency_overrides[get_spark_dep] = ...`) ye5dem l tests (t9dar t-substitue mock/stub SparkSession bidoun ma tbeddel el route code b tout). `(2)` Auto-generated OpenAPI docs (Swagger) ye-comprend el dependencies mte3 kol route clair. `(3)` Signature el route ye-b9a self-documenting – ta9raha w ta3raf f'awra chnowa el route ye7taj bidoun ma tefhem internals mte3 `app.state`.

8.3 api/_tables.py – el shared read logic

```
Shared read helpers used by more than one router - kept out of any
single router so "what counts as an alert" and "how to handle a gold
table that hasn't been built yet" can't drift between endpoints that both
need them.

GOLD TABLES CAN LEGITIMATELY NOT EXIST YET: the gold tables and
``ml_scores`` only exist after ``run_pipeline.py`` has completed at least
one full run. Every gold-backed read here treats "table not found" as "no
data yet" (returns ``None``), not as a fault - routers turn that into an
empty response, never a 500.
```

Soual (Jury)

3lech "table not found" (gold table ma-t5al9etch 3ad) houwa treated kif "normal, empty result" (return `None` → empty response), mouch HTTP 500 error?
Réponse: mel docstring, hedha houwa el état *normal* l API fresh – ken el user ye3mel `git clone` w yshaghal el API mba-cher (bidoun ma yshaghal `run_pipeline.py` el awel), el gold tables (`attacker_profiles`, `attack_timeline`, `ml_scores`) *ma-mawjoudinch* 3la disk 9ad. Hedha *mahouch bug* walla "erreur" mte3 el API – houwa expected state 3nd el deployment fresh. Return `None` (w routers ye-convert-ha l empty response, mathalan `AlertsResponse(total=0, items=[])`) ye5alli el frontend (dashboard) ye-3red "no data yet" gracefully, machi crash walla HTTP 500 confusing. HTTP 500 lezem yeb9a reserved l bugs 7a9i9iyin (unexpected exceptions), machi l état normal predictible.

8.3.1 _ALERT_SILVER_COLUMNS w ALERT_COLUMNS

```python
_ALERT_SILVER_COLUMNS = (
    "event_id", "event_time", "src_ip", "dst_ip", "dst_port",
    "source_type", "severity", "attack_category",
)

ALERT_COLUMNS = (
    "event_id", "scored_at", "alert_source", "anomaly_score", "is_anomaly",
    "event_time", "src_ip", "dst_ip", "dst_port", "source_type",
    "severity", "attack_category",
)
```

- `_ALERT_SILVER_COLUMNS`: el columns li ye3ayet-lhom mel silver table (context 3la el attack – IP, port, severity...) li `ml_scores` nafsou ma-3andou-ch (houwa 3andou bark `event_id`, `scored_at`, `anomaly_score`, `is_anomaly`, `alert_source`).
- `ALERT_COLUMNS`: el projection el kamla el akhira – comment ye9oul *"matching threatlake.api.schemas.AlertItem field-for-field"* – exactement el fields li `AlertItem` Pydantic model ye7taj-hom (chefna `schemas.py` nchoufouha ba3d). `ALERT_COLUMNS` *public* (f `__all__`) 3la 5atr mous-ta3mla mel 2 routers (`alerts.py` w `attacker_profiles.py`'s drilldown) – single source of truth l el projection, bech el 2 endpoints ye5serializ el alert row *identical* (nafs el fields, nafs el ordre).

8.3.2 read_gold_table w read_silver

```python
def read_gold_table(spark: SparkSession, table: str, settings: Settings) -> DataFrame | None:
    """Read a gold Delta table, or None if it hasn't been built yet."""
    try:
        return spark.read.format("delta").load(gold_path(table, settings))
    except Exception:
        return None


def read_silver(spark: SparkSession, settings: Settings) -> DataFrame | None:
    """Read the silver events table, or None if nothing has landed yet."""
    try:
        return spark.read.format("delta").load(silver_path("events", settings))
    except Exception:
        return None
```

Nafs el pattern `try/except Exception: return None` li chefnah f `score_events.py`'s `_already_scored_even` w `prompts.py`'s `_describe_table` – table mahouch mawjoud 3la disk (Delta load ye-fail) houwa treated kif `None`, machi propagated exception.

8.3.3 read_alerts – el join el asasi

```python
def read_alerts(spark: SparkSession, settings: Settings) -> DataFrame | None:
    """``ml_scores`` joined with silver, filtered to rows either detector
    actually flagged.

    ``is_anomaly`` is a NOT NULL column in
    ``threatlake.ml.score_events.SCORES_SCHEMA`` - so this can filter on
    it directly rather than needing a null-coalescing OR.
    """
    scores = read_gold_table(spark, "ml_scores", settings)
    if scores is None:
        return None
    silver = read_silver(spark, settings)
    if silver is None:
        return None

    joined = scores.join(silver.select(*_ALERT_SILVER_COLUMNS), on="event_id", how="inner")
    return joined.filter(F.col("is_anomaly"))
```

- Zouz early-returns séquentiels (`if scores is None: return None`, ba3d `if silver is None: return None`) – ken 7atta wa7ed mel 2 tables mahouch mawjoud, el function el kamla terja3 `None` (mahouch alerts 9ad, walaw el table el o5ra mawjouda).
- `scores.join(silver.select(*_ALERT_SILVER_COLUMNS), on="event_id", how="inner")`: INNER join (machi LEFT) – 3la 5atr `event_id` lezem ykoun f el zouz tables bech el row ykoun useful (score bidoun context mel silver, wela context bidoun score, mahomch useful l "alert" concept).
- `.filter(F.col("is_anomaly"))`: hedha el filter el akhir – `is_anomaly` boolean NOT NULL (`SCORES_SCHEMA`), fa `.filter(F.col("is_anomaly"))` houwa equivalent l `.filter(F.col("is_anomaly")`

`== True`) (Spark ye9bel boolean column direct kif filter condition). Docstring ye-explique el "why" el subtile: *"is_ anomaly is a NOT NULL column ... so this can filter on it directly rather than needing a null-coalescing OR"* – schema PFA (ml-only, bidoun classifier supplémentaire) ye-guarantee `is_anomaly` dima True/False, contra ThreatLake AI li 3andha detector ekstra (classifier) li momken y5alli el column null 3la certain rows, fa el filter hnak lezmou `OR` logic akthar complexe.

8.4 api/schemas.py

```
Pydantic response models for every API route.

Hand-written, not auto-derived from the Delta schemas: an API response is
a deliberately curated, presentation-shaped view - keeping it separate
from the storage layer means a storage-schema change doesn't silently
change the API contract, and vice versa.
```

Soual (Jury)

3lech el Pydantic models hand-written (rewrite manuel mte3 el fields), mouch auto-générés mel Delta schemas (`SILVER_EVENT_SCHEMA`, `SCORES_SCHEMA`...)? Ma-houch duplication (nafs el fields mektoubin marat 3adda)?
Réponse: docstring ye9olha directement – *"an API response is a deliberately curated, presentation-shaped view"*. El storage schema (`SILVER_EVENT_SCHEMA` mathalan) ye-optimize l storage/query concerns (types Delta/Spark, nullability constraints, partitioning...). El API response ye-optimize l consumer concerns (chnowa el frontend lezmou ya3raf, chnowa format ahsen l JSON serialization, chnowa fields lezmou t5abbaw – mathalan `geo` struct el kbir f gold table ye-flatten l `latitude`/`longitude` bark f API, machi el struct kamel). Duplication (nafs esm el field mektoub zouz marat) houwa trade-off mou9asad: separation ye5alli *storage-schema change ma-y-changich API contract silently* (tzid column jdid f silver, API response ma-tetbeddel-ch automatiquement – lezmek explicitement tzid-ha f Pydantic model ken 7bit t-esta3ml-ha), w el 3ks zeda (tbeddel API response shape ma-yehtajch tbeddel storage schema). El 2 evolve independamment.

8.4.1 AlertItem w AlertsResponse

```
class AlertItem(BaseModel):
    event_id: str
    scored_at: datetime
    alert_source: str | None
    anomaly_score: float | None
    is_anomaly: bool
    event_time: datetime | None
    src_ip: str | None
    dst_ip: str | None
    dst_port: int | None
    source_type: str | None
    severity: int | None
    attack_category: str | None


class AlertsResponse(BaseModel):
    total: int
    limit: int
    offset: int
    items: list[AlertItem]
```

- `AlertItem`: field-for-field match m3a `ALERT_COLUMNS` (`_tables.py`), b nullability correspondante l el schema el 7a9i9i (`event_id`/`is_anomaly` required, el ba9i optional – match `SCORES_SCHEMA` w `SILVER_EVENT_SCHEMA`'s nullable fields).

- `AlertsResponse`: `total`, `limit`, `offset` + `items` – pagination envelope el standard: `total` (kaddech row mawjoud kamlin, machi bark f had el page), `limit`/`offset` (echo mte3 el request params, useful l el frontend bech ya3raf win rahou f el pagination), `items` (el page el 7a9i9iya).

8.4.2 CredentialAttempt w AttackerProfile

```python
class CredentialAttempt(BaseModel):
    username: str | None
    password: str | None
    count: int


class AttackerProfile(BaseModel):
    src_ip: str
    first_seen: datetime | None
    last_seen: datetime | None
    total_events: int
    distinct_ports_hit: int
    distinct_honeypots_hit: int
    top_credentials_tried: list[CredentialAttempt]
    attack_categories: list[str]
    # Flattened out of the gold table's richer 'geo' struct - country/city/
    # ASN aren't surfaced here because nothing in this API currently uses
    # them; lat/lon are, for the dashboard's Map tab. Both null when the
    # IP didn't resolve (private/reserved range, or a MaxMind free-tier
    # coverage gap - both normal, not errors).
    latitude: float | None
    longitude: float | None
```

Soual (Jury)

Docstring ye9oul `latitude`/`longitude` bark ye-flatten mel `geo` struct (li 3andou 6 fields: country, city, latitude, longitude, asn, asn_org). 3lech bark 2 mel 6, w chnowa ye-sir l el 4 el o5rin?

Réponse: mel comment inline – *"country/city/ASN aren't surfaced here because nothing in this API currently uses them; lat/lon are, for the dashboard's Map tab"*. Hedhi el falsafa "presentation-shaped view" (el Soual el fou9) f action concrete: el gold table (`attacker_profiles`) 3andha kol el 6 fields (data complet, computed once via GeoEnricher). API response ye-expose bark el subset li *consumer 7a9i9i* (dashboard's Map tab) lezmou. El 4 fields el o5rin *mahomch mfoutin* – homa mawjoudin f el gold table nafsou (accessible via copilot query mathalan, walla direct query 3la Delta table), just *mahomch f had el API response el specifique* 3la 5atr ma-fama-ch consumer 7ali ye7taj-hom. Ken feature jdid (mathalan "show country flag") ye7taj `country`, tzid-ha l `AttackerProfile` w l `_select_profile_columns` f el router – one-line change, machi redesign.

8.4.3 AttackerProfilesResponse w AttackerDrilldown

```python
class AttackerProfilesResponse(BaseModel):
    total: int
    limit: int
    offset: int
    items: list[AttackerProfile]


class AttackerDrilldown(BaseModel):
    profile: AttackerProfile
    alerts: list[AlertItem]
```

`AttackerDrilldown`: nafs el fikra kif `AttackerProfilesResponse` walakin l endpoint `/ip` – ya3ayet zouz "concepts" mte3 el API sawa (`AttackerProfile` + `list[AlertItem]`) f response wa7ed, bech analyst ya5ou el profile *w* el alerts mte3 had el IP f call wa7ed, machi zouz calls separate.

8.4.4 CopilotQueryRequest w CopilotQueryResponse

```python
class CopilotQueryRequest(BaseModel):
    question: str


class CopilotQueryResponse(BaseModel):
    """Exactly one of two shapes, discriminated by ``rejected``: success ->
    sql/rows/row_count populated, reason null; rejected/failed -> reason
    populated (always specific), rows/row_count null. ``sql`` may still be
    present on a guardrail rejection (the query that got rejected) but is
    null if generation itself never produced one.
    """

    rejected: bool
    reason: str | None
    sql: str | None
    rows: list[dict[str, object]] | None
    row_count: int | None
```

Soual (Jury)

3lech `CopilotQueryResponse` houwa "one model, 2 shapes" (discriminated by `rejected` boolean, kol el fields optional 3da `rejected` nafsou), mouch zouz Pydantic models separate (mathalan `Union[CopilotSuccessResponse, CopilotErrorResponse]`)?

Réponse: mahiche documented explicitement f el code, walakin el trade-off houwa clair mel shape nafsou: model wa7ed (kifma docstring te-explique b "exactly one of two shapes, discriminated by rejected") houwa simple API contract l el frontend – `response_model=CopilotQueryResponse` dima, el frontend ye-check `if response.rejected:` w ye-access el fields el correctes. B `Union[...]`, FastAPI's OpenAPI schema generation ye-b9a akthar complexe (discriminated unions), w el frontend lezmou type-narrowing logic akthar sophistiquée. Trade-off: model wa7ed ye5sser type-safety chwaya (kol field optional, 7atta ken semantically "always present on success") mou9abel simplicité API/client-side.

8.5 api/routers/alerts.py

```
GET /alerts (paginated, filterable).

See threatlake.api._tables.read_alerts for what counts as an "alert"
here. PFA is batch-only, so there is no live push endpoint here: results
are as fresh as the last pipeline run, not real-time.
```

8.5.1 _DEFAULT_LIMIT, _MAX_LIMIT, w list_alerts

```python
_DEFAULT_LIMIT = 50
_MAX_LIMIT = 500


@router.get("/alerts", response_model=AlertsResponse)
def list_alerts(
    alert_source: str | None = Query(
        default=None, description="Filter to 'rule', 'ml', or 'both'."
    ),
    severity: int | None = Query(default=None, ge=1, le=5),
    limit: int = Query(default=_DEFAULT_LIMIT, ge=1, le=_MAX_LIMIT),
    offset: int = Query(default=0, ge=0),
    spark: SparkSession = Depends(get_spark_dep),
    settings: Settings = Depends(get_settings_dep),
) -> AlertsResponse:
    alerts = read_alerts(spark, settings)
    if alerts is None:
```

```
        return AlertsResponse(total=0, limit=limit, offset=offset, items=[])

    if alert_source is not None:
        alerts = alerts.filter(F.col("alert_source") == alert_source)
    if severity is not None:
        alerts = alerts.filter(F.col("severity") == severity)

    alerts = alerts.orderBy(F.col("scored_at").desc())
    total = alerts.count()
    page = alerts.offset(offset).limit(limit).collect()
    items = [AlertItem(**{c: row[c] for c in ALERT_COLUMNS}) for row in page]
    return AlertsResponse(total=total, limit=limit, offset=offset, items=items)
```

- Kol query parameter mou3arraf b `Query(...)` – FastAPI's way l declare query string params b validation (`ge=1, le=5` l `severity` mathalan – FastAPI ye-reject el request automatiquement b HTTP 422 ken el user ye7ott `severity=99`, bidoun ma tekteb el validation manuellement f el function body). `description=...` ye-populate el Swagger/OpenAPI docs automatiquement.
- `_DEFAULT_LIMIT = 50`, `_MAX_LIMIT = 500`: nafs el falsafa kif `MAX_ROWS` f el copilot guardrail – server-side cap 3la kaddech data ye5rej f response wa7da, machi trust el client ye-request kaddech y7eb.
- `if alerts is None: return AlertsResponse(total=0, ..., items=[])`: exactement el "empty response, never 500" el docstring mte3 `_tables.py` te-describe.
- `if alert_source is not None: alerts = alerts.filter(...)`: filters conditionnels – bark ye-apply-hom ken el query param mawjoud (`None` check), machi filter dima 3la `None` (li ye5dem differently f SQL/Spark, `col == None` mahouch nafs `col IS NULL`).
- `total = alerts.count()` *9bal* `.offset(offset).limit(limit).collect()`: el count houwa 3la el DataFrame el kamel (ba3d filters, 9bal pagination) – bech `total` f el response ye-represente kaddech row mawjoud kamlin (l el frontend bech ya3raf kaddech pages fama), machi bark kaddech f had el page.
- `.offset(offset).limit(limit)`: Spark's pagination – 3la DataFrame ordoné 9bal, `.offset()` ye-skip el rows el awlanin, `.limit()` ye5ou el N el jayya.
- `[AlertItem(**{c: row[c] for c in ALERT_COLUMNS}) for row in page]`: dict comprehension jowa `AlertItem(**...)` – ye5ou kol column mel `ALERT_COLUMNS` tuple mel Spark `Row` object (`row[c]`), yebni dict, w `**` unpacking ye3addi-ha ki keyword arguments l `AlertItem` constructor (Pydantic validation automatique 3la kol field f had el moment).

8.6 api/routers/attacker_profiles.py

```
GET /attacker_profiles (paginated) and GET /attacker_profiles/{ip} (drill-down).

Both read threatlake.transform.gold.attacker_profiles. The /{ip}
drill-down additionally pulls this IP's alerts (read_alerts) so an
analyst gets the profile and its flagged activity in one call instead of
two.
```

8.6.1 _PROFILE_COLUMNS w _select_profile_columns

```
_PROFILE_COLUMNS = (
    "src_ip", "first_seen", "last_seen", "total_events",
    "distinct_ports_hit", "distinct_honeypots_hit",
    "top_credentials_tried", "attack_categories",
)


```

```python
def _select_profile_columns(df: DataFrame) -> DataFrame:
    """_PROFILE_COLUMNS plus lat/lon, flattened out of the gold table's
    'geo' struct - see threatlake.api.schemas.AttackerProfile for why only
    those two geo fields are surfaced here.
    """
    return df.select(
        *_PROFILE_COLUMNS,
        F.col("geo.latitude").alias("latitude"),
        F.col("geo.longitude").alias("longitude"),
    )
```

`F.col("geo.latitude")`: dot notation 3la Spark column l access *nested struct field* – `geo` houwa struct column (kifma chefnah f `build_attacker_profiles`), w `.alias("latitude")` ye-flatten-ha l column top-level, machi struct nested, f el DataFrame el akhir – exactement el "flattening" el discuté f `schemas.py`'s Soual box.

8.6.2 _profile_from_row

```python
def _profile_from_row(row: Row) -> AttackerProfile:
    data: dict[str, Any] = row.asDict(recursive=True)
    return AttackerProfile(**data)
```

`row.asDict(recursive=True)`: `recursive=True` muhim hna – `top_credentials_tried` houwa array mte3 structs (`CredentialAttempt` shape). Bidoun `recursive=True`, `.asDict()` ye-convert bark el level el awel (top-level fields l dict values), w el nested structs/arrays ye-b9aou Spark Row objects (mahomch JSON-serializable direct, w Pydantic ma-y9bel-hom-ch direct). B `recursive=True`, kol nested Row (jowa arrays zeda) ye-convert l dict/list recursively, fa `AttackerProfile(**data)` ye9dar ye-validate el structure el kamla (`list[CredentialAttempt]` mel dict el nested).

8.6.3 list_attacker_profiles

```python
@router.get("/attacker_profiles", response_model=AttackerProfilesResponse)
def list_attacker_profiles(
    limit: int = Query(default=_DEFAULT_LIMIT, ge=1, le=_MAX_LIMIT),
    offset: int = Query(default=0, ge=0),
    spark: SparkSession = Depends(get_spark_dep),
    settings: Settings = Depends(get_settings_dep),
) -> AttackerProfilesResponse:
    profiles = read_gold_table(spark, "attacker_profiles", settings)
    if profiles is None:
        return AttackerProfilesResponse(total=0, limit=limit, offset=offset, items=[])

    profiles = _select_profile_columns(profiles).orderBy(F.col("total_events").desc())
    total = profiles.count()
    page = profiles.offset(offset).limit(limit).collect()
    return AttackerProfilesResponse(
        total=total, limit=limit, offset=offset, items=[_profile_from_row(r) for r in page]
    )
```

Nafs pattern exact kif `list_alerts` – ordoné b `total_events desc` (attackers el akthar actifs el awlanin, default sensible l analyst SOC).

8.6.4 get_attacker_profile – el drilldown

```python
@router.get("/attacker_profiles/{ip}", response_model=AttackerDrilldown)
def get_attacker_profile(
    ip: str,
    spark: SparkSession = Depends(get_spark_dep),
    settings: Settings = Depends(get_settings_dep),
) -> AttackerDrilldown:
    profiles = read_gold_table(spark, "attacker_profiles", settings)
    if profiles is None:
        raise HTTPException(status_code=404, detail=f"No attacker profile data for {ip!r} yet.")

```

```
    match = _select_profile_columns(profiles).filter(F.col("src_ip") == ip).collect()
    if not match:
        raise HTTPException(status_code=404, detail=f"No attacker profile for {ip!r}.")
    profile = _profile_from_row(match[0])

    alerts_df = read_alerts(spark, settings)
    alerts: list[AlertItem] = []
    if alerts_df is not None:
        rows = (
            alerts_df.filter(F.col("src_ip") == ip)
            .select(*ALERT_COLUMNS)
            .orderBy(F.col("scored_at").desc())
            .collect()
        )
        alerts = [AlertItem(**{c: row[c] for c in ALERT_COLUMNS}) for row in rows]

    return AttackerDrilldown(profile=profile, alerts=alerts)
```

Soual (Jury)

Fama zouz `HTTPException(status_code=404)` calls mo5tlifin f had el function (`"data for {ip!r} yet"` vs `"profile for {ip!r}"`) – 3lech el difference houwa muhimma, machi bark style?
Réponse: el 2 messages ye-distinguer bin zouz cas 404 mo5tlifin semantiquement. El awel (`profiles is None`): *el table kamla* mahiche mawjouda (`run_pipeline.py` ma-tshaghalch 3ad) – ma-3andiche 7atta data l ay IP. El theni (`not match`): el table mawjouda, fiha data, walakin had el IP el specifique *mahouch fiha* (attacker mahouch scanné el honeypot 9att). Zouz cas mo5tlifin l el analyst li ye-debug-i: el awel ye9oul "el pipeline ma-tshaghalch 3ad" (action: run el pipeline), el theni ye9oul "el pipeline yeshtaghel, walakin had el IP mahouch attacker ma3rouf" (action: verify el IP, machi bug). Messages distincts (walaw kolhom 404) ye5alliw el troubleshooting a9rab.

- `match = ....collect()`: `.collect()` 3la DataFrame filtré (bark row wa7da mtawa9a3a, 3la 5atr `src_ip` houwa el key mte3 el grain) – Python list, checked b `if not match`.
- `alerts_df = read_alerts(spark, settings)` + `if alerts_df is not None:`: had el partie *optional* – ken `ml_scores` walla silver mahomch mawjoudin (pipeline na9is), el drilldown ye-b9a ye5dem (profile mawjoud), just `alerts` ye-b9a empty list (machi 404 3la had el section bark).
- `AttackerDrilldown(profile=profile, alerts=alerts)`: el combination el akhira – el response wa7ed ye-includi el 2 "concepts" (profile + alerts) f call HTTP wa7ed.

8.7 api/routers/copilot.py – el 3-step flow

```
Three strictly separate steps, each of which can independently fail
without touching the next:
  1. threatlake.copilot.text_to_sql.generate_sql - ask the LLM for SQL.
     Never trusted.
  2. threatlake.copilot.guardrails.validate_and_bound - the ONLY thing
     that decides whether that SQL is allowed to run. Runs BEFORE any
     Spark call, on every request, no exceptions.
  3. Execute the validated SQL against temp views over the gold tables
     only - not the live gold Delta tables directly, so nothing this
     endpoint does can ever touch bronze, silver, or ml_scores even if
     steps 1-2 had a bug.

Every rejection - config, provider, guardrail, or execution failure - is
logged with its specific reason and returned to the caller the same way,
never a bare 500: this is a read-only analyst tool, a failure here should
never look like a crash.
```

Soual (Jury)

Hedha el endpoint el akthar critique securité-wise f el codebase kamel – comment (step 3) ye9oul "even if steps 1-2 had a bug" el temp views houma defense supplémentaire. 3lech had el "triple layer" (LLM prompt + guardrail + temp views scoped) mahouch redundant/overkill?
Réponse: hedha exemple concret mte3 "defense-in-depth" (lćoncept li et9echnah barcha marat f had el document): kol layer ye-address *failure mode mo5tlef*. Step 1 (LLM prompt, mel `build_system_prompt`) houwa *guidance*, ma-lezmech y-esta3mel mel LLM abadan (hallucination, prompt injection). Step 2 (guardrail AST parsing) houwa el security boundary el 7a9i9i – deterministe, testable, ma-y3tamedch 3la el LLM. Step 3 (temp views scoped 3la `ALLOWED_TABLES` bark) houwa layer *structurel* ekstra – *7atta ken* el guardrail (step 2) 3andou bug w y5alli SQL dangereux ye3addi (edge case ma-testeh-ch), el SQL el akhir ye-execute 3la `spark.sql(safe_sql)` *f context win bronze/silver/ml_scores ma-3andhomch-ch temp view registered 7atta*. Ye3ni: 7atta ken el SQL ye7awel `SELECT * FROM bronze_cowrie`, Spark ye-throw "table or view not found" – 3la 5atr el temp view l `bronze_cowrie` ma-tsawwaret-ch abadan f had el session. Had el layer el akhir ma ye3tamedch 3la "el guardrail logic sa7i7a" – houwa *structural impossibility*, machi behavioral check.

8.7.1 _rejected

```python
def _rejected(reason: str, sql: str | None = None) -> CopilotQueryResponse:
    return CopilotQueryResponse(rejected=True, reason=reason, sql=sql, rows=None, row_count=None)
```

Helper sghir – constructor shortcut l el "rejected shape" mte3 `CopilotQueryResponse` (`rejected=True`, `rows=None`, `row_count=None` dima), bech el function el kbira (`copilot_query`) ma tekteb-ch had el 5 fields kol marra manuellement f kol exception handler.

8.7.2 copilot_query – Step 1: generation

```python
@router.post("/copilot/query", response_model=CopilotQueryResponse)
def copilot_query(
    request: CopilotQueryRequest,
    spark: SparkSession = Depends(get_spark_dep),
    settings: Settings = Depends(get_settings_dep),
) -> CopilotQueryResponse:
    try:
        raw_sql = generate_sql(request.question, spark, settings)
    except SchemaUnavailableError as exc:
        _LOG.warning("copilot: gold schema unavailable for question=%r: %s", request.question, exc)
        return _rejected(str(exc))
    except CopilotConfigError as exc:
        _LOG.warning("copilot: not configured: %s", exc)
        return _rejected(str(exc))
    except CopilotProviderError as exc:
        _LOG.warning("copilot: provider error for question=%r: %s", request.question, exc)
        return _rejected(f"the language model call failed: {exc}")
```

- 3 `except` clauses séparés, kol wa7ed l exception type specifique (`SchemaUnavailableError`, `CopilotConfigError`, `CopilotProviderError` – kolhom chefnahom f `copilot/text_to_sql.py` w `copilot/prompts.py`). Kol wa7ed ye-log-i b message specifique (context mo5tlef l kol cas), walakin kolhom yerja3ou `_rejected(...)` – consistent response shape l el caller (HTTP 200 dima, mahouch 500), b `reason` specifique l kol cas.
- Zero `except Exception:` generic hna – ken exception mahouch mel 3 types mou3arrfin, ye-propagate (crash 7a9i9i, machi silently caught) – houwa distinguer bin "expected failure modes" (config, schema, provider – el 3 exceptions) w "unexpected bug" (ay chay o5or, li lezem ykoun visible, machi masked).

8.7.3 Step 2: guardrail

```
    try:
        safe_sql = validate_and_bound(raw_sql)
    except GuardrailRejectionError as exc:
        _LOG.warning(
            "copilot: guardrail REJECTED question=%r generated_sql=%r reason=%s",
            request.question,
            raw_sql,
            exc.reason,
        )
        return _rejected(exc.reason, sql=raw_sql)
```

`return _rejected(exc.reason, sql=raw_sql)`: mula7id – `sql=raw_sql` mawjoud hna (contra el 3 rejections el fou9, li `sql` mte3hom `None` 3la 5atr generation nafsou fashla, ma-famach SQL 7atta). Hedhi el "sql may still be present on a guardrail rejection" el mentioned f `CopilotQueryResponse`'s docstring – caller (analyst, walla developer li ye-debug) ye9dar ye-chouf *el SQL el exact li rejected* w *3lech*.

8.7.4 Step 3: temp views w execution

```
    for table in ALLOWED_TABLES:
        try:
            spark.read.format("delta").load(gold_path(table, settings)).createOrReplaceTempView(
                table
            )
        except Exception as exc:
            _LOG.warning("copilot: gold table %r unavailable: %s", table, exc)
            return _rejected(
                f"gold table {table!r} isn't available yet - the gold scheduler needs to "
                "complete at least one run before the copilot can answer questions"
            )

    try:
        result_df = spark.sql(safe_sql)
        rows = [row.asDict(recursive=True) for row in result_df.collect()]
    except Exception as exc:
        _LOG.warning("copilot: execution failed sql=%r error=%s", safe_sql, exc)
        return _rejected(f"the query failed to execute: {exc}", sql=safe_sql)

    _LOG.info("copilot: question=%r sql=%r rows=%d", request.question, safe_sql, len(rows))
    return CopilotQueryResponse(
        rejected=False, reason=None, sql=safe_sql, rows=rows, row_count=len(rows)
    )
```

- `for table in ALLOWED_TABLES: ... createOrReplaceTempView(table)`: el loop ye-register temp view l *kol* table f `ALLOWED_TABLES` (el 2 f PFA), machi bark el tables li `safe_sql` ye-reference-hom – simplicité (ma-lezmech t-parse el SQL besh ta3raf chnowa tables mous-ta3mlin, temp views cheap l register).
- `spark.sql(safe_sql)`: `safe_sql` houwa el SQL el rewritten (LIMIT bounded) mel guardrail, ye-execute f context win bark el temp views registered mawjoudin (bronze/silver/ml_scores *ma-3andhomch-ch* temp view – exactement el "structural impossibility" mentioned f el Soual box el fou9).
- `rows = [row.asDict(recursive=True) for row in result_df.collect()]`: nafs pattern kif `_profile_from_row` – `recursive=True` bech nested structures (result d'un query complexe, ken fama) ye-convert-iw l dicts JSON-serializable.
- `_LOG.info(...)` f el a5er (success case): logging dima – sawa success sawa rejection, kol chay mel copilot flow ye-log-i, useful l audit trail w debugging.

Chapter 9

Tableau récapitulatif – "win nal9a chnowa"

Su2al mte3 jury	File	Function/Class
Kifeh el config ye5dem (YAML + env vars)?	common/config.py	Settings, get_settings
Kifeh path mte3 table ye-construct?	common/paths.py	layer_path, bronze_path etc
Kifeh ta3raf Spark ye5dem 3la UTC 7a9i9i?	common/spark.py	_verify_utc_timezone, _check_utc_timezone
Kifeh files ye-move mel landing l processed?	common/fs.py	move
Fein el schema mte3 cowrie?	ingestion/schemas.py	source_schema
Kifeh malformed JSON ye-quarantine?	ingestion/bronze_transform.py	split_quarantine
Kifeh ingestion houwa at-least-once?	ingestion/bronze_writer.py	write_bronze
Chnowa el 18 fields mte3 silver?	transform/silver/schema.py	SILVER_EVENT_SCHEMA
Kifeh cowrie events ye-map l attack_category/severity?	transform/silver/cowrie.py	_CATEGORY_SEVERITY, map_cowrie
Kifeh gold tables ye-overwrite idempotently?	transform/gold/writer.py	write_gold_table
Kifeh top-5 credentials par IP ye7sbou?	transform/gold/attacker_profiles.py	top_credentials_by_ip
Kifeh geo struct ye5dem b/bidoun enricher?	transform/gold/attacker_profiles.py	build_attacker_profiles
Kifeh timeline el hourly ye7sab?	transform/gold/attack_timeline.py	build_attack_timeline
Kifeh trailing-window features ye7sbou?	ml/features.py	build_features
3lech threshold 5 l port scan?	ml/rules.py	port_scan_rule
Kifeh IsolationForest ye-train?	ml/train_anomaly.py	train_anomaly
Kifeh ML w rule ye-combine f alert_source?	ml/score_events.py	score_silver_events
Kifeh SQL ye-valid AST-based?	copilot/guardrails.py	validate_and_bound
Chnowa el ALLOWED_TABLES trimmed?	copilot/guardrails.py	ALLOWED_TABLES

Su2al mte3 jury	File	Function/Class
Kifeh Gemini ye3ayet-lha securement?	copilot/text_to_sql.py	_call_gemini, generate_sql
Kifeh el prompt ye-build b live schema?	copilot/prompts.py	build_system_prompt
Kifeh table not-found ye-handle f API?	api/_tables.py	read_gold_table
Kifeh alerts ye-join mel ml_scores+silver?	api/_tables.py	read_alerts
Chnowa el 3 endpoints?	api/routers/*.py	list_alerts, list_attacker_profiles, copilot_query
Kifeh copilot query ye5dem end-to-end (3 steps)?	api/routers/copilot.py	copilot_query
Kifeh tests ye5edmou b isolated app instance?	api/app.py	create_app